

IVC in the Open-and-sign Random Oracle Model

Mary Maller^{1,2}, Nicolas Mohnblatt³, and Aranxta Zapico¹

¹ Ethereum Foundation

² PQShield

³ Bain Capital Crypto

Abstract. Incrementally verifiable computation (IVC) is a powerful cryptographic primitive, particularly suited for proving long-running machine computations. Previous work shows that IVC can be constructed by recursively composing SNARKs. Unfortunately, theoretical challenges limit the provable security of known IVC constructions.

Recursive composition may quickly lead to a blowup in extraction time and may require arithmetic circuits to enforce constraints about random oracle calls. Furthermore, composition presents a practical challenge: proofs are often expressed in a form that is not friendly to the arithmetic circuits that produce them.

To mitigate the theoretical challenges, we present the Open-and-Sign Random Oracle Model (osROM) as an extension to the signed random oracle of Chiesa and Tromer (ICS ‘10). This model, while strictly harder to instantiate than the Random Oracle Model, allows the design of protocols that can efficiently verify calls to the oracle and support straight-line extractors. As a result, IVC constructions in the osROM can be shown to have provable security for polynomial depths of computation.

Under our new model, we construct a framework to build secure IVC schemes from simple non-interactive reductions of knowledge. Our construction natively supports cycles of elliptic curves in the style of Ben-Sasson *et al.* (CRYPTO ‘14), thus answering the practical challenge outlined above. Finally, we analyze the HyperNova (CRYPTO ‘24) IVC scheme in the osROM and show that it is secure over a two-cycle of elliptic curves, for polynomial depths of computation.

Keywords: Incrementally-verifiable computation · Reductions of knowledge · Elliptic curves

1 Introduction

Incrementally verifiable computation (IVC) and its generalization, proof-carrying data (PCD), are advanced cryptographic techniques particularly suited for verifying the correct execution of long-running machine computations [39]. In an IVC scheme, a prover and verifier are given a function F and values z_0, z_i as inputs. The IVC prover produces proofs π_i that attest to statements of the form $z_i = F^i(z_0)$. Importantly, proof size does not depend on the number of computation steps and each proof can be generated *incrementally*; that is, given a proof π_i , the prover can generate the proof π_{i+1} without recomputing the previous i steps. Compared to a monolithic prover, incremental proofs allow proving a machine computation without storing the entire computation trace in memory. As a result, IVC has gained traction in zero-knowledge virtual machines (zkVMs) and several have adopted IVC for its scalability [1, 32].

The first realization of IVC required the recursive composition of succinct non-interactive arguments of knowledge (SNARKs) for circuit satisfiability [39]. Later works [8, 9, 12, 27] show that IVC can be realized by combining weaker primitives, namely a non-interactive argument of knowledge (NARK) for circuit satisfiability with a non-interactive folding scheme (NIFS; also known as an accumulation scheme). A folding scheme is a protocol between a prover and a verifier, where the former takes as input two instance-witness pairs $(\phi_1, w_1), (\phi_2, w_2)$ in one relation $\mathcal{R}$, and outputs a new pair (ϕ, w) . The verifier takes as input ϕ_1, ϕ_2 and outputs ϕ . The folding scheme guarantees that if the prover knows a witness w such that $(\phi, w) \in \mathcal{R}$, then the prover also knows w_1, w_2 such that $(\phi_1, w_1) \in \mathcal{R}$ and $(\phi_2, w_2) \in \mathcal{R}$. These interactive protocols are then compiled into NIFSs in the random oracle model using the Fiat-Shamir transform. Most folding schemes require additively homomorphic commitment schemes and are implemented using elliptic curves over finite fields [10, 11, 22, 27, 28, 33]. Note however that a recent body of work concurrent to this one explores building folding schemes in the pure ROM [13, 14, 15] or from lattices [6, 7, 35].

Theoretical and practical challenges. All the curve-based realizations of IVC above suffer from some of the theoretical and practical challenges listed below:

1. *Relativizing arguments.* One of the key ideas in all folding-based IVC constructions is to use a NARK to prove statements about the execution of the step function F and the correct execution of a NIFS verifier. As noted above, known NIFS are described as random oracle protocols. Therefore, proving a statement about a NIFS verifier requires proving calls to the random oracle, contradicting the fact that succinct arguments in the ROM that prove statements about the random oracle do not exist [2], or heuristically instantiating the oracle, thus losing proven security.
2. *Constant-depth IVC.* Composing arguments of knowledge may lead to protocols for which we do not know how to construct polynomial-time extractors. As noted by Bitansky *et al.* [5], this limits the depth of composition (and therefore the number of IVC steps) to a constant. However, if the argument of knowledge admits a straight-line extractor, then composition may be performed securely for polynomial depth.
3. *Closing the loop.* As first noted by Ben-Sasson *et al.* [4], it is hard to recursively compose protocols that are instantiated with elliptic-curve-based commitments without incurring a large arithmetization cost. Their solution is to use a 2-cycle of elliptic curves: a pair of curves E_1 and E_2 where E_1 is of prime order p and defined over a field $\mathbb{F}_q$, and E_2 is of prime order q and defined over a field $\mathbb{F}_p$. However, lifting IVC constructions from the single-curve setting to the 2-cycle setting is highly non-trivial and has led to devastating soundness bugs in real-world implementations [34].

Our contributions. In response to the above challenge, we make the following contributions:

- we introduce the open-and-sign random oracle model (**osROM**) as variant of the signed ROM of Chiesa and Tromer [20]. While our model does not solve the above issues, it provides a framework in which protocols can be proven secure while deferring the challenges to the instantiation of the oracle. Given a security analysis in the **osROM**, practitioners can instantiate their protocols in a weaker model of their choice by providing a complete or heuristic instantiation of the **osROM**.
- we describe the first compiler that constructs a secure IVC scheme over a 2-cycle of elliptic curves from simple non-interactive reductions of knowledge (NIRoKs), thus addressing Challenge 3. A NIRoK is the non-interactive equivalent to the algebraic reductions of knowledge defined by Kothapalli and Parno [26].
- we apply our compiler to the NIRoKs that compose HyperNova [28] to show that HyperNova admits a straight-line extractor in the **osROM** and therefore IVC with polynomial depth.

Organization of the paper. The remaining sections of this introduction give a technical overview of our key ideas and compare our techniques to related work. In Section 2, we give some necessary background and notation. In Section 3, we define the **osROM** and give definitions for non-interactive reductions of knowledge and IVC in the **osROM**. In Section 4, we illustrate the properties of our model by analyzing HyperNova as a sequence of NIRoKs in the **osROM**. In Section 5 and Section 6, we construct the IVC compiler, first over a single curve and then over a two-cycle of curves.

1.1 Technical Overview

The osROM. The **osROM** is an extension of Chiesa and Tromer’s signed random oracle model (SROM) [20] that explicitly supports elliptic curve commitments. We first give a high-level description of the SROM and then explain how the **osROM** extends it.

Background. The SROM assumes the existence of a random oracle that is also able to issue digital signatures. The oracle samples a signing key and outputs its verification key to all parties. When a prover queries the oracle on a message m , the oracle replies with the tuple (α, σ) , where α is a standard random oracle response for m and σ is a signature over the pair (m, α) . Any third party can subsequently verify that the pair (m, α) is a valid random oracle query-response by verifying the signature σ . The motivation for this model is that arguments in the SROM can be relativized: the verifier does not make calls to oracles and can be described as an arithmetic circuit.

Overview. The **osROM** extends the SROM to also capture the assumption that elliptic curve commitments are extractable. Fix a group $\mathbb{G}$ of prime order p , a polynomial-size bound $s \in \mathbb{N}$ and a list of randomly sampled generators $\vec{g} \in \mathbb{G}^s$. A query to the **osROM** now has the form $([A_i, \vec{a}_i]_{i=1}^\ell, \mathbf{aux})$, where each $A_i \in \mathbb{G}$, each $\vec{a}_i \in (\mathbb{F}_p)^s$ and $\mathbf{aux}$ is an auxiliary input. If each $\vec{a}_i$ is a valid pre-image for the commitment A_i (i.e., if $A_i = \vec{a}_i \times \vec{g}$ for all i), then the oracle replies with the tuple (α, σ) , where α is a standard random oracle response for $([A_i]_{i=1}^\ell, \mathbf{aux})$ and σ is a signature over $([A_i]_{i=1}^\ell, \mathbf{aux}, \alpha)$. Otherwise, the oracle returns $\perp$.

Properties. As in the SROM, verifying the signature σ guarantees that the pair $(([A_i]_{i=1}^\ell, \mathbf{aux}), \alpha)$ is a valid random oracle query-response, therefore allowing the construction of relativizing arguments in the **osROM**. The additional property of the **osROM** is that an extractor that observes the prover's oracle queries can now extract the pre-images for the commitments $[A_i]_{i=1}^\ell$. In other words, the **osROM** allows for straight-line extractors. This assumption is reminiscent of the algebraic group model (AGM) [23]. However, it allows us to avoid the pitfalls that come with defining the AGM for cycles of elliptic curves (see Remark 1). We believe this accurately represents the assumptions being made in real-world use of folding protocols such as Nova [27], HyperNova [28], ProtoStar [10] and ProtoGalaxy [22].

Instantiation. As noted by Chiesa and Tromer [20], the SROM is itself difficult to instantiate. They suggest using secure hardware to perform the oracle's functionality. The **osROM** is comparatively more powerful and is related to the AGM. Since the AGM cannot be perfectly instantiated in the standard model [40], we cannot hope to have a perfect instantiation of our model either. Instead, as is common for popular models such as the ROM and the AGM, we propose heuristic instantiations in Section 3.2.

NIRoKs. Kothapalli and Parno [26] define algebraic reductions of knowledge as generalizations of arguments of knowledge and folding schemes. Reductions of knowledge allow us to describe protocols with greater granularity than if they are viewed as consolidated (S)NARKs. We aim to carry the same modularity into our IVC construction and therefore define non-interactive reductions of knowledge in the **osROM** as an analog to Kothapalli and Parno's reductions of knowledge. The interactive prover is replaced by a deterministic algorithm **Reduce** with access to an open-and-sign random oracle; the interactive verifier is replaced by a **ReduceInstance** deterministic algorithm.

HyperNova as NIRoKs. To illustrate the **osROM** and the modularity of NIRoKs, we describe HyperNova's folding scheme [28] as a sequence of NIRoKs. We will later use this result to show that HyperNova's IVC scheme instantiated over a 2-cycle of curves is secure in the **osROM** for polynomial depth of recursion.

IVC compiler: single curve. Our main technical contribution is an IVC construction that makes black-box use of simple NIRoKs and explicitly supports 2-cycles of elliptic curves. As we discuss in our Related Work (Section 1.2), existing compilers fall short of one of these features; either they do not support cycles of curves, or they are defined only for specific reductions of knowledge. We start with a simpler compiler, defined only over a single curve E of prime order p defined over a field $\mathbb{F}_q$. This compiler serves as a warm-up and is equivalent to the one presented by Bünz *et al.* [11].

Reductions and step circuit. We require the existence of two NIRoKs and their associated relations as described below:

- The *priming* reduction Π_{prime} is defined as a reduction for the $\mathbb{F}_p$ -arithmetic circuit satisfiability relation $\mathcal{R}_{\text{arith}}$ and an intermediate relation $\mathcal{R}_{\text{inter}}$

$$\Pi_{\text{prime}} : \mathcal{R}_{\text{arith}} \rightarrow \mathcal{R}_{\text{inter}}.$$

There are no requirements placed on $\mathcal{R}_{\text{inter}}$, however choosing this relation will determine the IVC proof length and the amount of prover work. We also define the relation $\mathcal{R}_{\text{prime}}$ that is associated with verifying the reduction Π_{prime} and the membership of a pair $(\phi, w)_{\text{inter}}$ in $\mathcal{R}_{\text{inter}}$:

$$\mathcal{R}_{\text{prime}} := \left\{ \begin{array}{l} \phi := (\phi_{\text{arith}}, \phi_{\text{inter}}, \pi_{\text{prime}}) \\ w := (w_{\text{inter}}) \end{array} \middle| \begin{array}{l} \phi_{\text{inter}} = \Pi_{\text{prime}}.\text{ReduceInstance}(\phi_{\text{arith}}, \pi_{\text{prime}}) \\ (\phi_{\text{inter}}, w_{\text{inter}}) \in \mathcal{R}_{\text{inter}} \end{array} \right\}$$

In the very common case where $\phi_{\text{inter}} = (\phi_{\text{arith}} || \mathbf{aux}_\phi)$ for some $\mathbf{aux}_\phi$, we can remove duplicate inputs and write $\phi_{\text{prime}} := (\phi_{\text{inter}}, \pi_{\text{prime}})$.

- The *accumulation* (or folding) reduction Π_{acc} is a 2-to-1 reduction defined with respect to $\mathcal{R}_{\text{prime}}$ and an accumulator relation $\mathcal{R}_{\text{acc}}$ for which we impose no restrictions:

$$\Pi_{\text{acc}} : \mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}} \rightarrow \mathcal{R}_{\text{acc}}.$$

We highlight that $\mathcal{R}_{\text{prime}}$ is defined in such a way that if a prover produces a valid pair $(\phi, \mathbf{w})_{\text{prime}} \in \mathcal{R}_{\text{prime}}$, then it must have known a satisfying pair $(\phi, \mathbf{w})_{\text{arith}} \in \mathcal{R}_{\text{arith}}$. Indeed, this follows from the knowledge soundness property of the Π_{prime} NIroK.

Finally, we define an arithmetic circuit $\mathcal{C}_{\text{step}}$ that checks one execution of the step function F and runs the $\Pi_{\text{acc}}.\text{ReduceInstance}$ algorithm. However, note that this final algorithm manipulates commitments (i.e., elements of E) and is therefore better represented as an $\mathbb{F}_q$ -arithmetic circuit.

Examples. We illustrate how the above reductions and relations map to existing protocols, ignoring the ying superscript. In the Halo protocol [9], the relations $\mathcal{R}_{\text{inter}}$ and $\mathcal{R}_{\text{acc}}$ are taken to be the “polynomial commitment opening” relation $\mathcal{R}_{\text{poly}}$. The reduction Π_{prime} is directly lifted from the Sonic protocol [31]; the reduction Π_{acc} first reduces the incoming Π_{prime} pair to a pair in $\mathcal{R}_{\text{poly}}$ and batches the resulting $\mathcal{R}_{\text{poly}}$ pairs into the accumulator.

The Nova protocol [27] aims to minimize the $\Pi_{\text{acc}}.\text{ReduceInstance}$ algorithm. This is done by setting $\mathcal{R}_{\text{inter}}$ and $\mathcal{R}_{\text{acc}}$ to be the committed relaxed R1CS (crR1CS) relation, an almost trivial variant of its $\mathcal{R}_{\text{arith}}$ relation (R1CS). $\mathcal{R}_{\text{prime}}$ simply corresponds to verifying that a crR1CS pair is derived from a strict R1CS pair, i.e. that the witness commitment is consistent and that the error term is 0. Π_{acc} corresponds to checking $\mathcal{R}_{\text{prime}}$ and then taking a random linear combination (with appropriate cross terms and error terms) of the crR1CS pairs.

IVC prover. We can now describe the IVC prover at a high level. A valid IVC proof for i iterations of the step function is a tuple

$$\pi_i = ((\phi, \mathbf{w})_{\text{prime}, i}, (\phi, \mathbf{w})_{\text{acc}, i}) \in \mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}}.$$

Intuitively, the pair $(\phi, \mathbf{w})_{\text{acc}, i}$ attests to the correct execution of steps 0 to $i - 1$, while $(\phi, \mathbf{w})_{\text{prime}, i}$ attests to the correct execution of step i .

To produce a new IVC proof π_{i+1} , the prover runs one execution of the **acc-arith-prime** loop described below:

1. **acc:** the prover first accumulates the existing instance-witness pairs to produce the $(i + 1)$ -th accumulator

$$(\phi, \mathbf{w})_{\text{acc}, i+1} = \Pi_{\text{acc}}.\text{Reduce}((\phi, \mathbf{w})_{\text{prime}, i}, (\phi, \mathbf{w})_{\text{acc}, i})$$

2. **arith:** the prover then performs the necessary computation to obtain a set of satisfying inputs to the step circuit $\mathcal{C}_{\text{step}}$. This includes running the $(i + 1)$ -th execution of the step function and the $\Pi_{\text{acc}}.\text{ReduceInstance}$ algorithm. The honest prover then generates a pair $(\phi, \mathbf{w})_{\text{arith}, i+1} \in \mathcal{R}_{\text{arith}}$.
3. **prime:** finally, the prover primes these inputs for the next step of IVC and computes

$$(\phi, \mathbf{w})_{\text{inter}, i+1} = \Pi_{\text{prime}}.\text{Reduce}((\phi, \mathbf{w})_{\text{arith}, i+1})$$

and derives the corresponding pair $(\phi, \mathbf{w})_{\text{prime}, i+1}$.

By construction, an honest prover will produce a new valid proof $((\phi, \mathbf{w})_{\text{prime}, i+1}, (\phi, \mathbf{w})_{\text{acc}, i+1}) \in \mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}}$. To see why this scheme is knowledge sound, we inspect the guarantees given by each step of the **acc-arith-prime** loop in reverse order. Consider an IVC proof $((\phi, \mathbf{w})_{\text{prime}, j}, (\phi, \mathbf{w})_{\text{acc}, j}) \in \mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}}$. A valid pair $(\phi, \mathbf{w})_{\text{prime}, j}$ tells us that a pair $(\phi, \mathbf{w})_{\text{arith}, j}$ is extractable, by the definition of $\mathcal{R}_{\text{prime}}$ and the knowledge soundness of the Π_{prime} NIroK. By definition of the step circuit, the pair $(\phi, \mathbf{w})_{\text{arith}, j}$ will contain the necessary inputs to run the Π_{acc} extractor. Since $(\phi, \mathbf{w})_{\text{acc}, j} \in \mathcal{R}_{\text{acc}}$, we can run this extractor and obtain an IVC proof for $j - 1$ steps $((\phi, \mathbf{w})_{\text{prime}, j-1}, (\phi, \mathbf{w})_{\text{acc}, j-1}) \in \mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}}$.

IVC compiler: 2-cycle. As noted in the previous description, the algorithm $\Pi_{\text{acc}}.\text{ReduceInstance}$ is better represented as an $\mathbb{F}_q$ -arithmetic circuit. Let $E^{(\text{ying})} := E$ be the curve specified previously (of order p and defined over the field $\mathbb{F}_q$) and consider a second curve $E^{(\text{yang})}$ that forms a 2-cycle with $E^{(\text{ying})}$. That is, $E^{(\text{yang})}$ is of prime order q and defined over the field $\mathbb{F}_p$. We can now define the $\mathbb{F}_q$ -arithmetic satisfiability relation $\mathcal{R}_{\text{arith}}^{(\text{yang})}$, the NIROKs $\Pi_{\text{prime}}^{(\text{yang})}$ and $\Pi_{\text{acc}}^{(\text{yang})}$ as well as all the associated relations for the yang curve.

An IVC proof is now defined as a tuple:

$$\pi_i = \left[(\phi, \mathbf{w})_{\text{prime}, i}^{(\text{yang})}, (\phi, \mathbf{w})_{\text{acc}, i}^{(\text{ying})}, (\phi, \mathbf{w})_{\text{acc}, i}^{(\text{yang})} \right].$$

Finally, the prover operates as in the single curve case, except that the **acc-arith-prime** loop is expanded into **acc^(yang)-arith^(ying)-prime^(ying)-acc^(ying)-arith^(yang)-prime^(yang)**.

1.2 Related Work

IVC with constant-sized proofs. The first constructions of IVC schemes [4, 5, 19, 39] were constructed from recursively composing SNARKs for circuit satisfiability; that is, NIROKs from an NP relation to the truth relation where the **ReduceInstance** algorithm is exponentially faster than the **Reduce** algorithm. Halo [9] and its generalizations by Boneh *et al.* [8] and Bünz *et al.* [12] mark a second generation of IVC schemes. In these constructions, it is enough to use a NIROK that reduces the circuit satisfiability problem to a relation that has short instances and can be decided in linear time, provided that this latter relation admits a 2-to-1 (or many-to-1) folding scheme. These constructions allow for a more efficient IVC prover since they greatly reduce the amount of work that needs to be proven at each step in addition to the step function. The third generation of IVC schemes — initiated by Bünz *et al.* [11] and Nova [27], and adopted in follow-up works [6, 10, 22, 28, 33, 41] — pushes this logic to its fullest. In these constructions, it is enough to choose a NIROK that reduces the circuit satisfiability problem to any NP relation that admits an efficient 2-to-1 (or many-to-1) folding scheme. Our NIROK-based IVC framework allows us to capture all three generations of IVC constructions in a single unifying description.

Support for cycles of curves. Unfortunately, with the exception of the works of Ben-Sasson *et al.* [4], HyperNova [28] and Nguyen, Boneh and Setty [34], the above IVC constructions do not explicitly prove security over a 2-cycle of elliptic curves. As noted by Nguyen, Boneh and Setty [34], this has led to a critical soundness bug in the implementation of Nova. All three of the above exceptions fall short of describing a fully general IVC construction over a 2-cycle. Indeed, Ben-Sasson *et al.* [4] describe a generic first generation IVC construction, while HyperNova [28] and Nguyen, Boneh and Setty [34] are both restricted to the HyperNova and Nova folding schemes respectively.

Analysis in the proven-security regime. Most of the above works require a heuristic instantiation step in their security proofs. Indeed, many of these constructions rely on protocols described in the random oracle model. In order to recursively compose these protocols, the random oracle is heuristically instantiated as a hash function. In doing so, these protocols lose their proven security.

Models for relativizing arguments. In order to avoid the above issue, one must design a model in which relativizing arguments exist. In their seminal work, Chiesa and Tromer solve this issue by introducing the *signed random oracle model* (SROM) [20]. Subsequent works remove the signature scheme and focus on endowing the oracle with algebraic structure [17, 18]. Notably, Chen *et al.* [17] introduce the *arithmetized random oracle model* (AROM), a model that allows both a provable construction of PCD *and* supports a heuristic, software-only instantiation [17]. Finally, Lee and Seo [29] remove the need for oracles altogether and instead rely only on particular properties of hash functions. Our model is a close variant to the SROM that makes the additional assumption that succinct elliptic curve commitments are extractable.

Straight-line extraction. Real-world deployments of IVC make the assumption that the chain of computation can be extended almost endlessly. However, most of our security proofs only allow for a constant number of computation steps. By proving that the underlying NIROKs admit a straight-line extractor, we can prove that the IVC scheme is secure for polynomial depths. To the best of our knowledge, the only analysis that gives these guarantees for elliptic-curve-based protocols is the analysis of Nova in the AGM by Lee and Seo [29].

Closer comparison to Lee and Seo [29]. The approach of Lee and Seo [29] relies on two components: a cryptographic assumption on the strength of hash functions (*generalized zero testing*, GZT) and a computational model they call the *extended AGM* (EAGM). The GZT assumption is successful in removing the need for random oracles. However, it does not provide straightline extractability and therefore requires to be combined with something along the lines of the AGM or knowledge assumptions. The EAGM fulfills that role. However, it is extremely novel and under-explored. It requires non-trivial changes to the AGM including provisions for the fact that group elements can also be interpreted as a pair of field elements [29, Modification 1] and a form of hybridisation with the GGM [29, Modification 2]. The security implications of these modifications are yet to be confidently understood. In contrast, we believe that our osROM model makes more conservative assumptions: we only require that the oracle produces random values and that it logs pre-images to the prover’s commitments. Both approaches suffer from the fact that the security proofs cannot be readily translated into the standard model.

IVC from other assumptions. A separate line of work [21, 36] constructs IVC from the learning with errors (LWE) assumption, without relying on random oracles. Follow-up work [25] shows that these constructions can also be instantiated from a combination of the decisional Diffie-Hellman (DDH) and quadratic residuosity (QR) assumptions, or from the decision linear assumption (DLIN) for bilinear maps. These are the first IVC schemes that do not rely on heuristics for instantiation. However, they do not achieve constant-sized IVC proofs. The constructions of Devadas *et al.* [21], and Paneth and Pass [36] (concurrent works) rely on rate-1 batch arguments (BARGs). BARGs are similar to folding schemes in that they reduce the task of checking m NP witnesses, to the task of verifying a single proof π . A BARG is said to be “rate-1” when the size of the proof π is no greater than that of a single witness and a small additive overhead. This differs from folding schemes, which require that the size of the output witness is strictly equal to the size of a single incoming witness. As a result, the IVC schemes obtained from rate-1 BARGs have proofs that grow at best polylogarithmically with the number of IVC steps.

Concurrent work. In a preprint released concurrently, Bünz *et al.* [14] introduce the notions of interactive oracle reductions (IOR) as an analog to interactive oracle proofs (IOP) [3]. They show how to compile an IOR to a non-interactive reduction in the random oracle model using a variant of the BCS transform [3]. Their IVC construction is mostly equivalent to our single-curve construction: where we require a “priming” and “accumulation” reduction (resp. Π_{prime} and Π_{acc}), they have reductions “cast” and “fold” that fulfill the same purposes. However, since their analysis is done in the “pure-ROM” (*i.e.*, the ROM and no further assumptions), their IVC scheme focuses on Merkle-tree commitments and therefore does not consider IVC over cycles of elliptic curves.

2 Preliminaries

General notation and definitions. Let $n \in \mathbb{N}$ denote the security parameter. A function $\nu : \mathbb{N} \rightarrow \mathbb{R}$ is called *negligible* if for all $c \in \mathbb{R}, c > 0$, there exists $k_0 \in \mathbb{N}$ such that $|\nu(k)| < \frac{1}{k^c}$ for all $k \in \mathbb{N}, k \geq k_0$. For a non-empty set S , let $x \xleftarrow{\$} S$ denote sampling an element of S uniformly at random and assigning it to x . We represent vectors as $\vec{a} = (a_1, \dots, a_n)$.

Let PPT denote probabilistic polynomial time. Let $y \leftarrow A(x; \rho)$ denote running algorithm A on input x and randomness ρ and assigning its output to y . Let $y \xleftarrow{\$} A(x)$ denote $y \leftarrow A(x; \rho)$ for a uniformly random ρ . The set of values that are output with non-zero probability by A on input x is denoted by $[A(x)]$.

Group generation. Let $\mathcal{G}$ be a deterministic polynomial-time algorithm that takes as input a security parameter n and outputs a group description $(\mathbb{G}, p, g)$ consisting of a group $\mathbb{G}$ of order p , where p is a n -bit prime, and a generator g of $\mathbb{G}$. We write $\mathbb{F}_p$ to denote a finite field with prime order p . We also assume the existence of an algorithm $\mathcal{CG}$ that outputs, on input n , two elliptic curves $E^{(\text{ying})}$ and $E^{(\text{yang})}$ of order $|\mathbb{F}^{(\text{ying})}|$ and $|\mathbb{F}^{(\text{yang})}|$, respectively, such that $E^{(\text{ying})}$ is defined over the field $\mathbb{F}^{(\text{yang})}$ and $E^{(\text{yang})}$ over $\mathbb{F}^{(\text{ying})}$.

Additional definitions. We include the standard definitions of Signature and Commitment Schemes in Appendix A. Unless stated otherwise, we adopt the Pedersen commitment scheme, which satisfies binding and is homomorphic, according to Definition 5.

3 The Open-and-Sign Model

3.1 Idealized Open-and-Sign Model

In this work we aim to formally prove the security of elliptic-curve-based IVC schemes up to polynomial depths of computation. Thus, we need a model in which relativizing arguments exist and support straight-line extraction, as opposed to relying on proofs in the ROM or rewinding adversaries, which are common in the existing literature. We propose a model of computation called the Open-and-Sign Random Oracle Model (**osROM**). Our model is an extension of the signed random oracle model of Chiesa and Tromer [20], which inherits proven security and additionally supports elliptic curve commitments.

Remark 1 (Comparison with the AGM and SRO). In many ways, our model can be seen as a combination of the Algebraic Group Model and the Signed Random Oracle Model. However, there are many subtleties in using the AGM for recursion. For example, in proving the security of Nova, Lee and Seo [29] discuss their need to modify the AGM due to the need to extract representations from R1CS encodings of group elements (as opposed to the group elements themselves). Our model neatly avoids extracting from circuit encodings of group elements by applying extraction only when an adversary makes a random oracle query.

In our model an adversary is given access to an oracle $\mathcal{O}_{\text{osRO}}$ which implements the functionality **osRandomOracle** as defined in Fig. 1. The oracle $\mathcal{O}_{\text{osRO}}$ is defined with respect to a signature scheme Π_{sign} and is initialized with a group description $\mathbb{G}$ and a list of generators $\vec{g}$. The fact that $(\mathbb{G}, \vec{g})$ are inputs to the initialization algorithm of the oracle is crucial here, as the prover must not be able to choose the generators. The external description of $\mathcal{O}_{\text{osRO}}$ includes a signature verification key vk and the list of generators $\vec{g}$.

The prover is equipped with an oracle and can make queries of the form

$$([A_i, \vec{a}_i]_{i=1}^\ell, \text{aux})$$

where $\text{aux} \in \{0, 1\}^*$ and for $i \in [1, \ell]$ we have $A_i \in \mathbb{G}$ and $\vec{a}_i$ has entries in $\mathbb{F}_p$.

The oracle checks if it has received this query before and if yes returns the same output. It also checks that for $i \in [1, \ell]$, $\vec{a}_i$ is a valid pre-image of A_i in the sense that $A_i = \vec{a}_i \times \vec{g}$ and aborts if not. Else, the oracle chooses random value α and computes $\sigma_\alpha \leftarrow \Pi_{\text{sign}}.\text{Sign}(\text{sk}, ([A_i]_{i=1}^\ell, \text{aux}, \alpha))$ for sk the signing key relating to vk . The oracle returns (α, σ_α) . Note that **Sign** does not take in the pre-images $[\vec{a}_i]_{i=1}^\ell$ as input.

The verifier does not have direct access to $\mathcal{O}_{\text{osRO}}$, it is instead given the description $\text{vk}, \mathbb{G}, \vec{g}$. If they want to enforce that an output α was generated as the result of querying $\mathcal{O}_{\text{osRO}}$ on $([A_i]_{i=1}^\ell, \text{aux})$, they check that the signature is valid, i.e., assert $\Pi_{\text{sign}}.\text{SignVerify}(\text{vk}, ([A_i]_{i=1}^\ell, \text{aux}, \alpha), \sigma_\alpha) = 1$. Furthermore, we can extract a decomposition for A_i simply by recording $\vec{a}_i$ when the query is made.

Assuming the existence of a secure signature scheme and a trusted party to run $\mathcal{O}_{\text{osRO}}$ we have that this idealized model exists. Still, such an instantiation is neither satisfying nor useful in real-world applications. Below, we propose heuristic instantiations.

3.2 Heuristic instantiations of the osROM

The **osROM** can be interpreted as an extension of the **SROM** with an additional restriction on all adversaries that is reminiscent of knowledge of exponent assumptions or the AGM. This interpretation serves as the basis for our heuristic instantiations. Unfortunately, as we will see later, it also suggests that the **osROM** cannot be instantiated in the standard model.

Smart cards in the AGM. The first natural heuristic instantiation is to consider an instantiation of the **SROM** in the AGM. In other words, we consider a smart card or other trusted hardware that performs the necessary signatures on $([A_i]_{i=1}^\ell, \text{aux})$ and assume that the pre-images $[\vec{a}_i]_{i=1}^\ell$ are provided directly by the adversary as required by the AGM.

```

InitOracle( $n, (\mathbb{G}, p, \vec{g}), \Pi_{\text{sign}}$ )
 $Q \leftarrow \emptyset$ 
 $(\text{sk}, \text{vk}) \leftarrow \Pi_{\text{sign}}.\text{KeyGen}(n)$ 
 $\mathcal{O}_{\text{osRO}} \leftarrow \text{osRandomOracle}_{(\mathbb{G}, p, \vec{g}), \Pi_{\text{sign}}, \text{sk}}$ 
return  $\text{vk}, \mathcal{O}_{\text{osRO}}$ 

osRandomOracle( $(\mathbb{G}, p, \vec{g}), \Pi_{\text{sign}}, \text{sk}([A_i]_{i=1}^\ell, \text{aux})$ )
if  $(\{A_i\}_{i=1}^\ell, \text{aux}, \alpha, \sigma_\alpha) \in Q$ :
    return  $(\alpha, \sigma_\alpha)$ 
for  $i \in [1, \ell]$ :
    assert  $A_i = \vec{a}_i \times \vec{g}$ 
 $\alpha \xleftarrow{\$} \mathbb{F}_p$ 
 $\sigma_\alpha \leftarrow \Pi_{\text{sign}}.\text{Sign}(\text{sk}, (\{A_i\}_{i=1}^\ell, \text{aux}, \alpha))$ 
add  $(\{A_i\}_{i=1}^\ell, \text{aux}, \alpha, \sigma_\alpha)$  to  $Q$ 
return  $(\alpha, \sigma_\alpha)$ 

```

Fig. 1. The InitOracle and osRandomOracle procedures for a security parameter n and signature scheme Π_{sign} .

Hash functions in the AGM. A second, highly optimistic instantiation, is to treat the signed random oracle as we would treat a random oracle and replace it with a hash function. We replace all $\mathcal{O}_{\text{osRO}}([A_i]_{i=1}^\ell, \text{aux})$ queries from the honest prover with $\alpha \leftarrow \text{Hash}([A_i]_{i=1}^\ell, \text{aux})$. Similarly we replace all checks $\text{SignVerify}(\text{vk}, ([A_i]_{i=1}^\ell, \text{aux}, \alpha), \sigma_\alpha)$ from the verifier with $\alpha = \text{Hash}(\{A_i\}_{i=1}^\ell, \text{aux})$. We then assume that the resulting protocol is secure in the AGM.

This instantiation is clearly broken: indeed one can easily devise a protocol that is secure in the osROM but insecure under the instantiation. However, this particular instantiation closely models how IVC schemes are implemented and deployed today.

Un-instantiability in the standard model. Since the AGM cannot be instantiated [40], we do not expect to be able to fully instantiate the osROM in the standard model. While this is undesirable, it also holds true for other popular models such as the ROM [16].

3.3 Non-interactive Reductions of Knowledge

We adapt the definition of algebraic reductions of knowledge of Kothapalli and Parno [26] to define Non-interactive Reductions of Knowledge in the osROM. There are two notable differences: naturally, our definition only supports non-interactive algorithms meaning that provers and verifiers cannot interact, and our extractor does not depend on the prover. Instead, it is allowed to observe the prover's queries to the oracle $\mathcal{O}_{\text{osRO}}$.

Definition 1 (Non-interactive Reduction of Knowledge). *A Non-interactive reduction of knowledge (NIRoK) from relation $\mathcal{R}_1$ to $\mathcal{R}_2$ in the open-and-sign model consists of polynomial time algorithms $\Pi = (\text{KeyGen}, \text{Reduce}, \text{ReduceInstance})$ that behave as follows in the presence of an open-and-sign oracle $\mathcal{O}_{\text{osRO}}$ and corresponding verification key vk (as defined in Figure 1):*

- $\text{pp} \leftarrow \text{KeyGen}(n, s)$: Takes as input a security parameter n , a polynomial-size bound $s \in \mathbb{N}$ and returns public parameters pp , including a group $\mathbb{G}$ of order p , where p is a n -bit prime and a list of generators $\vec{g} \leftarrow \mathbb{G}^s$.
- $(\phi_2, \mathbf{w}_2, \pi) \leftarrow \text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, \phi_1, \mathbf{w}_1)$: Takes as input pp and an instance-witness pair $(\phi_1, \mathbf{w}_1) \in \mathcal{R}_1$. Given access to the oracle $\mathcal{O}_{\text{osRO}}$, it returns a new instance and witness such that $(\phi_2, \mathbf{w}_2) \in \mathcal{R}_2$ and a proof.
- $\phi_2/\perp \leftarrow \text{ReduceInstance}(\text{pp}, \text{vk}, \phi_1, \pi)$: On input pp , the signature verification key vk , an instance, and a proof, returns either a new reduced instance or $\perp$ to indicate failure.

and satisfy completeness (Definition 2) and knowledge-soundness (Definition 3). When $(\mathbb{G}, p, \vec{g})$ are clear from the context, we write $\text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, \phi_1, \mathbf{w}_1)$ to simplify notation.

Definition 2. [Completeness] For a security parameter n and a number of generators n_{gen} , let $\Pi := (\text{KeyGen}, \text{Reduce}, \text{ReduceInstance})$ denote a non-interactive reduction of knowledge (Definition 1) from $\mathcal{R}_1$ to $\mathcal{R}_2$.

For all $(\phi_1, \mathbf{w}_1) \in \mathcal{R}_1$:

$$\Pr \left[\phi_2 = \phi'_2 \wedge (\phi_2, \mathbf{w}_2) \in \mathcal{R}_2 \mid \begin{array}{l} (\text{vk}, \mathcal{O}_{\text{osRO}}) \leftarrow \text{InitOracle}(n, \Pi_{\text{sign}}) \\ \text{pp} \leftarrow \text{KeyGen}(n, s) \\ (\phi_2, \mathbf{w}_2, \pi) \leftarrow \text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, \phi_1, \mathbf{w}_1) \\ \phi'_2 \leftarrow \text{ReduceInstance}(\text{pp}, \text{vk}, \phi_1, \pi) \end{array} \right] = 1$$

where InitOracle is the procedure defined in Figure 1.

Definition 3. [Knowledge-Soundness] Let $\Pi = (\text{KeyGen}, \text{Reduce}, \text{ReduceInstance})$ be a reduction. Let n denote a security parameter, $s \in \mathbb{N}$ a polynomial-size bound and Π_{sign} denote a signature scheme. We define the advantage $\text{Adv}_{\text{Ext}, \mathcal{A}}^{\text{rok}}(n)$ of an extractor Ext and adversary $\mathcal{A}$ to be

$$\text{Adv}_{\text{Ext}, \mathcal{A}}^{\text{rok}}(n) := \Pr[\text{Game}_{\text{Ext}, \mathcal{A}}^{\text{rok}}(n, \Pi_{\text{sign}}, s) = 1]$$

for $\text{Game}_{\text{Ext}, \mathcal{A}}^{\text{rok}}(n, \Pi_{\text{sign}}, s)$ in Fig. 2.

We say that Π is knowledge-sound if there exists a PPT extractor Ext such that for any PPT adversary $\mathcal{A}$ we have that $\text{Adv}_{\text{Ext}, \mathcal{A}}^{\text{rok}}(n) \leq \text{negl}(n)$.

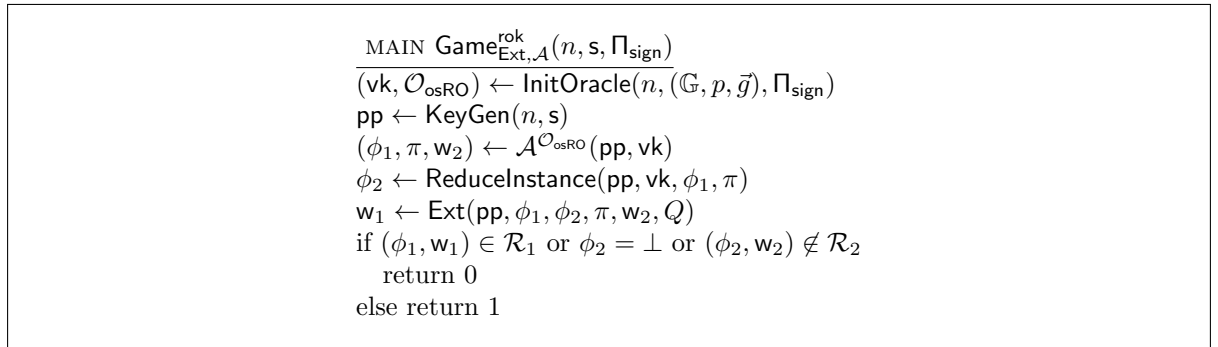


Fig. 2. Reduction of Knowledge k-s security game. Here, Q denotes the transcript of queries made by $\mathcal{A}$ to the open-and-sign oracle $\mathcal{O}_{\text{osRO}}$.

3.4 Incrementally Verifiable Computation

Incrementally verifiable computation (IVC) was first defined in the context of deterministic computations [39]. Subsequent works introduce the notions of “local code” [20] or a “local witness” [27]: in both cases, these are values that affect the output of F and are inserted non-deterministically into the chain of computation. We adapt the definition of IVC for non-deterministic computations from Nguyen, Boneh and Setty [34] to the osROM.

Definition 4. (IVC in the osROM) An IVC scheme is a tuple of efficient algorithms $(\text{Setup}, \mathcal{P}, \mathcal{V})$ with the following interface in the presence of an open-and-sign oracle $\mathcal{O}_{\text{osRO}}$ and corresponding verification key vk (as defined in Figure 1):

- $\text{Setup}(n, s) \rightarrow \text{pp}$: Given a security parameter n , a polynomial-size bound $s \in \mathbb{N}$, outputs public parameters pp . These parameters include a group $\mathbb{G}$ of order p , where p is an n -bit prime and a list of generators $\vec{g} \leftarrow \mathbb{G}^{n_{\text{gen}}}$.

- $\mathcal{P}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, F, (i, z_0, z_i), \omega_i, \pi_i) \rightarrow \pi_{i+1}$: Given access to the oracle $\mathcal{O}_{\text{osRO}}$, public parameters pp , a function $F : \{0, 1\}^a \times \{0, 1\}^b \rightarrow \{0, 1\}^a$ computable by a circuit bounded by $\mathbf{s}$, an index $i \in \mathbb{N}$, an initial input $z_0 \in \{0, 1\}^a$, a claimed output $z_i \in \{0, 1\}^a$, advice $\omega_i \in \{0, 1\}^b$, and an IVC proof π_i , outputs a new IVC proof π_{i+1} .
- $\mathcal{V}(\text{pp}, \text{vk}, F, (i, z_0, z_i), \pi_i) \rightarrow 0/1$: Given public parameters pp , the oracle verification key vk , a function F , an index i , an initial input z_0 , a claimed output z_i , and an IVC proof π_i , outputs 0 (reject) or 1 (accept).

An IVC scheme satisfies the following properties:

Completeness: For every polynomial-size bound $\mathbf{s} \in \mathbb{N}$, pp in the output space of $\text{Setup}(n, \mathbf{s})$, function $F : \{0, 1\}^a \times \{0, 1\}^b \rightarrow \{0, 1\}^a$ computable by a circuit within bound $\mathbf{s}$, open-and-sign oracle $\mathcal{O}_{\text{osRO}}$, collection of elements $(i \in \mathbb{N}, z_0, z_i \in \{0, 1\}^a)$, $\omega_i \in \{0, 1\}^b$ and IVC proof π_i , the following probability equals 1.

$$\Pr \left[\begin{array}{c} \mathcal{V}(\text{pp}, \text{vk}, F, (i, z_0, z_i), \pi_i) = 1 \\ \Downarrow \\ \mathcal{V}(\text{pp}, \text{vk}, F, (i+1, z_0, z_{i+1}), \pi_{i+1}) = 1 \end{array} \middle| \begin{array}{l} \pi_{i+1} \leftarrow \mathcal{P}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, F, (i, z_0, z_i), \omega_i, \pi_i), \\ z_{i+1} \leftarrow F(z_i, \omega_i) \end{array} \right]$$

Knowledge soundness: Let $\mathbf{s} \in \mathbb{N}$ be a polynomial-size bound and $\ell(n)$ be a polynomial in the security parameter. Let $\mathcal{O}_{\text{osRO}}$ be an open-and-sign random oracle with the associated verification key vk and query transcript Q . Let $\mathcal{F}$ be an efficient function sampling adversary that outputs a function $F : \{0, 1\}^a \times \{0, 1\}^b \rightarrow \{0, 1\}^a$ computable by a circuit within the polynomial-size bound $\mathbf{s}$. We say that an IVC scheme is knowledge sound if there exists an efficient extractor Ext such that for every efficient IVC prover $\mathcal{P}^*$ the probability of the following event is greater than $1 - \text{negl}(n)$

$$\left[\begin{array}{c} \mathcal{V}(\text{pp}, F, (i, z_0, z_i), \pi_i) = 1 \\ \Downarrow \\ z_i = F(z_{i-1}, \omega_{i-1}) \wedge \\ (i = 1 \implies z_{i-1} = z_0) \wedge \\ (i > 1 \implies \mathcal{V}(\text{pp}, F, (i-1, z_0, z_{i-1}), \pi_{i-1}) = 1) \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(n, \mathbf{s}), \\ \rho \leftarrow \{0, 1\}^{\ell(n)}, \\ F \leftarrow \mathcal{F}(\text{pp}; \rho), \\ (i, z_0, z_i, \pi_i) \leftarrow \mathcal{P}^{*\mathcal{O}_{\text{osRO}}}(\text{pp}, \rho), \\ (z_{i-1}, \omega_{i-1}), \pi_{i-1} \leftarrow \text{Ext}(\text{pp}, \rho, Q) \end{array} \right]$$

Note that the knowledge soundness definition above only requires the existence of a “single-step extractor”: from an accepting proof π_i , one can extract a previous state z_{i-1} , a step witness ω_{i-1} and a valid IVC proof for $(i-1)$ steps. As argued in [34] this notion is enough to capture previous security definitions of IVC. Indeed, one can extract a full witness $\omega_1, \omega_2, \dots, \omega_{i-1}$ by running the above extractor recursively. Importantly, the runtime of the step extractor bounds the recursion depth of the IVC scheme (Remark 1 in [34]).

Remark 2 (Difference with [34]). Our IVC definition differs from that in [34] in that it is formulated within the osROM . As a consequence, our knowledge soundness extractor is given as additional input the transcript Q of all queries made to $\mathcal{O}_{\text{osRO}}$. Furthermore, we require that a single extractor exists for all efficient provers; as opposed to requiring that for every prover there exists an extractor.

4 Capturing Systems in the Literature

As noted in [26], complex arguments of knowledge can be understood as compositions of simpler reductions of knowledge. This approach is implicit in most modern arguments of knowledge. For example, arguments of knowledge for general purpose relations often reduce to the security of the sumcheck protocol [30].

In this section we present HyperNova’s folding scheme [28] as a non-interactive reduction of knowledge and prove its security in the osROM .

This presentation serves two key purposes: (i) to familiarize the reader with the Open-and-Sign RO by presenting a well-known construction defined in terms of a NIROK and proving its security within the osRO model, and (ii) to lay the foundation for Section 6, where we introduce a framework for constructing IVC schemes that support cycles of elliptic curves from any folding scheme. By instantiating this framework with HyperNova, we derive an IVC scheme that supports polynomial-depth recursion.

4.1 HyperNova and folding schemes

HyperNova [28] is a folding scheme that reduces a Customizable Constraint System (CCS) [37] to a linearized-CCS System, and then folds the linearized relation. In this section we express it as a set of non-interactive reductions of knowledge and prove security accordingly. HyperNova is a recursive argument for NP relations expressed as CCS, namely the following relation:

$$\mathcal{R}_{\text{CCS}} \leftarrow \left\{ \left(((\mathbf{M}_j)_{j=1}^t, (S_i, c_i)_{i=1}^q, \vec{t}) \right) \left| \sum_{i=1}^q c_i \bigcirc_{j \in S_i} \mathbf{M}_j \vec{t} = 0 \right. \right\} \quad (1)$$

We decompose HyperNova's multifolding scheme as a composition of reductions of knowledge, and present them as non-interactive ones. We first describe the relations Committed-CCS ($\mathcal{R}_{\text{cCCS}}$) and Linearized-Committed-CCS ($\mathcal{R}_{\text{lcCCS}}$).

$$\mathcal{R}_{\text{cCCS}} \leftarrow \left\{ \left(((\mathbf{M}_j)_{j=1}^t, (S_i, c_i)_{i=1}^q, \phi_t, T), (\vec{t}) \right) \left| \begin{array}{l} T = \text{Commit}(\vec{t}) \\ \sum_{i=1}^q c_i \bigcirc_{j \in S_i} \mathbf{M}_j(\phi_t || \vec{t}) = 0 \end{array} \right. \right\} \quad (2)$$

$$\mathcal{R}_{\text{lcCCS}} \leftarrow \left\{ \left(((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}, \phi_u, U), (\vec{u}) \right) \left| \begin{array}{l} U = \text{Commit}(\vec{u}) \\ \forall j \in [t] \vec{r}^\top \mathbf{M}_j(\phi_u || \vec{u}) = v_j \end{array} \right. \right\} \quad (3)$$

In Section C.1 we describe and prove the security of a rok $\Pi_{\text{lcCCS}} : \mathcal{R}_{\text{cCCS}} \rightarrow \mathcal{R}_{\text{lcCCS}}$. We also describe a rok from $\mathcal{R}_{\text{lcCCS}}$ to itself that re-randomizes the instance such that it admits an efficient folding algorithm in Section C.3. Notably, in HyperNova Π_{lc} and Π_{rerand} are run together, so the re-randomization from Π_{rerand} is done along with the reduction Π_{lc} , resulting in both instances sharing the same randomness. Both reductions are combined with the Sumcheck reduction $\Pi_{\text{red-sc}}$ that we define and prove secure in Section B.

In Section C.2 we provide a rok to fold instances of $\mathcal{R}_{\text{lcCCS}}$ and in Section 4.2 the full rok from $\mathcal{R}_{\text{cCCS}} \times \mathcal{R}_{\text{lcCCS}} \mapsto \mathcal{R}_{\text{lcCCS}}$, i.e., HyperNova's folding scheme. All reductions share the same KeyGen algorithm, which takes as input a security parameter n and $s \in \mathbb{N}$ and outputs public parameters pp . KeyGen runs $\mathcal{G}$ on input n , and gets a group $\mathbb{G}$ of order p , where p is an n -bit prime and a list of generators $\vec{g} \leftarrow \mathbb{G}^s$. We also assume the presence of an open-and-sign oracle $\mathcal{O}_{\text{osRO}}$ with verification key vk , initialized with $(\mathbb{G}, p, \vec{g})$.

4.2 HyperNova's multifolding scheme

We present the reduction $\Pi_{\text{hn}} : \mathcal{R}_{\text{cCCS}} \times \mathcal{R}_{\text{lcCCS}} \rightarrow \mathcal{R}_{\text{lcCCS}}$ in Fig. 3. In a nutshell, given instance-witness pairs for $\mathcal{R}_{\text{cCCS}}$ and $\mathcal{R}_{\text{lcCCS}}$, the first one is reduced to a $\mathcal{R}_{\text{lcCCS}}$ pair using Π_{lc} , and then both are folded by following Π_{fold} . The Π_{rerand} is run inside Π_{fold} to ensure both linearized instances can be folded and compose a valid one.

$\Pi_{\text{hn}}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, (\phi_{1,a}, \phi_{1,b}, (\mathbf{w}_{1,a}, \mathbf{w}_{1,b})) \rightarrow (\phi_2, \mathbf{w}_2):$

- $(\phi_{lc}, \mathbf{w}_{lc}, \pi_{lc}) \leftarrow \Pi_{\text{lc}}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, \phi_{1,a}, \mathbf{w}_{1,a})$
- $(\phi_2, \mathbf{w}_2, \pi_{\text{fold}}) \leftarrow \Pi_{\text{fold}}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, (\phi_{1,b}, \phi_{lc})(\mathbf{w}_{1,b}), \mathbf{w}_{lc})$
- $\pi \leftarrow (\pi_{lc}, \pi_{\text{fold}})$
- return $(\phi_2, \mathbf{w}_2, \pi)$

$\Pi_{\text{hn}}.\text{ReduceInstance}(\text{pp}, \text{vk}, (\phi_{1,a}, \phi_{1,b}, \pi) \rightarrow \phi_2:$

- Parse $(\pi_{lc}, \pi_{\text{fold}}) \leftarrow \pi$
- $\phi_{lc} \leftarrow \Pi_{\text{lc}}.\text{ReduceInstance}_1(\text{pp}, \text{vk}, \phi_{1,a}, \pi_{lc})$
- $\phi_2 \leftarrow \Pi_{\text{fold}}.\text{ReduceInstance}(\text{pp}, \text{vk}, (\phi_{1,b}, \phi_{lc}), \pi_{\text{fold}})$
- return ϕ_2

Fig. 3. HyperNova folding scheme.

Theorem 1. *The protocol in Fig. 3 is a non-interactive reduction of knowledge from $\mathcal{R}_{\text{cCCS}} \times \mathcal{R}_{\text{lcCCS}}$ to $\mathcal{R}_{\text{lcCCS}}$.*

Proof. Completeness follows by inspection.

For knowledge soundness, let $\mathcal{A}$ be an adversary against the security game of Fig 2. On input pp , $\mathcal{A}$ outputs $((\phi_{1,a}, \phi_{1,b}), \pi, \mathbf{w}_2)$. We construct an extractor Ext that on input $(\text{pp}, (\phi_{1,a}, \phi_{1,b}), \phi_2, \pi, \mathbf{w}_2)$ where $\phi_2 \leftarrow \text{ReduceInstance}(\text{pp}, \text{vk}, (\phi_{1,a}, \phi_{1,b}), \pi)$, outputs $(\mathbf{w}_{1,a}, \mathbf{w}_{1,b})$. Adversary wins if (a) $((\phi_{1,a}, \phi_{1,b}), (\mathbf{w}_{1,a}, \mathbf{w}_{a,b})) \notin \mathcal{R}_{\text{cCCS}} \times \mathcal{R}_{\text{lcCCS}}$, (b) $\phi_2 \neq \perp$, and (c) $(\phi_2, \mathbf{w}_2) \in \mathcal{R}_{\text{lcCCS}}$. We assume (b) and (c) hold and prove that for $(\mathbf{w}_{1,a}, \mathbf{w}_{1,b})$ outputted by Ext , $((\phi_{1,a}, \phi_{1,b}), (\mathbf{w}_{1,a}, \mathbf{w}_{1,b})) \in \mathcal{R}_{\text{lcCCS}} \times \mathcal{R}_{\text{cCCS}}$ except with negligible probability.

Extractor Description: $\text{Ext}(\text{pp}, (\phi_{1,a}, \phi_{1,b}), \phi_2, \pi, \mathbf{w}_2)$:

- Parse $(\pi_{\text{lc}}, \pi_{\text{fold}}) \leftarrow \pi$
- Run $\phi_{\text{lc}} \leftarrow \Pi_{\text{lc}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_{1,a}, \pi_{\text{lc}})$
- Run Ext_{fold} on input $(\text{pp}, (\phi_{1,b}, \phi_{\text{lc}}), \phi_2, \pi, \mathbf{w}_2)$ to get $(\mathbf{w}_{1,b}, \mathbf{w}_{\text{lc}})$.
- Run Ext_{lc} on input $(\text{pp}, \phi_{1,a}, \phi_{\text{lc}}, \pi_{\text{lc}}, \mathbf{w}_{\text{lc}})$ to obtain $\mathbf{w}_{1,a}$.
- Output $(\mathbf{w}_{1,a}, \mathbf{w}_{1,b})$

$\text{Ext}(\text{pp}, (\phi_{1,a}, \phi_{1,b}), \phi_2, \pi, \mathbf{w}_2)$ does not abort except with negligible probability, which is the probability that either Ext_{lc} or Ext_{fold} abort since Π_{lc} and Π_{fold} are reductions of knowledge.

Now, because Π_{fold} is a RoK, we have that its output $(\mathbf{w}_{1,b}, \mathbf{w}_{\text{lc}})$ is such that $((\phi_{1,a}, \phi_{\text{lc}}), (\mathbf{w}_{1,a}, \mathbf{w}_{\text{lc}})) \in \mathcal{R}_{\text{lcCCS}} \times \mathcal{R}_{\text{lcCCS}}$ except with negligible probability.

Finally, since Π_{lc} is a reduction of knowledge, the output $\mathbf{w}_{1,a}$ from Ext_1 satisfies $(\phi_{1,a}, \mathbf{w}_{1,a}) \in \mathcal{R}_{\text{cCCS}}$ with overwhelming probability. We conclude that the output of Ext is a valid witness with probability $1 - \text{negl}(n)$ and thus the protocol of Fig. 3 is a reduction of knowledge from $\mathcal{R}_{\text{cCCS}} \times \mathcal{R}_{\text{lcCCS}} \rightarrow \mathcal{R}_{\text{lcCCS}}$. $\square$

5 An IVC framework: Warm-Up

The goal of this section and the next is to build a generic framework for constructing efficient IVC schemes using elliptic curve commitments. Given our framework, protocol designers can construct an IVC scheme by simply specifying two reductions of knowledge Π_{prime} and Π_{acc} that satisfy the properties we outline below.

In this section, we present the single-curve version of the framework as a warm-up. This version of the framework is equivalent to the PCD compiler of Bünz *et al.* [11] with the following modifications: (i) our framework is only concerned with IVC for non-deterministic functions rather than the full generality of PCD, (ii) our framework presents the building blocks as non-interactive reductions of knowledge, and (iii) we present the non-interactive reductions of knowledge in the **osROM**.

5.1 Relations

We assume the existence of four relations, with two corresponding reductions of knowledge:

1. $\mathcal{R}_{\text{arith}}$: the set of tuples $(\mathcal{C}_{\text{arith}}, \phi_{\text{arith}}, \mathbf{w}_{\text{arith}})$ such that $(\phi_{\text{arith}}, \mathbf{w}_{\text{arith}})$ satisfy the arithmetic circuit $\mathcal{C}_{\text{arith}}$.
2. $\mathcal{R}_{\text{inter}}$: an intermediate relation, which need not be NP-complete. We only require that there exists a NI-RoK $\Pi_{\text{prime}} : \mathcal{R}_{\text{arith}} \rightarrow \mathcal{R}_{\text{inter}}$.
3. $\mathcal{R}_{\text{prime}}$: a relation defined by the verification of the reduction of knowledge Π_{prime} and the verification of the resulting instance-witness pair $(\phi_{\text{inter}}, \mathbf{w}_{\text{inter}})$:

$$\mathcal{R}_{\text{prime}} := \left\{ \begin{array}{l} \phi := (\phi_{\text{arith}}, \phi_{\text{inter}}, \pi_{\text{prime}}) \\ \mathbf{w} := (\mathbf{w}_{\text{inter}}) \end{array} \middle| \begin{array}{l} \phi_{\text{inter}} = \Pi_{\text{prime}}.\text{ReduceInstance}(\phi_{\text{arith}}, \pi_{\text{prime}}) \\ (\phi_{\text{inter}}, \mathbf{w}_{\text{inter}}) \in \mathcal{R}_{\text{inter}} \end{array} \right\}$$

It is very common for $\mathcal{R}_{\text{arith}}$ and $\mathcal{R}_{\text{inter}}$ to be such that $\phi_{\text{inter}} = (\phi_{\text{arith}} || \text{aux}_{\phi})$ for some aux_{ϕ} . In this case, we can remove duplicate inputs and write $\phi_{\text{prime}} := (\phi_{\text{inter}}, \pi_{\text{prime}})$.

4. $\mathcal{R}_{\text{acc}}$: the accumulator relation. Like $\mathcal{R}_{\text{inter}}$ it need not be NP-complete. However, we require the existence of a non-interactive reduction of knowledge $\Pi_{\text{acc}} : (\mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}}) \rightarrow \mathcal{R}_{\text{acc}}$

Remark 3 (Comparison to [11]). The relations and reductions above correspond to the NARK and accumulation scheme of Bünz *et al.* [11]. The reduction Π_{prime} plays the role of the NARK, and $\mathcal{R}_{\text{prime}}$ corresponds to the NARK proof. Indeed, verifying that an instance-witness pair lies in $\mathcal{R}_{\text{prime}}$ guarantees extractability of the witness w_{arith} , just as with a NARK proof. The reduction Π_{acc} maps directly to the accumulation scheme.

Example 1. Instantiating the framework with HyperNova and the reductions from Section 4, the relations described above work as follows: $\mathcal{R}_{\text{arith}}$ is $\mathcal{R}_{\text{CCS}}$ as defined in Eq. (1) and $\mathcal{R}_{\text{inter}}$ is $\mathcal{R}_{\text{cCCS}}$ as in Eq. (2) in Section 4.1. Π_{prime} is the reduction from $\mathcal{R}_{\text{CCS}}$ to $\mathcal{R}_{\text{cCCS}}$: On input $(\phi_{\text{CCS}}, w_{\text{CCS}}) \in \mathcal{R}_{\text{CCS}}$, the prover sends a commitment $T := \text{Commit}(\vec{t})$ to the witness and both the prover and verifier output the new instance ϕ_{cCCS} to be $(\phi_{\text{CCS}} || T)$. The prover also outputs $w_{\text{cCCS}} := w_{\text{CCS}} = \vec{t}$. $\mathcal{R}_{\text{prime}}$ is essentially $\mathcal{R}_{\text{cCCS}}$: it checks that a CCS instance is satisfying and comes from committing to an existing CCS instance. The latter check is almost trivial and is essentially omitted. $\mathcal{R}_{\text{acc}}$ is $\mathcal{R}_{\text{lcCCS}}$ as in Eq. (3) while Π_{acc} is the reduction $\Pi_{\text{hn}} : \mathcal{R}_{\text{cCCS}} \times \mathcal{R}_{\text{lcCCS}} \rightarrow \mathcal{R}_{\text{lcCCS}}$ described in Fig. 3. The IVC proof is an instance-witness pair $((\phi_{1,a}, \phi_{1,b}), (w_{1,a}, w_{1,b})) \in \mathcal{R}_{\text{cCCS}} \times \mathcal{R}_{\text{lcCCS}}$.

5.2 The IVC framework

Overview. An IVC proof is a tuple $[(\phi, w)_{\text{prime}, i}, (\phi, w)_{\text{acc}, i}]$. Intuitively, $(\phi, w)_{\text{prime}, i} \in \mathcal{R}_{\text{prime}}$ attests to the i -th execution of the step function and the well-formation of the accumulator $(\phi, w)_{\text{acc}, i}$. On the other hand, the accumulator $(\phi, w)_{\text{acc}, i} \in \mathcal{R}_{\text{acc}}$ attests to the correctness of all previous pairs $(\phi, w)_{\text{prime}}$ for 1 to $i - 1$.

The prover updates a proof by performing one run of the **acc-arith-prime** loop. The prover first runs $\Pi_{\text{acc}}.\text{Reduce}$ on the existing proof elements to obtain a fresh accumulator $(\phi, w)_{\text{acc}, i+1}$. It then collects and computes the values $(\phi, w)_{\text{arith}, i+1}$ that satisfy the constraints of the circuit $\mathcal{C}_{\text{step}}$ (see Figure 4). The circuit enforces, among other constraints, that the step function F is iterated correctly and that $\phi_{\text{acc}, i+1}$ is correctly derived using the $\Pi_{\text{acc}}.\text{ReduceInstance}$ algorithm. Finally, $\mathcal{P}$ runs the $\Pi_{\text{prime}}.\text{Reduce}$ algorithm to obtain the values $(\phi, w)_{\text{prime}, i+1}$. Together, the tuple $((\phi, w)_{\text{prime}, i+1}, (\phi, w)_{\text{acc}, i+1})$ forms the next IVC proof.

$\mathcal{C}_{\text{step}}(\phi_{\text{arith}, i+1}, w_{\text{arith}, i+1}) \rightarrow \{0, 1\}$:
 Parse the instance $\phi_{\text{arith}, i+1} = h$.
 Parse the witness:

$$w_{\text{arith}, i+1} = (\text{pp}, \text{vk}, i, z_0, z_i, z_{i+1}, \omega_i, \phi_{\text{arith}, i}, \phi_{\text{inter}, i}, \phi_{\text{acc}, i}, \phi_{\text{acc}, i+1}, \pi_{\text{prime}, i}, \pi_{\text{acc}}).$$

Set $\phi_{\text{prime}, i} = (\phi_{\text{arith}, i}, \phi_{\text{inter}, i}, \pi_{\text{prime}, i})$.

Accept if all the conditions below are satisfied:

1. Check the hash for step i (input): $\phi_{\text{arith}, i} = H(\text{pp}, \text{vk}, i, z_0, z_i, \phi_{\text{acc}, i})$.
2. Check one iteration of the step function: $z_{i+1} = F(z_i, \omega_i)$.
3. **If** $i = 0$, assert that $\phi_{\text{acc}, i+1} = \phi_{\text{acc}, \perp}$ and $z_i = z_0$;
Else, check the instance reduction of Π_{acc} :

$$\phi_{\text{acc}, i+1} = \Pi_{\text{acc}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_{\text{prime}, i}, \phi_{\text{acc}, i}, \pi_{\text{acc}}).$$

4. Check the hash for step $i + 1$ (output):

$$h = H(\text{pp}, \text{vk}, i + 1, z_0, z_{i+1}, \phi_{\text{acc}, i+1}).$$

Fig. 4. Circuit (or constraints) $\mathcal{C}_{\text{step}}$ to be evaluated at each IVC step.

The **acc** step provides the 2-to-1 compression that allows the IVC proof to maintain a constant size in the number of computation steps. The **arith** step ensures that this compression was performed correctly,

and ensures that the verifier's work is also independent of the number of computation steps. Finally, the **prime** step allows to either convert that circuit inputs to a format that is more amenable to being accumulated (e.g. committed relaxed R1CS in Nova), or in a more succinct format (e.g. a SNARK proof [4], or a partial SNARK proof [9]).

Full single-curve scheme. The full IVC scheme over a single curve is described in Figure 5 and makes use of the helper circuit described in Figure 4. We prove scheme's security in Theorem 2.

Setup($1^n, s$) : Run $\text{pp}_{\text{acc}} \leftarrow \Pi_{\text{acc}}.\text{KeyGen}(n, s)$, $\text{pp}_{\text{prime}} \leftarrow \Pi_{\text{prime}}.\text{KeyGen}(n, s)$ and assume the presence of an oracle $\mathcal{O}$ with verification key vk , instantiated with $(\mathbb{G}, p, \vec{g})$.
Output $\text{pp} = (\text{pp}_{\text{acc}}, \text{pp}_{\text{prime}})$

$\mathcal{P}(\text{pp}, \text{vk}, F, (i, z_0, z_i), \omega_i, \pi_{\text{step}, i}) \mapsto \pi_{\text{step}, i+1}$

If $i = 0$:
Set $(\phi, w)_{\text{acc}, 0} \leftarrow (\phi, w)_{\text{acc}, \perp}$, where $(\phi, w)_{\text{acc}, \perp}$ represents a neutral for $\mathcal{R}_{\text{acc}}$.
 $(\phi, w)_{\text{acc}, 1} \leftarrow (\phi, w)_{\text{acc}, \perp}$
Compute $(\phi, w)_{\text{prime}, 1}$
– $\phi_{\text{arith}, 0} \leftarrow H(\text{pp}, \text{vk}, (0, z_0, z_0, \phi_{\text{acc}, \perp}))$
– $w_{\text{arith}, 0} \leftarrow (\text{pp}, \text{vk}, 0, z_0, z_0, z_0, \perp, \phi_{\text{arith}, 0}, \perp, \perp, \phi_{\text{acc}, 0}, \perp, \phi_{\text{acc}, 0})$
– $z_1 = F(z_0, w_0)$
– $\phi_{\text{arith}, 1} \leftarrow H(\text{pp}, \text{vk}, (1, z_0, z_1, \phi_{\text{acc}, 1}))$
– $w_{\text{arith}, 1} \leftarrow (\text{pp}, \text{vk}, 0, z_0, z_0, z_1, w_0, \phi_{\text{arith}, 0}, \text{aux}_{\phi}, \phi_{\text{acc}, 0}, \phi_{\text{acc}, 1}, \pi_{\text{prime}, 0}, \pi_{\text{acc}, 0})$
– $((\phi, w)_{\text{inter}, 1}, \pi_{\text{prime}, 1}) \leftarrow \Pi_{\text{prime}}.\text{Reduce}^{\mathcal{O}}(\text{pp}, (\phi, w)_{\text{arith}, 1})$
– Set $\phi_{\text{prime}, 1} \leftarrow (\phi_{\text{arith}, 1}, \phi_{\text{inter}, 1}, \pi_{\text{prime}, 1})$ and $w_{\text{prime}, 1} \leftarrow w_{\text{inter}, 1}$
Output $\pi_{\text{step}, 1} \leftarrow ((\phi, w)_{\text{prime}, 1}, (\phi, w)_{\text{acc}, 1})$

Else if $i \geq 1$:
Parse $((\phi, w)_{\text{prime}, i}, (\phi, w)_{\text{acc}, i}) \leftarrow \pi_{\text{step}, i}$
Compute $(\phi, w, \pi)_{\text{acc}, i+1} \leftarrow \Pi_{\text{acc}}.\text{Reduce}^{\mathcal{O}}((\phi, w)_{\text{prime}, i}, (\phi, w)_{\text{acc}, i})$.
Compute $(\phi, w)_{\text{arith}, i+1}$ as follows:
– Compute $z_{i+1} = F(z_i, \omega_i)$.
– $\phi_{\text{arith}, i+1} \leftarrow H(\text{pp}, \text{vk}, (i, z_0, z_{i+1}, \phi_{\text{acc}, i+1}))$.
– $w_{\text{arith}, i+1} \leftarrow (\text{pp}, \text{vk}, i, z_0, z_i, z_{i+1}, \omega_i, \phi_{\text{arith}, i}, \phi_{\text{inter}, i}, \phi_{\text{acc}, i}, \phi_{\text{acc}, i+1}, \pi_{\text{prime}, i}, \pi_{\text{acc}, i})$.
Compute $(\phi, w)_{\text{prime}, i+1}$ as follows:
– $((\phi, w)_{\text{inter}, i+1}, \pi_{\text{prime}, i+1}) \leftarrow \Pi_{\text{prime}}.\text{Reduce}^{\mathcal{O}}(\phi, w)_{\text{arith}, i+1}$
– Set $\phi_{\text{prime}, i+1} \leftarrow (\phi_{\text{arith}, i+1}, \phi_{\text{inter}, i+1}, \pi_{\text{prime}, i+1})$.
– Set $w_{\text{prime}, i+1} \leftarrow w_{\text{inter}, i+1}$.
Output $\pi_{\text{step}, i+1} \leftarrow ((\phi, w)_{\text{prime}, i+1}, (\phi, w)_{\text{acc}, i+1})$

V($\text{pp}, \text{vk}, F, (i, z_0, z_i), \pi_{\text{step}, i}$) :
Parse $(w_{\text{prime}, i}, w_{\text{acc}, i}) \leftarrow w_{\text{step}, i}$
Parse $(\phi_{\text{prime}, i}, \phi_{\text{acc}, i}) \leftarrow \phi_{\text{step}, i}$
Parse $(\phi_{\text{arith}, i}, \phi_{\text{inter}, i}, \pi_{\text{prime}, i}) \leftarrow \phi_{\text{prime}, i}$
Assert $\phi_{\text{arith}, i} = H(\text{pp}, \text{vk}, (i, z_0, z_i, \phi_{\text{acc}, i}))$
Output

$$b \leftarrow ((i \geq 1) \wedge (\phi_{\text{prime}, i}, w_{\text{prime}, i}) \in \mathcal{R}_{\text{prime}} \wedge (\phi_{\text{acc}, i}, w_{\text{acc}, i}) \in \mathcal{R}_{\text{acc}}).$$

Fig. 5. Protocol Π_{step} .

Theorem 2. The protocol Π_{step} in Figure 5 is an IVC scheme (Definition 4) assuming that Π_{prime} and Π_{acc} are the reductions of knowledge described in Section 5.1 and H is a collision-resistant hash function.

Proof. Completeness is proven in Section D.1.

For knowledge soundness, note that since Π_{prime} and Π_{acc} are both reductions of knowledge, there exist extractors $\text{Ext}_{\text{prime}}$ and Ext_{acc} such that

$$\begin{aligned} \mathbf{w}_{\text{arith},i-1} &\leftarrow \text{Ext}_{\text{prime}}(\mathbf{pp}_{\text{prime}}, \phi_{\text{inter},i}, \phi_{\text{arith},i-1}, \pi_{\text{prime},i}, \mathbf{w}_{\text{arith},i}) \\ (\mathbf{w}_{\text{prime},i-1}, \mathbf{w}_{\text{acc},i-1}) &\leftarrow \text{Ext}_{\text{acc}}(\mathbf{pp}_{\text{acc}}, \phi_{\text{acc},i}, \phi_{\text{acc},i-1}, \phi_{\text{prime},i-1}, \pi_{\text{acc},i}, \mathbf{w}_{\text{acc},i}), \end{aligned}$$

and for any PPT adversary $\mathcal{A}$,

$$\text{Adv}_{\Pi_{\text{prime}}, \text{Ext}_{\text{prime}}, \mathcal{A}}^{\text{rok}}(n) \leq \text{negl}(n), \quad \text{Adv}_{\Pi_{\text{acc}}, \text{Ext}_{\text{acc}}, \mathcal{A}}^{\text{rok}}(n) \leq \text{negl}(n).$$

Let $\mathcal{A}$ be an adversary against the knowledge soundness of Π_{step} . We define the IVC single-step extractor Ext_{step} that behaves as described in Fig. 6 and analyze the probability that it successfully outputs a valid instance-witness pair. For the case $i = 1$, Ext_{step} runs up to Abort Condition 4. Let F be the output of $\mathcal{A}$ and $(i, z_0, z_i, \pi_{\text{step},i})$ the output of some $\mathcal{P}^*$. We must prove that if $\mathcal{V}(\mathbf{pp}, F, (i, z_0, z_i), \pi_{\text{step},i}) = 1$ then

1. Ext_{step} does not abort (*i.e.*, it outputs a tuple $(z_{i-1}, w_{i-1}, \pi_{\text{step},i-1})$); and
2. $z_i = F(z_{i-1}, w_{i-1})$ and $z_{i-1} = z_0$ when $i = 1$, or $\mathcal{V}(\mathbf{pp}, F, (i-1, z_0, z_{i-1}), \pi_{\text{step},i-1}) = 1$ otherwise; except with negligible probability.

If Ext_{step} doesn't abort then condition (2) is satisfied:

Because Abort Condition 3 in Fig. 6 does not occur, we have $(\phi, \mathbf{w})_{\text{arith},i} \in \mathcal{R}_{\text{arith}}$, which implies $z_i^* = F(z_{i-1}, w_{i-1})$. Because Abort Condition 4 is also not satisfied, we have $z_i^* = z_i$ and thus $z_i = F(z_{i-1}, w_{i-1})$ as required.

For the case $i > 1$, we need to prove Ext_{step} outputs $(\phi, \mathbf{w})_{\text{prime},i-1} \in \mathcal{R}_{\text{prime},i-1}$ and $(\phi, \mathbf{w})_{\text{acc},i-1} \in \mathcal{R}_{\text{prime},i-1}$. Note that since Π_{acc} is a reduction of knowledge from $\mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}}$ to $\mathcal{R}_{\text{acc}}$, there exists an extractor Ext_{acc} that, on input $(\phi, \mathbf{w}, \pi)_{\text{acc},i}$ and $\phi_{\text{prime},i-1}, \phi_{\text{acc},i-1}$, outputs $\mathbf{w}_{\text{prime},i-1}$ and $\mathbf{w}_{\text{acc},i-1}$ such that $((\phi, \mathbf{w})_{\text{prime},i}, (\phi, \mathbf{w})_{\text{acc},i}) \in \mathcal{R}_{\text{prime}} \times \mathcal{R}_{\text{acc}}$ and $\mathcal{V}$ accepts.

Let us construct the inputs of Ext_{acc} :

- Since Abort Condition 1 is not satisfied, $(\phi, \mathbf{w})_{\text{acc},i} \in \mathcal{R}_{\text{acc}}$.
- Since Abort Condition 3 does not happen, $(\phi, \mathbf{w})_{\text{arith},i} \in \mathcal{R}_{\text{arith}}$ and thus $\pi_{\text{acc},i}$ is a valid proof for the reduction to $(\phi, \mathbf{w})_{\text{acc},i}$ and $\phi_{\text{acc},i-1}$ a valid instance.
- Finally, $\phi_{\text{prime},i-1}$ is built from $\phi_{\text{prime},i-1}$ and $\pi_{\text{prime},i-1}$, which are obtained from the valid $\mathbf{w}_{\text{arith},i}$ as well.

Then, except with probability $\text{Adv}_{\Pi_{\text{acc}}, \text{Ext}_{\text{acc}}, \mathcal{A}}$, if the extractor Ext_{step} does not abort, it outputs a proof that is accepted by $\mathcal{V}$.

Ext_{step} doesn't abort: We use an inductive argument.

First, note that Abort Conditions 1 and 2 do not occur at step N of the recursion because the verifier algorithm $\mathcal{V}$ asserts that $(\phi_{\text{acc},N}, \mathbf{w}_{\text{acc},N}) \in \mathcal{R}_{\text{acc}}$ and $(\phi_{\text{prime},N}, \mathbf{w}_{\text{prime},N}) \in \mathcal{R}_{\text{prime}}$.

Suppose that Abort Conditions 1, 2 do not occur at step i of the recursion. We show that: Abort Conditions 3 and 4 do not occur at step i and Abort Conditions 1 and 2 do not occur at step $i-1$ (except with small probability) when $i > 1$.

Abort Condition 3: If Abort Condition 2 do not occur, then $(\phi, \mathbf{w})_{\text{prime},i} \in \mathcal{R}_{\text{prime}}$. Hence, for $\mathbf{w}_{\text{inter},i} = \mathbf{w}_{\text{prime},i}$ and $\phi_{\text{inter}} = \Pi_{\text{prime}}.\text{ReduceInstance}(\phi_{\text{arith},i}, \pi_{\text{prime},i})$, $(\phi, \mathbf{w})_{\text{inter},i} \in \mathcal{R}_{\text{inter}}$. Since

$$\mathbf{w}_{\text{arith},i} \leftarrow \text{Ext}_{\text{prime}}(\mathbf{pp}, (\phi, \mathbf{w})_{\text{inter},i}, \phi_{\text{arith},i}, \pi_{\text{prime},i}),$$

the probability that $(\phi, \mathbf{w})_{\text{arith},i} \notin \mathcal{R}_{\text{arith}}$ is bounded by $\mathcal{A}_{\Pi_{\text{prime}}, \text{Ext}_{\text{prime}}, \mathcal{A}}^{\text{rok}}(n)$.

Abort Condition 4: Since $(\phi, \mathbf{w})_{\text{arith},i} \in \mathcal{R}_{\text{arith}}$, for $(\mathbf{pp}^*, \mathbf{vk}^*, i^*, z_0^*, z_i^*, \phi_{\text{acc},i}^*)$ extracted from $\mathbf{w}_{\text{arith},i}$, $\phi_{\text{arith}} = H(\mathbf{pp}^*, \mathbf{vk}^*, i^*, z_0^*, z_i^*, \phi_{\text{acc},i}^*)$. Also, the verifier checks $\phi_{\text{arith},i} = H(\mathbf{pp}, \mathbf{vk}, i, z_0, z_i, \phi_{\text{acc},i})$ and since H is a collision-resistant hash function, the extractor does not abort except with negligible probability.

Abort Condition 1 for $i-1$ (assuming $i > 1$): Now, we prove that if Abort Conditions 1, 2, 3, and 4 do not occur at step i , then Abort Conditions 1 and 2 do not occur at step $i-1$, except with negligible probability.

Since Ext_{acc} exists, it is enough to prove that its inputs $(\phi, \mathbf{w}, \pi)_{\text{acc},i}$, $\phi_{\text{acc},i-1}$, and $\phi_{\text{prime},i-1}$ are correctly computed.

Since Abort Condition 1 does not happen, $(\phi, \mathbf{w})_{\text{acc},i} \in \mathcal{R}_{\text{acc}}$, and for A.C 3 and 4, we have that π_{acc} is a valid proof for Π_{acc} . Furthermore, Abort Condition 4 ensures that $\phi_{\text{acc},i-1}$ contained in $\mathbf{w}_{\text{arith},i}$ is a

1. Given $\text{pp}, \text{vk}, F, (i, z_i, z_0, w_i)$
2. Parse $((\phi, \mathbf{w})_{\text{prime}, i}, (\phi, \mathbf{w})_{\text{acc}, i}) \leftarrow \pi_{\text{step}, i}$
3. Assert $(\phi, \mathbf{w})_{\text{acc}, i} \in \mathcal{R}_{\text{acc}}$ # Abort Condition 1
4. Assert $(\phi, \mathbf{w})_{\text{prime}, i} \in \mathcal{R}_{\text{prime}}$ # Abort Condition 2
5. Parse $(\phi_{\text{arith}, i}, \phi_{\text{inter}, i}, \pi_{\text{prime}}) \leftarrow \phi_{\text{prime}, i}$
6. Set $\mathbf{w}_{\text{inter}, i} \leftarrow \mathbf{w}_{\text{prime}, i}$ and extract

$$\mathbf{w}_{\text{arith}, i} \leftarrow \text{Ext}_{\text{prime}}(\text{pp}_{\text{prime}}, \phi_{\text{arith}, i}, \phi_{\text{prime}, i}, \mathbf{w}_{\text{prime}, i}, \pi_{\text{prime}, i})$$
7. Check $(\phi, \mathbf{w})_{\text{arith}, i} \in \mathcal{R}_{\text{arith}}$ # Abort Condition 3
8. Parse $(\text{pp}^*, \text{vk}^*, (i^* - 1, z_0^*, z_{i-1}^*, z_i^*, w_{i-1}), \phi_{\text{arith}, i-1}, \phi_{\text{inter}, i-1}, \phi_{\text{acc}, i-1}, \phi_{\text{acc}, i}^*, \pi_{\text{prime}, i-1}, \pi_{\text{acc}, i}) \leftarrow \mathbf{w}_{\text{arith}, i}$
9. Assert $(\text{pp}^*, \text{vk}^*, i^*, z_0^*, z_i^*, \phi_{\text{acc}, i}^*) = (\text{pp}, \text{vk}, i, z_0, z_i, \phi_{\text{acc}, i})$ # Abort Condition 4
10. Set $\phi_{\text{prime}, i-1} \leftarrow (\phi_{\text{arith}, i-1}, \phi_{\text{inter}, i-1}, \pi_{\text{prime}, i-1})$
11. Run $(\mathbf{w}_{\text{prime}, i-1}, \mathbf{w}_{\text{acc}, i-1}) \leftarrow \text{Ext}_{\text{acc}}(\text{pp}_{\text{acc}}, (\phi, \mathbf{w}, \pi)_{\text{acc}, i}, \phi_{\text{acc}, i-1}, \phi_{\text{prime}, i-1})$
12. $\pi_{\text{step}, i-1} \leftarrow ((\phi, \mathbf{w})_{\text{prime}, i-1}(\phi, \mathbf{w})_{\text{acc}, i-1})$
13. Output $(z_{i-1}, w_{i-1}, \pi_{\text{step}, i-1})$

Fig. 6. Ext_{step} , defined in the proof of Theorem 2

correctly computed instance. Finally, $\phi_{\text{prime}, i-1}$ is also constructed from $\phi_{\text{arith}, i-1}$, $\phi_{\text{inter}, i-1}$ and $\pi_{\text{prime}, i-1}$ contained in the valid witness $\mathbf{w}_{\text{arith}, i}$. Then, we have $(\phi, \mathbf{w})_{\text{prime}, i-1} \in \mathcal{R}_{\text{prime}}$ and $(\phi, \mathbf{w})_{\text{acc}, i-1} \in \mathcal{R}_{\text{acc}}$ except with probability $\text{Adv}_{\Pi_{\text{acc}}, \text{Ext}_{\text{acc}}, \mathcal{A}}$. This completes the inductive argument.

$i = 1$: The extractor runs up to Abort Condition 4. Since Abort Condition 3 does not occur, $(\phi, \mathbf{w})_{\text{arith}, 1} \in \mathcal{R}_{\text{arith}}$. Also, because Abort Condition 4 does not happen, $z_0^* = z_0$, as required. $\square$

Remark 4 (Malleability attack of Nguyen, Boneh and Setty [34]). It is important to note that an IVC proof contains an instance-witness pair $(\phi, \mathbf{w})_{\text{prime}}$ in $\mathcal{R}_{\text{prime}}$. In some cases, $\mathcal{R}_{\text{arith}}$ and $\mathcal{R}_{\text{inter}}$ are such that a pair $(\phi, \mathbf{w})_{\text{inter}}$ contains a pair $(\phi, \mathbf{w})_{\text{arith}}$. In this setting, given a proof for t steps of computation and final state z_t , a prover can “revert” to the $(t - 1)$ -th arith pair by parsing $(\phi, \mathbf{w})_{\text{inter}}$, compute an alternative state z'_t and inter pair $(\phi', \mathbf{w}')_{\text{inter}}$ and output an accepting proof for z'_t . This property has been described specifically for Nova [27] by Nguyen, Boneh and Setty [34] along with mitigations.

6 IVC over Curve Cycles

As we have seen in Section 5, one can construct IVC from two NIROKs Π_{prime} and Π_{acc} . A key aspect of the construction is to represent $\Pi_{\text{acc}}.\text{ReduceInstance}$ as an arithmetic circuit over a prime field $\mathbb{F}$ to be consumed by the input relation of Π_{prime} . Unfortunately, as noted by Ben-Sasson *et al.* [4], it is hard to “close the loop”. Indeed, the protocol Π_{prime} and the corresponding relation $\mathcal{R}_{\text{prime}}$ will often make use of binding commitments. Furthermore, to obtain an efficient Π_{acc} protocol, previous constructions use additively homomorphic commitments (except in [13]). A binding and additively homomorphic commitment scheme for elements of $\mathbb{F}$ can be efficiently instantiated by using an elliptic curve E of order $\#E = |\mathbb{F}|$. However, such a curve cannot be defined over the field $\mathbb{F}$ since that would make the discrete logarithm problem easy [38]. Therefore, while our circuits express arithmetic over $\mathbb{F}$, the outputs of Π_{prime} and Π_{acc} are expressed over a different field. Computing this *non-native* field arithmetic in our circuit can lead to larger and inefficient circuits.

Cycles of curves refer to a special kind of relationship between two elliptic curves, say $E^{(\text{ying})}$ and $E^{(\text{yang})}$, where $E^{(\text{ying})}$ is of order $|\mathbb{F}^{(\text{ying})}|$ and defined over a field $\mathbb{F}^{(\text{yang})}$, and $E^{(\text{yang})}$ is of order $|\mathbb{F}^{(\text{yang})}|$ and defined over a field $\mathbb{F}^{(\text{ying})}$. Thus, a constraint system defined over $\mathbb{F}^{(\text{ying})}$ can *natively* express statements about points on $E^{(\text{yang})}$ and vice versa.

Previous work [4, 9, 28] exploits this structure to “close the loop” and construct efficient IVC while minimizing the number of non-native arithmetic constraints. The most prevalent approach is that of Ben-Sasson *et al.* [4]. The idea is to run the IVC prover over $(\mathbb{F}^{(\text{ying})}, E^{(\text{ying})})$ as in our warm up; but rather than

feeding the output straight into the next cycle, the prover runs a “dummy” cycles over $(\mathbb{F}^{(\text{yang})}, E^{(\text{yang})})$. The dummy cycle has the effect of resetting the IVC to a state where it can be consumed by the original circuit over $\mathbb{F}^{(\text{ying})}$. Note however that lifting a scheme from the single-curve setting to the curve-cycle setting is highly non-trivial and can lead to critical soundness bugs, as studied by Nguyen, Boneh and Setty [34].

In this section, we show how to adapt the generic single-curve protocol of Section 5 to the curve-cycle setting in line with the techniques of Ben-Sasson *et al.* [4].

Remark 5 (The CycleFold [28] approach). Kothapalli and Setty [28] introduce CycleFold, a different approach to “closing the loop”. While we observed that Π_{prime} (and consequently $\mathcal{R}_{\text{prime}}$) rely on elliptic curve commitments, the “closing the loop” challenge only arises if Π_{acc} .ReduceInstance cannot be natively expressed as constraints over the initial field. CycleFold solves this by designing Π_{acc} to *internally* use cycles of curves such that its ReduceInstance algorithm can be efficiently expressed over that native field of the circuit in Π_{prime} .

6.1 Protocol Overview

Following the approach of Ben-Sasson *et al.* [4], we define the same 4 relations and NIROKs as in Section 5. However, each relation is defined for both $(\mathbb{F}^{(\text{ying})}, E^{(\text{ying})})$ and $(\mathbb{F}^{(\text{yang})}, E^{(\text{yang})})$:

- The protocol $\Pi_{\text{prime}}^{(\text{ying})}$ (and resp. $\Pi_{\text{prime}}^{(\text{yang})}$) checks satisfaction of constraints defined over values in $\mathbb{F}^{(\text{ying})}$ (resp. $\mathbb{F}^{(\text{yang})}$) and outputs values in $\mathbb{F}^{(\text{ying})}$ (resp. $\mathbb{F}^{(\text{yang})}$) and commitments in $E^{(\text{ying})}$ (resp. $E^{(\text{yang})}$)

$$\Pi_{\text{prime}}^{(\text{ying})} : \mathcal{R}_{\text{arith}}^{(\text{ying})} \rightarrow \mathcal{R}_{\text{inter}}^{(\text{ying})}, \quad \Pi_{\text{prime}}^{(\text{yang})} : \mathcal{R}_{\text{arith}}^{(\text{yang})} \rightarrow \mathcal{R}_{\text{inter}}^{(\text{yang})}$$

- The protocol $\Pi_{\text{acc}}^{(\text{ying})}$ (and resp. $\Pi_{\text{acc}}^{(\text{yang})}$) accumulates instance-witness pairs of the $(\mathcal{R}_{\text{prime}}^{(\text{ying})} \times \mathcal{R}_{\text{acc}}^{(\text{ying})})$ (resp. $(\mathcal{R}_{\text{prime}}^{(\text{yang})} \times \mathcal{R}_{\text{acc}}^{(\text{yang})})$) into a single instance-witness pair of $\mathcal{R}_{\text{acc}}^{(\text{ying})}$ (resp. $\mathcal{R}_{\text{acc}}^{(\text{yang})}$)

$$\Pi_{\text{acc}}^{(\text{ying})} : (\mathcal{R}_{\text{prime}}^{(\text{ying})} \times \mathcal{R}_{\text{acc}}^{(\text{ying})}) \rightarrow \mathcal{R}_{\text{acc}}^{(\text{ying})}$$

$$\Pi_{\text{acc}}^{(\text{yang})} : (\mathcal{R}_{\text{prime}}^{(\text{yang})} \times \mathcal{R}_{\text{acc}}^{(\text{yang})}) \rightarrow \mathcal{R}_{\text{acc}}^{(\text{yang})}$$

As a result, the algorithms $\Pi_{\text{acc}}^{(\text{ying})}$.ReduceInstance (and resp. $\Pi_{\text{acc}}^{(\text{yang})}$.ReduceInstance) primarily perform operations on elements of $E^{(\text{ying})}$ (resp. $E^{(\text{yang})}$). As noted, these are efficiently expressed as arithmetic in $\mathbb{F}^{(\text{yang})}$ (resp. $\mathbb{F}^{(\text{ying})}$).

We also define two arithmetization-friendly hash functions [24] $H^{(\text{ying})}$ and $H^{(\text{yang})}$ for the fields $\mathbb{F}^{(\text{ying})}$ and $\mathbb{F}^{(\text{yang})}$ respectively.

IVC proof over cycles. Assume without loss of generality that an IVC cycles start with a circuit defined over $\mathbb{F}^{(\text{ying})}$. An IVC proof over such a curve cycle is defined as:

$$\pi_i = \left[(\phi, \mathbf{w})_{\text{prime}, i}^{(\text{yang})}, (\phi, \mathbf{w})_{\text{acc}, i}^{(\text{ying})}, (\phi, \mathbf{w})_{\text{acc}, i}^{(\text{yang})} \right]$$

The pair $(\phi, \mathbf{w})_{\text{prime}, i}^{(\text{yang})}$ attests to the correctness of the i -th execution of the **yang** part of the IVC cycle, $(\phi, \mathbf{w})_{\text{acc}, i}^{(\text{yang})}$ attest to all previous **yang** executions (1 to $i - 1$), and $(\phi, \mathbf{w})_{\text{acc}, i}^{(\text{ying})}$ attests to all executions of the **ying** part of the IVC cycle (1 to i).

Prover over cycles. The prover’s work is similar to the prover of Section 5, with the difference that the “acc-arith-prime” loop is now doubled and works over two circuits $\mathcal{C}_{\text{step}}^{(\text{ying})}$ and $\mathcal{C}_{\text{step}}^{(\text{yang})}$. At a high level, the prover runs as follows:

1. **acc**^(yang): run $\Pi_{\text{acc}}^{(\text{yang})}$ on the **yang** proof elements to obtain $(\phi, \mathbf{w})_{\text{acc}, i+1}^{(\text{yang})}$.
2. **arith**^(ying): collect $(\phi, \mathbf{w})_{\text{arith}, i+1}^{(\text{ying})}$ that satisfy the circuit $\mathcal{C}_{\text{step}}^{(\text{ying})}$.
3. **prime**^(ying): run $\Pi_{\text{prime}}^{(\text{ying})}$ on the **ying** circuit assignment from step 2 to obtain $(\phi, \mathbf{w})_{\text{prime}, i+1}^{(\text{ying})}$.

4. $\text{acc}^{(\text{ying})}$: run $\Pi_{\text{acc}}^{(\text{ying})}$ on the ying accumulator (from the input proof π_i) and the fresh $(\phi, \mathbf{w})_{\text{prime}, i+1}^{(\text{ying})}$ to obtain $(\phi, \mathbf{w})_{\text{acc}, i+1}^{(\text{ying})}$.
5. $\text{arith}^{(\text{yang})}$: collect $(\phi, \mathbf{w})_{\text{arith}, i+1}^{(\text{yang})}$ that satisfy the circuit $\mathcal{C}_{\text{step}}^{(\text{yang})}$.
6. $\text{prime}^{(\text{yang})}$: run $\Pi_{\text{prime}}^{(\text{yang})}$ on the yang circuit assignment from step 5 to obtain $(\phi, \mathbf{w})_{\text{prime}, i+1}^{(\text{yang})}$.

Circuits. As in Section 5, $\mathcal{C}_{\text{step}}^{(\text{ying})}$ enforces that: the circuit's variables are correctly linked to the i -th and $(i+1)$ -th statements and proofs; the function F is correctly iterated; the yang accumulator is correctly computed from the yang proof elements. An additional circuit $\mathcal{C}_{\text{step}}^{(\text{yang})}$ is introduced to allow for the “dummy cycle”. Note that since our hash functions are now field-specific, we follow the approach of Nguyen, Boneh and Setty [34], and defer the verification of a ying hash until the next execution of a ying circuit (and similarly for yang hashes). One side effect of this optimisation is that instances of $\mathcal{R}_{\text{arith}}^{(\text{ying})}$ and $\mathcal{R}_{\text{arith}}^{(\text{yang})}$ are now composed of two hash values, as opposed to one in the single-curve setting.

6.2 Full Protocol

We now proceed to formally describe an IVC protocol over a cycle of curves that makes black-box use of the reductions $\Pi_{\text{prime}}^{(\text{yang})}$, $\Pi_{\text{acc}}^{(\text{ying})}$, $\Pi_{\text{prime}}^{(\text{ying})}$, $\Pi_{\text{acc}}^{(\text{yang})}$.

Initial Setup. We first describe the $\text{Setup}(n, s)$ procedure for a security parameter n and a poly-size bound $s \in \mathbb{N}$:

- Run $\mathcal{CG}(n)$ and output $\mathbb{G}^{(\text{ying})}, \mathbb{G}^{(\text{yang})}$ of prime orders $p^{(\text{ying})}, p^{(\text{yang})} > 2^n$. Let $\mathbb{F}^{(\text{ying})}$ denote the prime order field $\mathbb{F}_{p^{(\text{ying})}}$ and $\mathbb{F}^{(\text{yang})}$ the prime order field $\mathbb{F}_{p^{(\text{yang})}}$.
- Sample generators $\vec{g} \in (\mathbb{G}^{(\text{ying})})^s$ and $\vec{h} \in (\mathbb{G}^{(\text{yang})})^s$.
- Sample collision-resistant hash functions $H^{(\text{ying})}$ and $H^{(\text{yang})}$.
- Run the relevant KeyGen algorithms for the NIROKs $\Pi_{\text{prime}}^{(\text{yang})}$, $\Pi_{\text{acc}}^{(\text{ying})}$, $\Pi_{\text{prime}}^{(\text{ying})}$, $\Pi_{\text{acc}}^{(\text{yang})}$, and output $\text{pp} = (\text{pp}_{\text{prime}}^{(\text{yang})}, \text{pp}_{\text{acc}}^{(\text{ying})}, \text{pp}_{\text{prime}}^{(\text{ying})}, \text{pp}_{\text{acc}}^{(\text{yang})})$.
- output pp collecting the above parameters.

Independently, we assume the existence of an open-and-sign random oracle $\mathcal{O}_{\text{osRO}}$ with corresponding verification key vk ⁴. $\mathcal{O}_{\text{osRO}}^{(\text{ying})}$ denotes for the open-and-sign oracle initialized with $(\mathbb{G}^{(\text{ying})}, p^{(\text{ying})}, \vec{g})$ and $\mathcal{O}_{\text{osRO}}^{(\text{yang})}$ for the open-and-sign oracle initialized with $(\mathbb{G}^{(\text{yang})}, p^{(\text{yang})}, \vec{h})$.

The IVC Prover. *Case $i = 0$.* $\mathcal{P}$ is given as input $(\text{pp}, \mathbb{F}, (0, z_0), \omega_0, \pi_0 := \perp)$ and outputs π_1 :

0. $\phi_{\text{arith}, 0}^{(\text{yang})} = (H^{(\text{ying})}(\text{pp}, \text{vk}, 0, z_0, z_0, \phi_{\text{acc}, \perp}^{(\text{yang})}), H^{(\text{yang})}(\text{pp}, \text{vk}, 0, \phi_{\text{acc}, \perp}^{(\text{ying})}))$.
1. Set $(\phi, \mathbf{w})_{\text{acc}, 1}^{(\text{yang})} = (\phi, \mathbf{w})_{\text{acc}, \perp}^{(\text{yang})}$.
2. Generate $(\phi, \mathbf{w})_{\text{arith}, 1}^{(\text{ying})}$ as follows:
 - Compute $z_1 = F(z_0, \omega_0)$.
 - Set $\phi_{\text{arith}, 1}^{(\text{ying})}.h_0 = \phi_{\text{arith}, 0}^{(\text{yang})}.h_1$.
 - Set $\phi_{\text{arith}, 1}^{(\text{ying})}.h_1 = H^{(\text{ying})}(\text{pp}, \text{vk}, 1, z_0, z_1, \phi_{\text{acc}, \perp}^{(\text{yang})})$.
 - Set $\mathbf{w}_{\text{arith}, 1}^{(\text{ying})}$ to be:

$$\mathbf{w}_{\text{arith}, 1}^{(\text{ying})} = \left(\begin{array}{l} \text{pp}, \text{vk}, 0, z_0, z_0, z_1, \omega_0, \\ \phi_{\text{arith}, 0}^{(\text{yang})}, \perp, \phi_{\text{acc}, \perp}^{(\text{yang})}, \phi_{\text{acc}, \perp}^{(\text{yang})}, \\ \perp, \perp \end{array} \right).$$

3. Derive $(\phi, \mathbf{w})_{\text{prime}, 1}^{(\text{ying})}$ as follows:
 - $((\phi, \mathbf{w})_{\text{inter}, 1}^{(\text{ying})}, \pi_{\text{prime}, 1}^{(\text{ying})}) \leftarrow \Pi_{\text{prime}}^{(\text{ying})}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}^{(\text{ying})}}(\text{pp}, (\phi, \mathbf{w})_{\text{arith}, 1}^{(\text{ying})})$

⁴ We set vk to be the same for ying and yang, but the framework can be generalized to work with two different keys.

$\mathcal{C}_{\text{step}}^{(\text{ying})}(\phi_{\text{arith},i+1}^{(\text{ying})}, \mathbf{w}_{\text{arith},i+1}^{(\text{ying})}) \rightarrow \{0, 1\}$:

Parse the instance $\phi_{\text{arith},i+1}^{(\text{ying})} = (h_0, h_1)$.

Parse the witness:

$$\mathbf{w}_{\text{arith},i+1}^{(\text{ying})} = \left(\begin{array}{l} \text{pp}, \text{vk}, i, z_0, z_i, z_{i+1}, \omega_i, \\ \phi_{\text{arith},i}^{(\text{yang})}, \phi_{\text{inter},i}^{(\text{yang})}, \phi_{\text{acc},i}^{(\text{yang})}, \phi_{\text{acc},i+1}^{(\text{yang})}, \\ \pi_{\text{prime},i}^{(\text{yang})}, \pi_{\text{acc}}^{(\text{yang})} \end{array} \right)$$

Set $\phi_{\text{prime},i}^{(\text{yang})} = (\phi_{\text{arith},i}^{(\text{yang})}, \phi_{\text{inter},i}^{(\text{yang})}, \pi_{\text{prime},i}^{(\text{yang})})$.

Parse the instance $\phi_{\text{arith},i}^{(\text{yang})} = (h'_0, h'_1)$.

Accept if all the conditions below are satisfied:

1. Check the h'_0 hash (yang input): $h'_0 = H^{(\text{ying})}(\text{pp}, \text{vk}, i, z_0, z_i, \phi_{\text{acc},i}^{(\text{yang})})$.
2. Carry the h'_1 hash from the yang circuit (yang output): $h_0 = h'_1$.
3. Check one iteration of the step function: $z_{i+1} = F(z_i, \omega_i)$.
4. **If** $i = 0$, assert that $\phi_{\text{acc},i+1}^{(\text{yang})} = \phi_{\text{acc},\perp}^{(\text{yang})}$ and $z_i = z_0$;
Else, check the instance reduction of $\Pi_{\text{acc}}^{(\text{yang})}$:

$$\phi_{\text{acc},i+1}^{(\text{yang})} = \Pi_{\text{acc}}^{(\text{yang})}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_{\text{prime},i}^{(\text{yang})}, \phi_{\text{acc},i}^{(\text{yang})}, \pi_{\text{acc}}^{(\text{yang})})$$

5. Check the ying hash for circuit output:

$$h_1 = H^{(\text{ying})}(\text{pp}, \text{vk}, i+1, z_0, z_{i+1}, \phi_{\text{acc},i+1}^{(\text{yang})})$$

$\mathcal{C}_{\text{step}}^{(\text{yang})}(\phi_{\text{arith},i+1}^{(\text{yang})}, \mathbf{w}_{\text{arith},i+1}^{(\text{yang})}) \rightarrow \{0, 1\}$:

Parse the instance $\phi_{\text{arith},i+1}^{(\text{yang})} = (h_0, h_1)$.

Parse the witness:

$$\mathbf{w}_{\text{arith},i+1}^{(\text{yang})} = \left(\begin{array}{l} \text{pp}, \text{vk}, i, \\ \phi_{\text{arith},i+1}^{(\text{ying})}, \phi_{\text{inter},i+1}^{(\text{ying})}, \phi_{\text{acc},i}^{(\text{ying})}, \phi_{\text{acc},i+1}^{(\text{ying})}, \\ \pi_{\text{prime},i+1}^{(\text{ying})}, \pi_{\text{acc}}^{(\text{ying})} \end{array} \right)$$

Set $\phi_{\text{prime},i+1}^{(\text{ying})} = (\phi_{\text{arith},i+1}^{(\text{ying})}, \phi_{\text{inter},i+1}^{(\text{ying})}, \pi_{\text{prime},i+1}^{(\text{ying})})$.

Parse the instance $\phi_{\text{arith},i+1}^{(\text{ying})} = (h'_0, h'_1)$.

Accept if all the conditions below are satisfied:

1. Check the h'_0 hash (ying input): $h'_0 = H^{(\text{yang})}(\text{pp}, \text{vk}, i, \phi_{\text{acc},i}^{(\text{ying})})$.
2. Carry the h'_1 hash from the ying circuit (ying output): $h_0 = h'_1$.
3. **If** $i = 0$, check that $\phi_{\text{acc},i+1}^{(\text{ying})} = \phi_{\text{acc},\perp}^{(\text{ying})}$;
Else, check the instance reduction of $\Pi_{\text{acc}}^{(\text{ying})}$:

$$\phi_{\text{acc},i+1}^{(\text{ying})} = \Pi_{\text{acc}}^{(\text{ying})}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_{\text{prime},i+1}^{(\text{ying})}, \phi_{\text{acc},i}^{(\text{ying})}, \pi_{\text{acc}}^{(\text{ying})})$$

4. Check the yang hash for circuit output:

$$h_1 = H^{(\text{yang})}(\text{pp}, \text{vk}, i+1, \phi_{\text{acc},i+1}^{(\text{ying})})$$

Fig. 7. Circuits $\mathcal{C}_{\text{step}}^{(\text{ying})}$ and $\mathcal{C}_{\text{step}}^{(\text{yang})}$ to be evaluated at each IVC step.

- Set $\phi_{\text{prime},1}^{(\text{ying})} = (\phi_{\text{inter},1}^{(\text{ying})}, \pi_{\text{prime},1}^{(\text{ying})})$ and $w_{\text{prime},1}^{(\text{ying})} = w_{\text{inter},1}^{(\text{ying})}$.
4. Accumulate $(\phi, w)_{\text{prime},1}^{(\text{ying})}$ with $(\phi, w)_{\text{acc},\perp}^{(\text{ying})}$ to obtain $(\phi, w)_{\text{acc},1}^{(\text{ying})}$:

$$((\phi, w)_{\text{acc},1}^{(\text{ying})}, \pi_{\text{acc}}^{(\text{ying})}) \leftarrow \Pi_{\text{acc}}^{(\text{ying})}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}^{(\text{ying})}}(\text{pp}, (\phi, w)_{\text{prime},1}^{(\text{ying})}, (\phi, w)_{\text{acc},\perp}^{(\text{ying})}).$$

5. Generate $(\phi, w)_{\text{arith},1}^{(\text{yang})}$ as follows:

- Set $\phi_{\text{arith},1}^{(\text{yang})}.h_0 = \phi_{\text{arith},1}^{(\text{ying})}.h_1$.
- Set $\phi_{\text{arith},1}^{(\text{yang})}.h_1 = H^{(\text{yang})}(\text{pp}, \text{vk}, 1, \phi_{\text{acc},1}^{(\text{ying})})$.
- Set $w_{\text{arith},1}^{(\text{yang})}$ to be

$$w_{\text{arith},1}^{(\text{yang})} = \left(\text{pp}, \text{vk}, 0, \phi_{\text{arith},1}^{(\text{ying})}, \phi_{\text{inter},1}^{(\text{ying})}, \phi_{\text{acc},\perp}^{(\text{yang})}, \phi_{\text{acc},1}^{(\text{ying})}, \pi_{\text{prime},1}^{(\text{ying})} \leftarrow \Pi_{\text{prime}}^{(\text{ying})}, \pi_{\text{acc}}^{(\text{ying})} \right)$$

6. Derive $(\phi, w)_{\text{prime},1}^{(\text{yang})}$:

- Run $\Pi_{\text{prime}}^{(\text{yang})}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}^{(\text{yang})}}$:

$$((\phi, w)_{\text{inter},1}^{(\text{yang})}, \pi_{\text{prime},1}^{(\text{yang})}) \leftarrow \Pi_{\text{prime}}^{(\text{yang})}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}^{(\text{yang})}}(\text{pp}, (\phi, w)_{\text{arith},1}^{(\text{yang})}).$$

- Collect instance and witness terms:

$$\begin{aligned} \phi_{\text{prime},1}^{(\text{yang})} &= (\phi_{\text{arith},1}^{(\text{yang})}, \phi_{\text{inter},1}^{(\text{yang})}, \pi_{\text{prime},1}^{(\text{yang})}), \\ w_{\text{prime},1}^{(\text{yang})} &= w_{\text{inter},1}^{(\text{yang})}. \end{aligned}$$

7. Output $\pi_1 = ((\phi, w)_{\text{prime},1}^{(\text{yang})}, (\phi, w)_{\text{acc},1}^{(\text{ying})}, (\phi, w)_{\text{acc},1}^{(\text{yang})})$.

Recursive case, $i \geq 1$. $\mathcal{P}$ is given as input $(\text{pp}, F, (i, z_0, z_i), \omega_i, \pi_i)$ and must output π_{i+1} . The procedure is as follows:

1. Parse π_i as $(\phi, w)_{\text{prime},i}^{(\text{yang})}$, $(\phi, w)_{\text{acc},i}^{(\text{ying})}$ and $(\phi, w)_{\text{acc},i}^{(\text{yang})}$.
2. Compute $(\phi, w)_{\text{acc},i+1}^{(\text{yang})}$ and $\pi_{\text{acc}}^{(\text{yang})}$ as the acc of the incoming yang pairs:

$$((\phi, w)_{\text{acc},i+1}^{(\text{yang})}, \pi_{\text{acc}}^{(\text{yang})}) \leftarrow \Pi_{\text{acc}}^{(\text{yang})}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}^{(\text{yang})}}(\text{pp}, (\phi, w)_{\text{prime},i}^{(\text{yang})}, (\phi, w)_{\text{acc},i}^{(\text{yang})})$$

3. Compute $(\phi, w)_{\text{arith},i+1}^{(\text{ying})}$ that satisfy the circuit $\mathcal{C}_{\text{step}}^{(\text{ying})}$. Amongst other constraints, this instance-witness pair attests to one execution of the step function and the correctness of the reduction of Step 2 above:

- Parse $\phi_{\text{prime},i}^{(\text{yang})}$ as $(\phi_{\text{arith},i}^{(\text{yang})}, \phi_{\text{inter},i}^{(\text{yang})}, \pi_{\text{prime},i}^{(\text{yang})})$.
- Compute $z_{i+1} = F(z_i, \omega_i)$.
- Set $\phi_{\text{arith},i+1}^{(\text{ying})} := (h_0, h_1)$, where

$$\begin{cases} h_0 = \phi_{\text{arith},i}^{(\text{yang})}.h_1 \\ h_1 = H^{(\text{ying})}(\text{pp}, \text{vk}, i+1, z_0, z_{i+1}, \phi_{\text{acc},i+1}^{(\text{yang})}) \end{cases}$$

$$- w_{\text{arith},i+1}^{(\text{ying})} := \left(\text{pp}, \text{vk}, i, z_0, z_i, z_{i+1}, \omega_i, \phi_{\text{arith},i}^{(\text{yang})}, \phi_{\text{inter},i}^{(\text{yang})}, \phi_{\text{acc},i}^{(\text{yang})}, \phi_{\text{acc},i+1}^{(\text{yang})}, \pi_{\text{prime},i}^{(\text{yang})}, \pi_{\text{acc}}^{(\text{yang})} \right)$$

4. Compute $(\phi, w)_{\text{prime},i+1}^{(\text{ying})}$ by running $\Pi_{\text{prime}}^{(\text{ying})}$:

$$- ((\phi, w)_{\text{inter},i+1}^{(\text{ying})}, \pi_{\text{prime},i+1}^{(\text{ying})}) \leftarrow \Pi_{\text{prime}}^{(\text{ying})}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}^{(\text{ying})}}(\text{pp}, (\phi, w)_{\text{arith},i+1}^{(\text{ying})})$$

- Set $\phi_{\text{prime},i+1}^{(\text{ying})} := (\phi_{\text{arith},i+1}^{(\text{ying})}, \phi_{\text{inter},i+1}^{(\text{ying})}, \pi_{\text{prime},i+1}^{(\text{ying})})$ and $w_{\text{prime},i+1}^{(\text{ying})} := w_{\text{inter},i+1}^{(\text{ying})}$.

5. Compute $(\phi, \mathbf{w})_{\text{acc}, i+1}^{(\text{ying})}$ and $\pi_{\text{acc}, i+1}^{(\text{ying})}$ as the accumulation of $(\phi, \mathbf{w})_{\text{prime}, i+1}^{(\text{ying})}$ computed above with the ying accumulator available in π_i :

$$((\phi, \mathbf{w})_{\text{acc}, i+1}^{(\text{ying})}, \pi_{\text{acc}, i+1}^{(\text{ying})}) \leftarrow \Pi_{\text{acc}}^{(\text{ying})}.\text{Reduce}_{\text{osRO}}^{(\text{ying})}(\text{pp}, (\phi, \mathbf{w})_{\text{prime}, i+1}^{(\text{ying})}, (\phi, \mathbf{w})_{\text{acc}, i}^{(\text{ying})})$$

6. Compute $(\phi, \mathbf{w})_{\text{arith}, i+1}^{(\text{yang})}$ that satisfy the circuit $\mathcal{C}_{\text{step}}^{(\text{yang})}$. Amongst other constraints, this instance-witness pair attests to the correctness Step 5 :
 - Set $\phi_{\text{arith}, i+1}^{(\text{yang})} := (h_0, h_1)$ where:

$$\begin{cases} h_0 = \phi_{\text{arith}, i+1}^{(\text{ying})}.h_1 \\ h_1 = H^{(\text{yang})}(\text{pp}, \text{vk}, i+1, \phi_{\text{acc}, i+1}^{(\text{ying})}) \end{cases}$$

$$- \mathbf{w}_{\text{arith}, i+1}^{(\text{yang})} := \begin{pmatrix} \text{pp}, \text{vk}, i, \\ \phi_{\text{arith}, i+1}^{(\text{ying})}, \phi_{\text{inter}, i+1}^{(\text{ying})}, \phi_{\text{acc}, i}^{(\text{ying})}, \phi_{\text{acc}, i+1}^{(\text{ying})}, \\ \pi_{\text{prime}, i+1}^{(\text{ying})}, \pi_{\text{acc}}^{(\text{ying})} \end{pmatrix}$$

7. Compute $(\phi, \mathbf{w})_{\text{prime}, i+1}^{(\text{yang})}$ by running $\Pi_{\text{prime}}^{(\text{yang})}$:

$$- ((\phi, \mathbf{w})_{\text{inter}, i+1}^{(\text{yang})}, \pi_{\text{prime}, i+1}^{(\text{yang})}) \leftarrow \Pi_{\text{prime}}^{(\text{yang})}.\text{Reduce}_{\text{osRO}}^{(\text{yang})}(\text{pp}, (\phi, \mathbf{w})_{\text{arith}, i+1}^{(\text{yang})})$$

$$- \text{Set } \phi_{\text{prime}, i+1}^{(\text{yang})} := (\phi_{\text{arith}, i+1}^{(\text{yang})}, \phi_{\text{inter}, i+1}^{(\text{yang})}, \pi_{\text{prime}, i+1}^{(\text{yang})}) \text{ and } \mathbf{w}_{\text{prime}, i+1}^{(\text{yang})} := \mathbf{w}_{\text{inter}, i+1}^{(\text{yang})}.$$

8. Output $\pi_{i+1} = [(\phi, \mathbf{w})_{\text{prime}, i+1}^{(\text{yang})}, (\phi, \mathbf{w})_{\text{acc}, i+1}^{(\text{ying})}, (\phi, \mathbf{w})_{\text{acc}, i+1}^{(\text{yang})}]$

The IVC Verifier. The verifier $\mathcal{V}$ receives the oracles $\mathcal{O}_{\text{osRO}}^{(\text{ying})}$ and $\mathcal{O}_{\text{osRO}}^{(\text{yang})}$, verification key vk , public parameters pp , a statement (F, i, z_0, z_i) and a proof π_i , and outputs a decision $\{0, 1\}$:

1. Parse π_i as $(\phi, \mathbf{w})_{\text{prime}, i}^{(\text{yang})}$, $(\phi, \mathbf{w})_{\text{acc}, i}^{(\text{ying})}$ and $(\phi, \mathbf{w})_{\text{acc}, i}^{(\text{yang})}$.
2. Parse $\phi_{\text{arith}, i}^{(\text{yang})} = (h_0, h_1)$ out of $\phi_{\text{prime}, i}^{(\text{yang})}$.
3. Accept (return 1) if and only if all the following conditions are satisfied:
 - (a) $i \geq 1$, and
 - (b) $h_0 = H^{(\text{ying})}(\text{pp}, \text{vk}, i, z_0, z_i, \phi_{\text{acc}, i}^{(\text{yang})})$, and
 - (c) $h_1 = H^{(\text{yang})}(\text{pp}, \text{vk}, i, \phi_{\text{acc}, i}^{(\text{ying})})$, and
 - (d) $\pi_i \in (\mathcal{R}_{\text{prime}}^{(\text{yang})} \times \mathcal{R}_{\text{acc}}^{(\text{ying})} \times \mathcal{R}_{\text{acc}}^{(\text{yang})})$.

Theorem 3 (IVC Security). *If $\Pi_{\text{prime}}^{(\text{ying})}$, $\Pi_{\text{prime}}^{(\text{yang})}$, $\Pi_{\text{acc}}^{(\text{ying})}$, and $\Pi_{\text{acc}}^{(\text{yang})}$ are NIROKs in the osROM and $H^{(\text{ying})}, H^{(\text{yang})}$ are collision-resistant hash functions, then the IVC construction above is complete and knowledge-sound in the osROM.*

Proof Overview. A full proof is given in Section D.2. We prove knowledge-soundness by describing the single-step extractor Ext_{step} . We then show that this extractor outputs $z_{i-1}, \omega_{i-1}, \pi_{i-1}$ such that $z_i = F(z_{i-1}, \omega_{i-1})$ and the proof π_{i-1} is an accepting proof for the statement $(F, (i, z_0, z_{i-1}))$ — as required by the knowledge-soundness definition.

At a high level, the single-step extractor Ext_{step} will use the extractors for $\Pi_{\text{prime}}^{(\text{yang})}$, $\Pi_{\text{acc}}^{(\text{ying})}$, $\Pi_{\text{prime}}^{(\text{ying})}$ and $\Pi_{\text{acc}}^{(\text{yang})}$ (in that order) to “undo” the actions taken by the IVC prover. It proceeds in two nearly symmetrical phases:

In the first half, Ext_{step} runs the extractor for $\Pi_{\text{prime}}^{(\text{yang})}$ on $(\phi, \mathbf{w})_{\text{prime}, i}^{(\text{yang})}$ (available in the proof π_i) to obtain a pair $(\phi, \mathbf{w})_{\text{arith}, i}^{(\text{yang})}$ that satisfies the circuit $\mathcal{C}_{\text{step}}^{(\text{yang})}$. Parsing this pair, Ext_{step} obtains the necessary material to run the $\Pi_{\text{acc}}^{(\text{ying})}$ extractor on $(\phi, \mathbf{w})_{\text{acc}, i}^{(\text{ying})}$ (available in the proof π_i). The output of this extraction is a previous ying accumulator $(\phi, \mathbf{w})_{\text{acc}, i-1}^{(\text{ying})}$ and $(\phi, \mathbf{w})_{\text{prime}, i}^{(\text{ying})}$ a primed version of the inputs to $\mathcal{C}_{\text{step}}^{(\text{ying})}$. See Steps 1–9 in Section D.2 for the extractor description.

The above process is repeated for the ying and yang counterparts, running on the freshly extracted $(\phi, \mathbf{w})_{\text{prime}, i}^{(\text{ying})}$ and the yang accumulator $(\phi, \mathbf{w})_{\text{acc}, i}^{(\text{yang})}$ (available in the proof π_i). The result of this process is to obtain the pairs $(\phi, \mathbf{w})_{\text{prime}, i-1}^{(\text{yang})}$ and $(\phi, \mathbf{w})_{\text{acc}, i-1}^{(\text{yang})}$. Additionally, since $\mathcal{C}_{\text{step}}^{(\text{ying})}$ enforces the correct execution

of the step function, extracting from $(\phi, w)_{\text{prime}, i}^{(\text{ying})}$ will give a previous state z_{i-1} and witness ω_{i-1} . See steps 10-18 in Section D.2 for the formal extractor description.

If any of the calls to the inner extractors fail with non-negligible probability, then we can construct an explicit adversary against the knowledge soundness one of the inner NIRoKs. If any of the extracted values are not consistent with the original statement and proof $(F, (i, z_0, z_i), \pi_i)$, we can construct an explicit collision finder for one of the hash functions $H^{(\text{ying})}$ or $H^{(\text{yang})}$. Therefore, the extractor succeeds with probability greater than $1 - \text{negl}(n)$.

6.3 HyperNova over cycles of curves

Theorem 4. *When instantiating Π_{acc} with the HyperNova folding scheme in Figure 3 over a cycle of curves $(E^{(\text{ying})}, E^{(\text{yang})})$, and two collision resistant hash functions $H^{(\text{ying})}, H^{(\text{yang})}$, we have a complete and knowledge-sound IVC scheme that supports up to polynomial-depth levels of recursion in the osROM.*

Proof. We instantiate $\mathcal{R}_{\text{arith}}, \mathcal{R}_{\text{inter}}, \Pi_{\text{prime}}, \mathcal{R}_{\text{prime}}, \mathcal{R}_{\text{acc}}$, and Π_{acc} as in Example 1. Π_{prime} is the reduction from $\mathcal{R}_{\text{CCS}}$ to $\mathcal{R}_{\text{cCCS}}$ as presented in the example, whereas $\Pi_{\text{acc}} : \mathcal{R}_{\text{cCCS}} \rightarrow \mathcal{R}_{\text{lcCCS}}$ is described in Figure 3 and proven secure in the osROM in Theorem 8. From Theorem 3 we have that the IVC construction is complete and knowledge-sound in the osROM. Finally, as shown in the proof of Theorem 8, Ext_{acc} operates in a straight-line (non-rewinding) fashion, and thus the IVC scheme admits polynomial steps. $\square$

References

- [1] A. Arun, S. T. V. Setty, and J. Thaler. “Jolt: SNARKs for Virtual Machines via Lookups”. In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI*. Ed. by M. Joye and G. Leander. Vol. 14656. Lecture Notes in Computer Science. Springer, 2024, pp. 3–33. DOI: [10.1007/978-3-031-58751-1_1](https://doi.org/10.1007/978-3-031-58751-1_1). URL: https://doi.org/10.1007/978-3-031-58751-1_1.
- [2] A. Barbara, A. Chiesa, and Z. Guan. *Relativized Succinct Arguments in the ROM Do Not Exist*. Cryptology ePrint Archive, Paper 2024/728. <https://eprint.iacr.org/2024/728>. 2024. URL: <https://eprint.iacr.org/2024/728>.
- [3] E. Ben-Sasson, A. Chiesa, and N. Spooner. “Interactive Oracle Proofs”. In: *Theory of Cryptography - 14th International Conference, TCC 2016-B, Beijing, China, October 31 - November 3, 2016, Proceedings, Part II*. Ed. by M. Hirt and A. D. Smith. Vol. 9986. Lecture Notes in Computer Science. 2016, pp. 31–60. DOI: [10.1007/978-3-662-53644-5_2](https://doi.org/10.1007/978-3-662-53644-5_2). URL: https://doi.org/10.1007/978-3-662-53644-5_2.
- [4] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. “Scalable Zero Knowledge Via Cycles of Elliptic Curves”. In: *Algorithmica* 79 (2014), pp. 1102–1160. URL: <https://api.semanticscholar.org/CorpusID:8825569>.
- [5] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. “Recursive composition and bootstrapping for SNARKs and proof-carrying data”. In: *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*. 2013, pp. 111–120.
- [6] D. Boneh and B. Chen. “LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 257. URL: <https://eprint.iacr.org/2024/257>.
- [7] D. Boneh and B. Chen. “LatticeFold+: Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 247. URL: <https://eprint.iacr.org/2025/247>.
- [8] D. Boneh, J. Drake, B. Fisch, and A. Gabizon. “Halo infinite: Proof-carrying data from additive polynomial commitments”. In: *Advances in Cryptology-CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part I* 41. Springer. 2021, pp. 649–680.
- [9] S. Bowe, J. Grigg, and D. Hopwood. *Recursive Proof Composition without a Trusted Setup*. Cryptology ePrint Archive, Paper 2019/1021. <https://eprint.iacr.org/2019/1021>. 2019. URL: <https://eprint.iacr.org/2019/1021>.
- [10] B. Bünz and B. Chen. “ProtoStar: Generic Efficient Accumulation/Folding for Special Sound Protocols”. In: *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4–8, 2023, Proceedings, Part IV*. Lecture Notes in Computer Science. Springer, 2023.
- [11] B. Bünz, A. Chiesa, W. Lin, P. Mishra, and N. Spooner. “Proof-Carrying Data Without Succinct Arguments”. In: *Advances in Cryptology - CRYPTO 2021*. Ed. by T. Malkin and C. Peikert. Cham: Springer International Publishing, 2021, pp. 681–710. ISBN: 978-3-030-84242-0.
- [12] B. Bünz, A. Chiesa, P. Mishra, and N. Spooner. “Recursive Proof Composition from Accumulation Schemes”. In: *Theory of Cryptography*. Ed. by R. Pass and K. Pietrzak. Cham: Springer International Publishing, 2020, pp. 1–18. ISBN: 978-3-030-64378-2.
- [13] B. Bünz, P. Mishra, W. Nguyen, and W. Wang. “Accumulation without Homomorphism”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 474. URL: <https://eprint.iacr.org/2024/474>.
- [14] B. Bünz, P. Mishra, W. Nguyen, and W. Wang. “Arc: Accumulation for Reed-Solomon Codes”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 1731. URL: <https://eprint.iacr.org/2024/1731>.
- [15] B. Bünz, A. Chiesa, G. Fenzi, and W. Wang. *Linear-Time Accumulation Schemes*. Cryptology ePrint Archive, Paper 2025/753. 2025. URL: <https://eprint.iacr.org/2025/753>.
- [16] R. Canetti, O. Goldreich, and S. Halevi. “The random oracle methodology, revisited”. In: *J. ACM* 51.4 (2004), pp. 557–594. DOI: [10.1145/1008731.1008734](https://doi.org/10.1145/1008731.1008734). URL: <https://doi.org/10.1145/1008731.1008734>.
- [17] M. Chen, A. Chiesa, T. Gur, J. O’Connor, and N. Spooner. “Proof-Carrying Data from Arithmetized Random Oracles”. In: *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual Inter-*

- national Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II*. Ed. by C. Hazay and M. Stam. Vol. 14005. Lecture Notes in Computer Science. Springer, 2023, pp. 379–404. DOI: [10.1007/978-3-031-30617-4_13](https://doi.org/10.1007/978-3-031-30617-4_13). URL: https://doi.org/10.1007/978-3-031-30617-4_13.
- [18] M. Chen, A. Chiesa, and N. Spooner. “On Succinct Non-interactive Arguments in Relativized Worlds”. In: *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II*. Ed. by O. Dunkelman and S. Dziembowski. Vol. 13276. Lecture Notes in Computer Science. Springer, 2022, pp. 336–366. DOI: [10.1007/978-3-031-07085-3_12](https://doi.org/10.1007/978-3-031-07085-3_12). URL: https://doi.org/10.1007/978-3-031-07085-3_12.
 - [19] A. Chiesa, D. Ojha, and N. Spooner. “Fractal: Post-quantum and transparent recursive proofs from holography”. In: *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I* 39. Springer, 2020, pp. 769–793.
 - [20] A. Chiesa and E. Tromer. “Proof-Carrying Data and Hearsay Arguments from Signature Cards.” In: *ICS*. Vol. 10. 2010, pp. 310–331.
 - [21] L. Devadas, R. Goyal, Y. Kalai, and V. Vaikuntanathan. “Rate-1 Non-Interactive Arguments for Batch-NP and Applications”. In: *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*. IEEE, 2022, pp. 1057–1068. DOI: [10.1109/FOCS54457.2022.00103](https://doi.org/10.1109/FOCS54457.2022.00103). URL: <https://doi.org/10.1109/FOCS54457.2022.00103>.
 - [22] L. Eagen and A. Gabizon. “ProtoGalaxy: Efficient ProtoStar-style folding of multiple instances”. In: *Cryptology ePrint Archive* (2023).
 - [23] G. Fuchsbauer, E. Kiltz, and J. Loss. “The Algebraic Group Model and its Applications”. In: *CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018*. Ed. by H. Shacham and A. Boldyreva. Vol. 10992. LNCS. Springer, 2018, pp. 33–62.
 - [24] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger. “Poseidon: A New Hash Function for Zero-Knowledge Proof Systems”. In: *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*. Ed. by M. D. Bailey and R. Greenstadt. USENIX Association, 2021, pp. 519–535. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>.
 - [25] Y. Kalai, A. Lombardi, V. Vaikuntanathan, and D. Wichs. “Boosting Batch Arguments and RAM Delegation”. In: *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*. Ed. by B. Saha and R. A. Servedio. ACM, 2023, pp. 1545–1552. DOI: [10.1145/3564246.3585200](https://doi.org/10.1145/3564246.3585200). URL: <https://doi.org/10.1145/3564246.3585200>.
 - [26] A. Kothapalli and B. Parno. “Algebraic Reductions of Knowledge”. In: *Advances in Cryptology – CRYPTO 2023*. Ed. by H. Handschuh and A. Lysyanskaya. Cham: Springer Nature Switzerland, 2023, pp. 669–701. ISBN: 978-3-031-38551-3.
 - [27] A. Kothapalli, S. Setty, and I. Tzialla. “Nova: Recursive Zero-Knowledge Arguments from Folding Schemes”. In: *Advances in Cryptology – CRYPTO 2022*. Ed. by Y. Dodis and T. Shrimpton. Cham: Springer Nature Switzerland, 2022, pp. 359–388.
 - [28] A. Kothapalli and S. T. V. Setty. “HyperNova: Recursive Arguments for Customizable Constraint Systems”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*. Ed. by L. Reyzin and D. Stebila. Vol. 14929. Lecture Notes in Computer Science. Springer, 2024, pp. 345–379. DOI: [10.1007/978-3-031-68403-6_11](https://doi.org/10.1007/978-3-031-68403-6_11). URL: https://doi.org/10.1007/978-3-031-68403-6_11.
 - [29] H. Lee and J. H. Seo. “On the Security of Nova Recursive Proof System”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 232. URL: <https://eprint.iacr.org/2024/232>.
 - [30] C. Lund, L. Fortnow, H. J. Karloff, and N. Nisan. “Algebraic Methods for Interactive Proof Systems”. In: *J. ACM* 39.4 (1992), pp. 859–868. DOI: [10.1145/146585.146605](https://doi.org/10.1145/146585.146605). URL: <https://doi.org/10.1145/146585.146605>.
 - [31] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn. “Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updatable Structured Reference Strings”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK,*

- November 11-15, 2019. Ed. by L. Cavallaro, J. Kinder, X. Wang, and J. Katz. ACM, 2019, pp. 2111–2128. DOI: [10.1145/3319535.3339817](https://doi.org/10.1145/3319535.3339817). URL: <https://doi.org/10.1145/3319535.3339817>.
- [32] D. Marin et al. *Nexus 1.0: Enabling Verifiable Computation*. 2024.
- [33] N. Mohnblatt. *Sangria: a folding scheme for PLONK*. Online: <https://geometry.xyz/notebook/sangria-a-folding-scheme-for-plonk>. 2023.
- [34] W. Nguyen, D. Boneh, and S. Setty. *Revisiting the Nova Proof System on a Cycle of Curves*. Cryptology ePrint Archive, Paper 2023/969. <https://eprint.iacr.org/2023/969>. 2023. URL: <https://eprint.iacr.org/2023/969>.
- [35] W. D. Nguyen and S. T. V. Setty. “Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 294. URL: <https://eprint.iacr.org/2025/294>.
- [36] O. Paneth and R. Pass. “Incrementally Verifiable Computation via Rate-1 Batch Arguments”. In: *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*. IEEE, 2022, pp. 1045–1056. DOI: [10.1109/FOCS54457.2022.00102](https://doi.org/10.1109/FOCS54457.2022.00102). URL: <https://doi.org/10.1109/FOCS54457.2022.00102>.
- [37] S. Setty, J. Thaler, and R. Wahby. *Customizable constraint systems for succinct arguments*. Cryptology ePrint Archive, Paper 2023/552. 2023. URL: <https://eprint.iacr.org/2023/552>.
- [38] N. P. Smart. “The Discrete Logarithm Problem on Elliptic Curves of Trace One”. In: *J. Cryptol.* 12.3 (1999), pp. 193–196. DOI: [10.1007/S001459900052](https://doi.org/10.1007/S001459900052). URL: <https://doi.org/10.1007/S001459900052>.
- [39] P. Valiant. “Incrementally verifiable computation or proofs of knowledge imply time/space efficiency”. In: *Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19-21, 2008. Proceedings 5*. Springer, 2008, pp. 1–18.
- [40] M. Zhandry. “To Label, or Not To Label (in Generic Groups)”. In: *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part III*. Ed. by Y. Dodis and T. Shrimpton. Vol. 13509. Lecture Notes in Computer Science. Springer, 2022, pp. 66–96. DOI: [10.1007/978-3-031-15982-4_3](https://doi.org/10.1007/978-3-031-15982-4_3). URL: https://doi.org/10.1007/978-3-031-15982-4_3.
- [41] T. Zheng, S. Gao, Y. Guo, and B. Xiao. “KiloNova: Non-Uniform PCD with Zero-Knowledge Property from Generic Folding Schemes”. In: *IACR Cryptol. ePrint Arch.* (2023), p. 1579. URL: <https://eprint.iacr.org/2023/1579>.

A Additional Background

A.1 Commitment Schemes

Definition 5. A commitment scheme is parametrized over a message space $\mathcal{M}$ and consists on a tuple $\Pi = (\text{KeyGen}, \text{Commit})$ of PPT algorithms such that:

- $\text{ck} \leftarrow \text{KeyGen}(n)$: On input security parameter n , KeyGen outputs a commitment key ck .
- $C \leftarrow \text{Commit}(\text{ck}, \text{msg})$: On input ck and a message $\text{msg} \in \mathcal{M}$, Commit outputs a commitment C .

and satisfies the following property:

Binding: For all PPT adversaries $\mathcal{A}$,

$$\Pr \left[\begin{array}{c} \text{Commit}(\text{ck}, \text{msg}_1) = \text{Commit}(\text{ck}, \text{msg}_2) \\ \wedge \text{msg}_1 \neq \text{msg}_2 \end{array} \middle| \begin{array}{c} \text{ck} \leftarrow \text{KeyGen}(n) \\ (\text{msg}_1, \text{msg}_2) \leftarrow \mathcal{A}(\text{pk}) \end{array} \right] = \text{negl}(n)$$

Homomorphic: A commitment scheme can additionally be homomorphic, meaning that for all $C_1 = \text{Commit}(\text{msg}_1)$, and $C_2 = \text{Commit}(\text{msg}_2)$, we have $C_1 + C_2 = \text{Commit}(\text{msg}_1 + \alpha \text{msg}_2)$.

A.2 Signature Schemes

Definition 6. A signature scheme is parametrized over a message space $\mathcal{M}$ and consists of a tuple of PPT algorithms $\text{KeyGen}, \text{Sign}, \text{Sign.Verify}$ that behave as follows:

- $(\text{sk}, \text{vk}) \leftarrow \text{KeyGen}(n)$: On input security parameter n , KeyGen outputs public parameters pp , signing key sk and verification key vk . The keys sk and vk implicitly include pp .
- $\sigma_{\text{msg}} \leftarrow \text{Sign}(\text{sk}, \text{msg})$: On input sk and a message $\text{msg} \in \mathcal{M}$, outputs a signature σ_{msg} on msg .
- $1/0 \leftarrow \text{Sign.Verify}(\text{vk}, \text{msg}, \sigma_{\text{msg}})$: Verify takes as input vk , message msg and signature σ_{msg} and produces a bit expressing acceptance (1), or rejection (0);

This scheme must satisfy correctness and unforgeability:

Correctness: For all $\text{msg} \in \mathcal{M}$:

$$\Pr \left[\text{Sign.Verify}(\text{vk}, \text{msg}, \sigma_{\text{msg}}) = 1 \mid \begin{array}{l} (\text{sk}, \text{vk}) \leftarrow \text{KeyGen}(n) \\ \sigma_{\text{msg}} \leftarrow \text{Sign}(\text{sk}, \text{msg}) \end{array} \right] = 1.$$

Unforgeability: For all PPT adversaries $\mathcal{A}$,

$$\Pr \left[(\text{msg}, \sigma_{\text{msg}}) \notin \mathcal{Q} \mid \begin{array}{l} (\text{sk}, \text{vk}) \leftarrow \text{KeyGen}(n) \\ (\text{msg}, \sigma_{\text{msg}}) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}}(\text{sk}, \text{vk}) \\ \text{Sign.Verify}(\text{vk}, \text{msg}, \sigma_{\text{msg}}) = 1 \end{array} \right] = \text{negl}(n)$$

where $\mathcal{Q}$ is a list of all the queries $\mathcal{A}$ makes to $\mathcal{O}_{\text{Sign}}$.

A.3 Algebraic Reductions of Knowledge

Definition 7. [26] An Algebraic Reduction of Knowledge from relation $\mathcal{R}_1$ to relation $\mathcal{R}_2$ is a tuple of three PPT algorithms $(\mathcal{G}, \mathcal{P}, \text{Reduce})$ with the following interface:

- $\text{pp} \leftarrow \mathcal{G}(n)$: On input the security parameter n , outputs public parameters pp .
- $(\phi_2, \text{w}_2) \leftarrow \mathcal{P}(\text{pp}, \phi_1, \text{w}_1)$: On input pp and $(\phi_1, \text{w}_1) \in \mathcal{R}_1$, outputs a pair $(\phi_2, \text{w}_2) \in \mathcal{R}_2$.
- $\phi_2 \leftarrow \mathcal{V}(\text{pp}, \phi_1)$: Take as input pp and ϕ_1 and outputs a new instance ϕ_2 .

A Reduction of Knowledge $(\mathcal{G}, \mathcal{P}, \text{Reduce})$ satisfies the following properties:

- **Completeness:** For any PPT adversary $\mathcal{A}$:

$$\Pr \left[\begin{array}{l} \phi'_2 \leftarrow \langle \mathcal{P}, \mathcal{V} \rangle(\text{pp}, \phi_1) \\ \wedge \phi_2 = \phi'_2 \\ \wedge (\phi_2, \text{w}_2) \in \mathcal{R}_2 \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(n) \\ (\phi_1, \text{w}_1) \leftarrow \mathcal{A}(\text{pp}) \\ (\phi_2, \text{w}_2) \leftarrow \langle \mathcal{P}, \mathcal{V} \rangle(\text{pp}, \phi_1, \text{w}_1) \end{array} \right] = 1.$$

- **Knowledge Soundness:** For any PPT adversary $\mathcal{A}$ and prover $\mathcal{P}^*$, there exists an expected polynomial time extractor $\mathcal{E}$ such that

$$\Pr \left[(\phi_1, \text{w}_1) \in \mathcal{R}_1 \mid \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(n) \\ (\phi_1, \text{st}) \leftarrow \mathcal{A}(\text{pp}) \\ \text{w}_1 \leftarrow \mathcal{E}(\text{pp}, \phi_1, \text{st}) \end{array} \right] \\ \approx \Pr \left[(\phi_2, \text{w}_2) \in \mathcal{R}_2 \mid \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(n) \\ (\phi_1, \text{st}) \leftarrow \mathcal{A}(\text{pp}) \\ (\phi_2, \text{w}_2) \leftarrow \langle \mathcal{P}^*, \mathcal{V} \rangle(\text{pp}, \phi_1, \text{st}) \end{array} \right]$$

- **Public Reducibility:** There exists a deterministic polynomial time function Φ such that for any PPT adversary $\mathcal{A}$ and expected polynomial time P^* , given $\text{pp} \leftarrow \mathcal{G}(n)$, $(\phi_1, \text{st}) \leftarrow \mathcal{A}(\text{pp})$, and $(\phi_2, \text{w}_2) \leftarrow \langle \mathcal{P}, \mathcal{V} \rangle(\text{pp}, \phi_1, \text{st})$ with interaction transcript tr , we have that $\Phi(\text{pp}, \phi_1, \text{tr}) = \phi_2$.

B The Sumcheck Protocol

In this section, we present the sumcheck protocol as a NIROK and prove its security in the osRO model.

The sumcheck protocol [30] is an interactive protocol between an untrusted prover and a verifier, where the former wants to convince the latter that the evaluations of a polynomial $G(\vec{X}) \in \mathbb{F}[X_1, \dots, X_m]$ over a set S^m sum to a specified value sum . Formally, the sumcheck relation $\mathcal{R}_{\text{sc}}$ is defined as:

$$\mathcal{R}_{\text{sc}} = \left\{ \phi = (m, G(\vec{X}) \in \mathbb{F}[X_1, \dots, X_m], S \subset \mathbb{F}, \text{sum}), \mid \begin{array}{l} \text{w} = \perp \\ \text{sum} = \sum_{\vec{x} \in S^m} G(\vec{x}) \end{array} \right\}$$

We identify two protocols that reduce $\mathcal{R}_{\text{sc}}$ to simpler relations: The sumcheck *reduction*, reduces $\mathcal{R}_{\text{sc}}$ to checking one polynomial evaluation, whereas the sumcheck *protocol* reduces $\mathcal{R}_{\text{sc}}$ to $\mathcal{R}_{\text{true}}$. Indeed, the sumcheck protocol invokes an additional `ReduceInstance` check, distinguishing it from the sumcheck reduction, which omits this step. Both the sumcheck protocol and reduction can be described as NIROKs, by inductively applying simpler non-interactive reductions of knowledge.

Specifically, for $\mathcal{R}_{\text{round}}(r)$ as defined below and a polynomial defined over m variables, we have that $\mathcal{R}_{\text{round}}(0) = \mathcal{R}_{\text{sc}}$ and $\mathcal{R}_{\text{round}}(m)$ can be trivially checked by the verifier. Verifying $\mathcal{R}_{\text{round}}(m)$ is computationally more efficient than verifying $\mathcal{R}_{\text{round}}(0)$.

$$\mathcal{R}_{\text{round}}(r) = \left\{ \phi = (m, r, G(\vec{X}), \vec{\alpha} \in \mathbb{F}^r, S \subset \mathbb{F}, \text{sum}), \left| \begin{array}{l} G(\vec{X}) \in \mathbb{F}[X_1, \dots, X_m] \\ \text{sum} = \sum_{\vec{x} \in S^{m-r}} G(\vec{\alpha} || \vec{x}) \end{array} \right. \right\} \quad (4)$$

In Figure 8, we present the reduction Π_{round} from $\mathcal{R}_{\text{round}}(r)$ to $\mathcal{R}_{\text{round}}(r+1)$ and prove its security below. The reduction assumes the presence of an open-and-sign oracle $\mathcal{O}_{\text{osRO}}$ with verification key vk . It consists of iteratively running $\Pi_{\text{round}}.\text{Reduce}$ and $\Pi_{\text{round}}.\text{ReduceInstance}$ to go from $\mathcal{R}_{r(0)} = \mathcal{R}_{\text{sc}}$ to $\mathcal{R}_r(m)$.

In Fig. 9, we describe the sumcheck reduction $\Pi_{\text{red-sc}}$, which iteratively runs Π_{round} to go from ϕ_0 to ϕ_m , and protocol Π_{sc} , which reduces $\mathcal{R}_{\text{sc}} = \mathcal{R}_{r(0)}$ to $\mathcal{R}_{\text{true}}$ by running Π_{round} and asking the verifier to check $\phi_m \in \mathcal{R}_{\text{round}}(m)$.

Theorem 5. *The protocol in Figure 8 is a reduction of knowledge from $\mathcal{R}_{\text{round}}(r)$ to $\mathcal{R}_{\text{round}}(r+1)$.*

Proof. Completeness follows by inspection.

Since no witness is involved in the relations, following Game 2, we need to prove that for any $\mathcal{A}$ that outputs $(\phi_r, \perp, \pi)$, if $\phi_{r+1} \leftarrow \text{ReduceInstance}(\text{pp}, \text{vk}, \phi_r, \pi)$, then $(\phi_r, \perp) \in \mathcal{R}_{\text{round}}(r)$ with overwhelming probability.

```

 $\Pi_{\text{round}}.\text{KeyGen}(n, s) \mapsto \text{pp} :$ 
return  $\text{pp} = (\mathbb{G}, p, \vec{g})$ 

 $\Pi_{\text{round}}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, \phi_r, \perp) \mapsto (\phi_{r+1}, \perp, \pi)$ 
parse  $(m, r, G(\vec{X}), \vec{\alpha}_r, S, \text{sum}_r) \leftarrow \phi_r$ 
 $g(X) \leftarrow \sum_{\vec{y} \in S^{m-r-1}} G(\vec{\alpha}_r, X, \vec{y})$ 
 $(\alpha_{r+1}, \sigma_{r+1}) \leftarrow \mathcal{O}_{\text{osRO}}(\text{aux} = (\phi_r, g))$            # Oracle query
 $\text{sum}_{r+1} \leftarrow g(\alpha_{r+1})$ 
 $\vec{\alpha}_{r+1} = \vec{\alpha}_r || \alpha_{r+1}$ 
 $\pi \leftarrow (g, \alpha_{r+1}, \sigma_{r+1})$ 
 $\phi_{r+1} \leftarrow (m, r+1, G(\vec{X}), \vec{\alpha}_{r+1}, S, \text{sum}_{r+1})$ 
return  $(\phi_{r+1}, \perp, \pi)$ 

 $\Pi_{\text{round}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_r, \pi) \mapsto \phi_{r+1}$ 
parse  $(m, r, G(\vec{X}), \vec{\alpha}_r, S, \text{sum}_r) \leftarrow \phi_r$ 
parse  $\pi \leftarrow (g, \alpha_{r+1}, \sigma_{r+1})$ 
 $\text{sum}_{r+1} \leftarrow g(\alpha_{r+1})$ 
assert  $\text{sum}_{r+1} = \sum_{x \in S} g(x)$ 
assert  $\text{Sign.Verify}(\text{vk}, (\text{aux} = (\phi_r, g), \alpha_{r+1}), \sigma_{r+1}) = 1$ 
 $\vec{\alpha}_{r+1} = \vec{\alpha}_r || \alpha_{r+1}$ 
return  $\phi_{r+1} \leftarrow (m, r+1, G(\vec{X}), \vec{\alpha}_{r+1}, S, \text{sum}_{r+1})$ 

```

Fig. 8. Reduction $\Pi_{\text{round}} : \mathcal{R}_{\text{round}}(r) \rightarrow \mathcal{R}_{\text{round}}(r+1)$.

Consider $(\phi_r, \pi) \leftarrow \mathcal{A}(\text{pp})$ and $\phi_{r+1} \leftarrow \text{ReduceInstance}(\text{pp}, \text{vk}, \phi_r, \pi)$.

We assume $(\phi_{r+1}, \perp) \in \mathcal{R}_{\text{round}}(r+1)$ and thus, for $(\alpha_{r+1}, \sigma_{r+1}, g(X)) \leftarrow \pi$ and $(m, r+1, G(\vec{X}), \vec{\alpha}_{r+1}, S, \text{sum}_{r+1}) \leftarrow \phi_{r+1}$, $\sum_{\vec{x} \in S^{m-(r+1)}} G(\vec{\alpha}_{r+1} || \vec{x}) = \text{sum}_{r+1}$, where $\text{sum}_{r+1} = g_1(\alpha_{r+1})$, $\vec{\alpha}_{r+1} = \vec{\alpha}_r || \alpha_{r+1}$.

Replacing, we have $\sum_{\vec{x} \in S^{m-r_2}} G(\vec{\alpha}_1 || \alpha_2 || \vec{x}) = g_1(\alpha_2)$. By Schwartz-Zippel, if that equation holds for a random α_{r+1} , except with probability $\deg_{r+1}(G)/|\mathbb{F}|$, it holds that $\sum_{\vec{x} \in S^{m-r_2}} G(\vec{\alpha}_1 || X || \vec{x}) = g_1(X)$.

If $\mathcal{A}$ can make q_H queries to the oracle, we know the equation hold with probability at least $(q_H \deg_{r+1}(G))/|\mathbb{F}|$. That is unless $\mathcal{A}$ forges a signature that happens with maximum probability $\text{Adv}_{\mathcal{B}}^{\text{unf}}$.

Finally, since $\text{sum}_r = \sum_{x \in S} g(X)$, we have $\sum_{\vec{x} \in S^{m-r}} G(\vec{\alpha}_1 || \vec{x}) = \text{sum}_r$. We conclude $(\phi_r, \perp) \in \mathcal{R}_{\text{round}}(r)$ except with probability bounded by $\text{Adv}_{\mathcal{B}}^{\text{unf}} + \frac{q_H \deg_{r+1}(G)}{|\mathbb{F}|}$. $\square$

```

 $\Pi_{\text{red-sc}}.\text{KeyGen}(n, s) \mapsto \text{pp}:$ 
return  $\text{pp} \leftarrow \Pi_{\text{round}}.\text{KeyGen}(n, s)$ 
 $\Pi_{\text{red-sc}}.\text{Reduce}(\text{pp}, \phi_{\text{sc}}, \perp) \mapsto (\phi_m, \perp, \pi):$ 
parse  $(m, r = 0, G, \vec{\alpha} = \emptyset, S, \text{sum}) \leftarrow \phi_{\text{sc}}$ 
 $\phi_0 \leftarrow \phi_{\text{sc}}$ 
 $\pi \leftarrow []$ 
for  $r = 0$  to  $m - 1$ :
     $(\phi_{r+1}, \perp, \pi_{r+1}) \leftarrow \Pi_{\text{round}}.\text{Reduce}(\text{pp}, \phi_r, \perp)$ 
     $\pi.\text{append}(\pi_{r+1})$ 
return  $(\phi_m, \perp, \pi)$ 
 $\Pi_{\text{red-sc}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_{\text{sc}}, \pi) \mapsto \phi_m:$ 
parse  $(m, r = 0, G, \vec{\alpha} = \emptyset, S, \text{sum}) \leftarrow \phi_{\text{sc}}$ 
parse  $(\pi_1, \dots, \pi_m) \leftarrow \pi$ 
 $\phi_0 \leftarrow \phi_{\text{sc}}$ 
for  $r = 0$  to  $m - 1$ :
     $\phi_{r+1} \leftarrow \Pi_{\text{round}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_r, \pi_r)$ 
    assert  $\phi_{r+1} \neq \perp$ 
parse  $(m, r = m, G, \vec{\alpha}_m \in \mathbb{F}^m, S, \text{sum}_m) \leftarrow \phi_m$ 
return  $\phi_m$ 

```

Fig. 9. The Sumcheck Reduction

Finally, we present the reduction Π_{sc} from $\mathcal{R}_{\text{sc}}$ to $\mathcal{R}_{\text{true}}$, that is, the sumcheck protocol. The reduction Π_{sc} uses Π_{round} from Section 4 iteratively as a backend. We prove security with respect to Definition 1 in Theorem 6.

Theorem 6. *The protocol in Figure 10 is a reduction of knowledge from relation $\mathcal{R}_{\text{sc}}$ to $\mathcal{R}_{\text{true}}$.*

Proof. We need to prove that for every adversary $\mathcal{A}(\text{pp})$'s output $(\phi, \pi, \perp)$, whenever $\text{true} \leftarrow \Pi_{\text{sc}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_{\text{sc}}, \pi)$, we have that $(\phi_{\text{sc}}, \perp) \in \mathcal{R}_{\text{sc}}$ except with negligible probability.

Since ReduceInstance outputs true, we have that for $(m, m, G, \vec{\alpha}, S, \text{sum}_m) \leftarrow \phi_m$, it holds that $G(\vec{\alpha}_m) = \text{sum}_m$, which implies $\phi_m \in \mathcal{R}_{\text{round}}(m)$.

Because ϕ_m is the output of $\Pi.\text{ReduceInstance}(\text{pp}, \phi_r, \pi_r)$ and Π_{round} is a reduction of knowledge, we have that $(\phi_{m-1}, \perp) \in \mathcal{R}_{\text{round}}(m-1)$ with probability $\text{Adv}_{\mathcal{A}}^{\text{unf}} + \frac{q_H \deg_{m-1}(G)}{|\mathbb{F}|}$. Inductively applying the definition of RoK to Π_{round} , we get that $(\phi_0, \perp) \in \mathcal{R}_{\text{round}}(0)$, which by definition, is the same as saying $(\phi_{\text{sc}}, \perp) \in \mathcal{R}_{\text{sc}}$, except with probability $\text{Adv}_{\mathcal{A}}^{\text{unf}} + \sum_{r=0}^{m-1} \frac{q_H \deg_{r+1}(G)}{|\mathbb{F}|}$. $\square$

```

 $\Pi_{sc}.\text{KeyGen}(n, s) \mapsto \text{pp}:$ 
return  $\text{pp} \leftarrow \Pi_{\text{round}}.\text{KeyGen}(n, s)$ 
 $\Pi_{sc}.\text{Reduce}(\text{pp}, \phi_{sc}, \perp) \mapsto (\text{true}, \pi, \perp):$ 
parse  $(m, r = 0, G, \vec{\alpha} = \emptyset, S, \text{sum}) \leftarrow \phi_{sc}$ 
 $\phi_0 \leftarrow \phi_{sc}$ 
 $\pi \leftarrow []$ 
for  $r = 0$  to  $m - 1$ :
     $(\phi_{r+1}, \perp, \pi_{r+1}) \leftarrow \Pi_{\text{round}}.\text{Reduce}(\text{pp}, \phi_r, \perp)$ 
     $\pi.\text{append}(\pi_{r+1})$ 
return  $(\text{true}, \pi, \perp)$ 
 $\Pi_{sc}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_{sc}, \pi) \mapsto \{\text{true}, \perp\}:$ 
parse  $(m, r = 0, G, \vec{\alpha} = \emptyset, S, \text{sum}) \leftarrow \phi_{sc}$ 
parse  $(\pi_1, \dots, \pi_m) \leftarrow \pi$ 
 $\phi_0 \leftarrow \phi_{sc}$ 
for  $r = 0$  to  $m - 1$ :
     $\phi_{r+1} \leftarrow \Pi_{\text{round}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_r, \pi_r)$ 
    assert  $\phi_{r+1} \neq \perp$ 
parse  $(m, r = m, G, \vec{\alpha}_m \in \mathbb{F}^m, S, \text{sum}_m) \leftarrow \phi_m$ 
assert  $G(\vec{\alpha}_m) = \text{sum}_m$ 
return true.

```

Fig. 10. The Sumcheck Protocol as a NIroK

C Deferred Protocols and Proofs from Section 4

Notation: For matrices $\mathbf{M} \in \mathbb{F}^{m \times m'}$, $s = \log(m)$ and $s' = \log(m')$ we denote as $M(\vec{X}, \vec{Y})$ the unique multilinear extension of function $M : \{0, 1\}^s \times \{0, 1\}^{s'} \rightarrow \mathbb{F}$ where $M(\vec{x}, \vec{y}) = M_{x,y}$ for x, y such that $\vec{x}, \vec{y}$ are its binary representations. For vectors $\vec{z} \in \mathbb{F}^{m'}$, $Z(\vec{X})$ is the corresponding multilinear representation. Finally, we set $eq(\vec{X}, \vec{x}) = \prod_{i=1}^s (x_i \times X_i + (1 - x_i)(1 - X_i))$ and recall that for a multilinear polynomial $P(\vec{X})$, $P(\vec{X}) = \sum_{\vec{x} \in \{0,1\}^s} eq(\vec{X}, \vec{x}) P(\vec{x})$. As we have been doing throughout this paper, we adopt homomorphic Pedersen-like commitment schemes (KeyGen, Commit), as in Definition 5.

C.1 Reduction $\Pi_{lc} : \mathcal{R}_{cccs} \rightarrow \mathcal{R}_{lcscs}$

In Figure 11 we present Π_{lc} 's reduction from $\mathcal{R}_{cccs}$ to $\mathcal{R}_{lcscs}$. The Reduce algorithm runs the sumcheck reduction of Figure 9 to obtain, from the multilinear expansions of $\mathbf{M}$ and $\vec{t}$, values $v_i = \vec{r}^\top \mathbf{M}_j \vec{z}$ for randomness $\vec{r}$ defined during the sumcheck. It then provides a proof that, if the v_i 's are correctly formed, then ϕ_1 is an instance of $\mathcal{R}_1 = \mathcal{R}_{cccs}$. The well-formedness of the v_i 's can be expressed as an instance of $\mathcal{R}_{lcscs}$.

Theorem 7. Π_{lc} in Fig. 11 is a reduction of knowledge.

Proof. Completeness follows by inspection.

For knowledge soundness, let $\mathcal{A}$ be an adversary against the security game of Fig 2. On input pp , $\mathcal{A}$ outputs $(\phi_1, \pi, \mathbf{w}_2)$. We will construct an extractor Ext_1 that on input $(\text{pp}, \phi_1, \phi_2, \pi, \mathbf{w}_2)$ where $\phi_2 \leftarrow \text{ReduceInstance}(\text{pp}, \phi_1, \pi)$, outputs a witness $\mathbf{w}_2$. Adversary wins if (a) $(\phi_1, \mathbf{w}_1) \notin \mathcal{R}_{cccs}$, (b) $\phi_2 \neq \perp$, and (c) $(\phi_2, \mathbf{w}_2) \in \mathcal{R}_{lcscs}$. We assume (b) and (c) hold and prove that for $\mathbf{w}_2$ outputted by Ext_1 , $(\phi_2, \mathbf{w}_2) \in \mathcal{R}_{lcscs}$ except with negligible probability.

Extractor description: $\text{Ext}_1(\text{pp}, \phi_1, \phi_2, \pi, \mathbf{w}_2):$

- Parse $\phi_1 = (((\mathbf{M}_j)_{j=1}^t, (S_i, c_i)_{i=1}^q, \phi_t, T)), \pi = (\beta, \sigma_\beta, Q(\vec{X}), \pi_{sc}, \{v_j\}_{j=1}^t)$

```

 $\Pi_{lc}.\text{Reduce}(\text{pp}, \phi_1, \mathbf{w}_1) \mapsto (\phi_2, \mathbf{w}_2, \pi)$ 
parse  $((\mathbf{M}_j)_{j=1}^t, (S_i, c_i)_{i=1}^q, \phi_t, T), \vec{t} \leftarrow (\phi_1, \mathbf{w}_1)$ 
 $(\beta, \sigma_\beta) \leftarrow \text{osRO}((T, \vec{t}), \text{aux} = \phi_1)$  # osRO query
 $\vec{\beta} \leftarrow (\beta, \beta^2, \dots, \beta^s)$ 
 $\vec{z} \leftarrow (\phi_t \parallel \vec{t})$ 
 $Q(\vec{X}) = eq(\vec{\beta}, \vec{X}) \sum_{i=1}^q c_i \prod_{j \in S_i} \left( \sum_{y \in \{0,1\}^{s'}} M_j(\vec{X}, y) z_2(y) \right)$ 
 $\phi_0 \leftarrow (s, Q(\vec{X}), \{0, 1\}, 0)$ 
 $(\phi_m, \perp, \pi_{\text{red-sc}}) \leftarrow \Pi_{\text{red-sc}}.\text{Reduce}(\text{pp}, \phi_0, \perp)$  # Sumcheck reducer
parse  $(m, m, G, \vec{r}, S, \text{sum}_m) \leftarrow \phi_m$ 
for  $j \in [t]$ 
   $v_j \leftarrow \vec{r}^\top \mathbf{M}_j(\phi_t \parallel \vec{t})$ 
 $(\phi_2, \mathbf{w}_2) \leftarrow ((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}, \phi_t, T), \vec{t}$ 
 $\pi \leftarrow (\beta, \sigma_\beta, Q(\vec{X}), \pi_{\text{sc}}, \{v_j\}_{j=1}^t)$ 
return  $(\phi_2, \mathbf{w}_2, \pi)$ 
 $\Pi_{lc}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_1, \pi) \mapsto \phi_2$ 
parse  $((\mathbf{M}_j)_{j=1}^t, (S_i, c_i)_{i=1}^q, \phi_t, T) \leftarrow \phi_1$ 
parse  $(\beta, \sigma_\beta, Q(\vec{X}), \pi_{\text{sc}}, \{v_j\}_{j=1}^t) \leftarrow \pi$ 
assert  $\text{Sign.Verify}(\text{vk}, (T, \phi_1, \beta), \sigma_\beta)$  # osRO check
 $\vec{\beta} \leftarrow (\beta, \beta^2, \dots, \beta^s)$ 
 $\phi_0 \leftarrow (s, Q(\vec{X}), \{0, 1\}, 0)$ 
 $\phi_m \leftarrow \Pi_{\text{red-sc}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_0, \pi_{\text{red-sc}})$  # Sumcheck inst reducer
parse  $(m, m, G, \vec{r}, S, \text{sum}_m) \leftarrow \phi_m$ 
assert  $G(\vec{r}) = \text{sum}_m$ 
Compute  $e_2 \leftarrow eq(\vec{\beta}, \vec{r})$  and assert  $e_2 \sum_{i=1}^q c_i \prod_{j \in S_i} v_j = \text{sum}_m$ 
 $\phi_2 \leftarrow ((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}, \phi_t, T)$ 
return  $\phi_2$ 

```

Fig. 11. Reduction Π_{lc} from $\mathcal{R}_{\text{cCCS}} \rightarrow \mathcal{R}_{lc\text{CCS}}$

- Search for query

$$((T, \vec{t}), \text{aux} = \phi_t, \beta) \in Q$$

such that $T = \vec{t} \times \vec{g}$, aborts if it does not exist.

- Output $\vec{t}$.

Extractor does not abort: We claim that if Ext_1 aborts, then there exist an adversary $\mathcal{B}_1$ against unforgeability of the signature scheme with a non-negligible advantage.

Indeed, $\mathcal{B}_1^{\text{O}_{\text{Sign}}}(\text{vk})$ chooses pp uniformly at random and runs $\mathcal{A}^{\text{O}_{\text{osRO}}}(\text{pp})$. When $\mathcal{A}$ queries $\text{O}_{\text{osRO}}((A_i, \vec{a}_i)_{i=1}^\ell, \text{aux})$, $\mathcal{B}_1$ verifies that $A_i = \vec{a}_i \times \vec{g}$. If yes, $\mathcal{B}_1$ samples $\beta \leftarrow \mathbb{F}$, queries the signing oracle on input $((A_i, \vec{a}_i)_{i=1}^\ell, \text{aux}), \beta$ to obtain σ_β and outputs (β, σ_β) as an answer to $\mathcal{A}$'s query.

When $\mathcal{A}$ queries $(\phi_1, \pi, \mathbf{w}_2)$, $\mathcal{B}_1$ parses $((\mathbf{M}_j)_{j=1}^t, (S_i, c_i)_{i=1}^q, \phi_t, T) \leftarrow \phi_1$ and returns $((T, \phi_t, \beta), \sigma_\beta)$. Because $1 = \text{Sign.Verify}((T, \phi_1, \beta), \sigma_\beta)$ but Ext_1 aborts, $\mathcal{B}_1$ never queried the signing oracle on (T, ϕ_1, β) , $\mathcal{B}_1$ wins. Thus, we have that the probability that Ext_1 aborts is less or equal than the advantage of $\mathcal{B}_1$ against unforgeability (Definition 6).

Outputs a valid witness:

- Parse $\vec{t} \leftarrow \mathbf{w}_2$. Recall $T = \vec{t} \times \vec{g}$ and, because $\vec{t}$ is a valid witness for $\mathcal{R}_2$, $T = \vec{t} \times \vec{g}$. By definition of the binding property of the Pedersen commitment scheme, the probability $\Pr[\vec{t}' \neq \vec{t} \mid T = \vec{t} \times \vec{g} = \vec{t}' \times \vec{g}]$ is negligible in n . Indeed, consider an adversary $\mathcal{B}_2$ that runs $\mathcal{A}$ as a subroutine. $\mathcal{B}_2$ acts as the challenge of the NIRoK game (see Figure 2) and runs the extractor Ext_1 defined above. Finally, $\mathcal{B}_2$ outputs the tuple $(T, \vec{t}, \vec{t}')$. It holds by definition of the collision-finding game that $\Pr[\vec{t}' \neq \vec{t} \mid T = \vec{t} \times \vec{g} = \vec{t}' \times \vec{g}] \leq \text{Adv}_{\mathcal{B}_2}^{\text{bind}}(n)$.
- Because $\vec{t}$ is such that $(\phi_2, \vec{t}) \in \mathcal{R}_2$, we have that for all $j \in [t]$, $v_j = \vec{r}^\top \mathbf{M}_j \vec{t} = \sum_{y \in \{0,1\}^{s'}} M_j(\vec{r}, \vec{y}) z(\vec{y})$, for $M_j(\vec{X}, \vec{Y})$, $z(\vec{Y})$, the multilinear extensions of $\mathbf{M}_j$ and $\vec{z} = \phi_t || \vec{t}$, respectively.
- Because ReduceInstance_1 does not abort, we have that $Q(\vec{r}) = \text{sum}_m$ and $eq(\vec{\beta}, \vec{r}) \sum_{i=1}^q c_i \bigcirc_{j \in S_i} v_j = \text{sum}_m$.
- Since $\vec{r}$ is sampled after $\mathcal{A}$ includes $Q(\vec{X})$ in a query to O_{osRO} (as part of $\Pi_{\text{red-sc}}$), by the Schwartz-Zippel lemma, we have that except with probability $q_H * s / |\mathbb{F}|$,

$$Q(\vec{X}) = eq(\vec{\beta}, \vec{X}) \sum_{i=1}^q c_i \bigcirc_{j \in S_i} \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{X}, \vec{y}) t(\vec{y})$$

- Since ReduceInstance checks $G(\vec{r}) = \text{sum}_m$, from soundness of the sumcheck protocol we have that $\sum_{\vec{x} \in \{0,1\}^s} Q(\vec{x}) = 0$ except with probability $\epsilon_{\text{sc}}(n)$, the soundness error of the sumcheck. Then,

$$\begin{aligned} 0 &= \sum_{\vec{x} \in \{0,1\}^s} eq(\vec{\beta}, \vec{X}) \sum_{i=1}^q c_i \bigcirc_{j \in S_i} \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{X}, \vec{y}) t(\vec{y}) \\ &= \sum_{i=1}^q c_i \bigcirc_{j \in S_i} \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{\beta}, \vec{y}) t(\vec{y}) \end{aligned}$$

- The equation above implies that $\sum_{i=1}^q c_i \bigcirc_{j \in S_i} \vec{\beta}^\top \mathbf{M}_j \vec{t} = 0$. Suppose by contradiction that $\sum_{i=1}^q c_i \bigcirc_{j \in S_i} \mathbf{M}_j \vec{t} \neq 0$. The probability that, for a random β' , $\sum_{i=1}^q c_i \bigcirc_{j \in S_i} \beta'^\top \mathbf{M}_j \vec{t} = 0$ is $s / |\mathbb{F}|$. Let q_H be the number of queries $\mathcal{A}$ can make to $\mathcal{O}$, the probability it finds a β satisfying the equation above for a non valid witness $\vec{t}$ is $q_H * s / |\mathbb{F}|$.

We conclude

$$\text{Adv}_{\text{Ext}_1, \mathcal{A}}^{\text{rok}}(n) \leq \text{Adv}_{\mathcal{B}_1}^{\text{unf}}(n) + \text{Adv}_{\mathcal{B}_2}^{\text{bind}}(n) + q_H \frac{s}{|\mathbb{F}|} + \epsilon_{\text{sc}}(n) + q_H \frac{s}{|\mathbb{F}|}. \quad \square$$

C.2 Folding scheme for $\mathcal{R}_{\text{lcCCS}}$

In Fig. 12 we introduce a reduction from $\mathcal{R}_{\text{lcCCS}} \times \mathcal{R}_{\text{lcCCS}}$ to $\mathcal{R}_{\text{lcCCS}}$. For simplicity, we present the folding scheme for two arbitrary ϕ_{lcCCS} , that may not share randomness, but we express this condition as a requirement for the protocol to complete.

Theorem 8. *The protocol in Fig. 12 is a non-interactive reduction of knowledge from $\mathcal{R}_{\text{lcCCS}} \times \mathcal{R}_{\text{lcCCS}}$ to $\mathcal{R}_{\text{lcCCS}}$.*

```


$$\frac{\Pi_{\text{fold}}.\text{Reduce}^{\mathcal{O}_{\text{osRO}}}(\text{pp}, (\phi_{1,a}, \phi_{1,b}), (\mathbf{w}_{1,a}, \mathbf{w}_{1,b})) \mapsto (\phi_2, \mathbf{w}_2, \pi)}{\text{parse } ((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}_1, \phi_t, T), \vec{t} \leftarrow (\phi_{1,a}, \mathbf{w}_{1,a})}$$


$$\text{parse } ((\mathbf{M}_j, u_j)_{j=1}^t, \vec{r}_2, \phi_w, W), \vec{w} \leftarrow (\phi_{1,b}, \mathbf{w}_{1,b})$$


$$(\phi_{\text{rerand}}, \mathbf{w}_{\text{rerand}}, \pi_{\text{rerand}}) \leftarrow \Pi_{\text{rerand}}.\text{Reduce}(\phi_{1,a}, \mathbf{w}_{1,a})$$


$$\text{parse } ((\mathbf{M}_j, \rho_j)_{j=1}^t, \vec{r}, \phi_t, T) \leftarrow \phi_{\text{rerand}}$$


$$\text{assert } \vec{r} = \vec{r}_2$$


$$(\alpha, \sigma_\alpha) \leftarrow \mathcal{O}_{\text{osRO}}((T, \vec{t}), (W, \vec{w}), \text{aux} = (\phi_{1,a}, \phi_{1,b}))$$


$$(\phi_2, \mathbf{w}_2) \leftarrow ((\mathbf{M}_j, \rho_j + \alpha u_j)_{j=1}^t, \vec{r}, \phi_t + \alpha \phi_w, T + \alpha W), \vec{t} + \alpha \vec{w}$$


$$\pi \leftarrow (\alpha, \sigma_\alpha, \pi_{\text{rerand}})$$


$$\text{return } (\phi_2, \mathbf{w}_2, \pi)$$



$$\frac{\Pi_{\text{fold}}.\text{ReduceInstance}(\text{pp}, \text{vk}, (\phi_{1,a}, \phi_{1,b}), \pi) \mapsto \phi_2}{\text{parse } ((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}_1, \phi_t, T) \leftarrow \phi_{1,a}}$$


$$\text{parse } ((\mathbf{M}_j, u_j)_{j=1}^t, \vec{r}_2, \phi_w, W), \vec{w} \leftarrow (\phi_{1,b}, \mathbf{w}_{1,b})$$


$$\phi_{\text{rerand}} \leftarrow \Pi_{\text{rerand}}.\text{ReduceInstance}(\phi_{1,a}, \pi_{\text{rerand}})$$


$$\text{parse } ((\mathbf{M}_j, \rho_j)_{j=1}^t, \vec{r}, \phi_t, T) \leftarrow \phi_{\text{rerand}}$$


$$\text{assert } \vec{r} = \vec{r}_2$$


$$\text{parse } (\alpha, \sigma_\alpha, \pi_{\text{rerand}}) \leftarrow \pi$$


$$\text{assert } \text{Sign.Verify}(\text{vk}, (T, \vec{t}), (W, \vec{w}), \text{aux} = (\phi_{1,a}, \phi_{1,b}, \alpha), \sigma_\alpha)$$


$$\text{return } \phi_2 \leftarrow ((\mathbf{M}_j, \rho_j + \alpha u_j)_{j=1}^t, \vec{r}, \phi_t + \alpha \phi_w, T + \alpha W).$$


```

Fig. 12. Reduction of knowledge Π_{fold} from $\mathcal{R}_{\text{lcCCS}} \times \mathcal{R}_{\text{lcCCS}} \rightarrow \mathcal{R}_{\text{lcCCS}}$. This reduction is a folding scheme.

Proof. Completeness follows by inspection.

For knowledge soundness, let $\mathcal{A}$ be an adversary against the security game of Fig 2. On input pp , $\mathcal{A}$ outputs $((\phi_{1,a}, \phi_{1,b}), \pi, \mathbf{w}_2)$. We will construct an extractor Ext_3 that on input $(\text{pp}, (\phi_{1,a}, \phi_{1,b}), \phi_2, \pi, \mathbf{w}_2)$ where $\phi_2 \leftarrow \text{ReduceInstance}(\text{pp}, (\phi_{1,a}, \phi_{1,b}), \pi)$, outputs a witness $(\mathbf{w}_{1,a}, \mathbf{w}_{1,b})$. Adversary wins if (a) $((\phi_{1,a}, \phi_{1,b}), (\mathbf{w}_{1,a}, \mathbf{w}_{1,b})) \notin \mathcal{R}_{\text{lcCCS}} \times \mathcal{R}_{\text{lcCCS}}$, (b) $\phi_2 \neq \perp$, and (c) $(\phi_2, \mathbf{w}_2) \in \mathcal{R}_{\text{lcCCS}}$. We assume (b) and (c) hold and prove that for $(\mathbf{w}_{1,a}, \mathbf{w}_{1,b})$ outputted by Ext_3 , $((\phi_{1,a}, \phi_{1,b}), (\mathbf{w}_{1,a}, \mathbf{w}_{1,b})) \in \mathcal{R}_{\text{lcCCS}} \times \mathcal{R}_{\text{lcCCS}}$ except with negligible probability.

$\text{Ext}(\text{pp}, (\phi_{1,a}, \phi_{1,b}), \phi_2, \pi, \mathbf{w}_2)$:

- parses $\phi_{1,a} = ((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}_1, \phi_t, T)$
- parses $\phi_{1,b} = ((\mathbf{M}_j, u_j)_{j=1}^t, \vec{r}_2, \phi_w, W)$
- Search for queries $((T, \vec{t}), (W, \vec{w}), \text{aux} = (\phi_{1,a}, \phi_{1,b}), \alpha) \in Q$ such that $(T, W) = \vec{t} \times \vec{g}, \vec{w} \times \vec{g}$, and aborts if no such query exists.
- Output $(\vec{t}, \vec{w})$

Extractor does not abort: Similar to the proof of Theorem 7, if Ext_3 aborts we can build an adversary $\mathcal{B}_1$ against unforgeability of the signature scheme with non-negligible advantage.

$\mathcal{B}_1^{\mathcal{O}_{\text{Sign}}}(\text{vk})$ chooses pp uniformly at random and runs $\mathcal{A}^{\mathcal{O}_{\text{osRO}}}(\text{pp})$. When $\mathcal{A}$ queries $\mathcal{O}_{\text{osRO}}((A_i, \vec{a}_i)_{i=1}^\ell, \text{aux})$, $\mathcal{B}_1$ verifies that $A_i = \vec{a}_i \times \vec{g}$. If yes, $\mathcal{B}_1$ samples $\alpha \leftarrow \mathbb{F}$, queries the signing oracle on input $((A_i, \vec{a}_i)_{i=1}^\ell, \text{aux})$ to obtain σ_α and outputs (α, σ_α) .

When $\mathcal{A}$ queries $(\phi_1, \pi, \mathbf{w}_2)$, $\mathcal{B}_1$ parses $((\mathbf{M}_j)_{j=1}^t, (S_i, c_i)_{i=1}^q, \phi_t, T) \leftarrow \phi_1$ and returns $((T, \phi_t, \beta), \sigma_\beta)$. Because $1 = \text{Sign.Verify}((T, \phi_1, \beta), \sigma_\beta)$ but Ext_1 aborts, $\mathcal{B}_1$ never queried the signing oracle on (T, ϕ_1, β) , $\mathcal{B}_1$ wins. Thus, we have that the probability that Ext_1 aborts is less or equal than the advantage of $\mathcal{B}_1$ against unforgeability (Definition 6).

Witness is valid:

- Parse $\vec{z} \leftarrow \mathbf{w}_2$. Because $(\phi_2, \mathbf{w}_2) \in \mathcal{R}_{\text{lcCCS}}$, $\vec{z}$ is such that $T + \alpha W = \vec{z} \times \vec{g}$ and for all $j \in [t]$, $\vec{r}^\top \mathbf{M}_j \vec{z} = \rho_j + \alpha u_j$.

- Recall that $T + \alpha W = (\vec{t} + \alpha \vec{w}) \times \vec{g}$ as well. By the binding property of the commitment scheme, we have that except with negligible probability, $\vec{z} = \vec{t} + \alpha \vec{w}$. Indeed, consider an adversary $\mathcal{B}_2$ that runs $\mathcal{A}$ as a subroutine. $\mathcal{B}_2$ runs Ext_1 . When $\mathcal{A}$ finalizes, $\mathcal{B}_2$ outputs $(T, \vec{t} + \alpha \vec{w}, \vec{z})$ and breaks binding.
- Replacing in the first item, we have $\vec{r}^\top \mathbf{M}_j (\vec{t} + \alpha \vec{w}) = \rho_j + \alpha u_j$. By Schwartz- Zippel lemma we have that, except with probability $1/|\mathbb{F}|$, if $\vec{r}^\top \mathbf{M}_j (\vec{t} + \alpha \vec{w}) - \rho_j + \alpha u_j = 0$ for a random α , then $\vec{r}^\top \mathbf{M}_j \vec{t} = \rho_j$ and $\vec{r}^\top \mathbf{M}_j \vec{w} = \alpha u_j = 0$. If q_H is the number of queries $\mathcal{A}$ can make to the oracle, we have the two last equalities hold except with probability $q_H/|\mathbb{F}|$.
- Because ReduceInstance does not abort, $\vec{r} = \vec{r}_2$ and $\vec{r}_2^\top \mathbf{M}_j \vec{w} = u_j$ for all $j \in [t]$, meaning $\vec{w}$ is a valid witness for $\phi_{1,b}$ in $\mathcal{R}_{lcCS}$.
- Last, we argue that since $\vec{r}^\top \mathbf{M}_j \vec{t} = v_j$, we also have that $\vec{r}_1^\top \mathbf{M}_j \vec{t} = v_j$ with overwhelming probability. Indeed, if it is not true, we can build an adversary $\mathcal{B}_3$ against knowledge soundness of the reduction Π_{rerand} . $\mathcal{B}_3$ samples pp at random and runs $\mathcal{A}(\text{pp})$. When $\mathcal{A}$ outputs $((\phi_{1,a}, \phi_{1,b}), \pi, \mathbf{w}_2)$, $\mathcal{B}_3$ parses $\pi = (\alpha, \sigma_\alpha, \pi_{\text{rerand}})$ and outputs $(\phi_{1,a}, \vec{t}, \pi_{\text{rerand}})$. Since $(\phi_{1,b}, \vec{t}) \in \mathcal{R}_{lcCS}$ and $\perp \neq \phi_{1,b} = \Pi_{\text{rerand}} \cdot \text{ReduceInstance}(\phi_{1,b}, \pi_{\text{rerand}})$ but $(\phi_{1,a}, \vec{t}) \notin \mathcal{R}_{lcCS}$, $\mathcal{A}$ wins the game.

Then, $\text{Adv}_{\mathcal{A}, \text{Ext}_3}^{\text{rok}}(n) \leq \text{Adv}_{\mathcal{B}_1}^{\text{unf}}(n) + \text{Adv}_{\mathcal{B}_2}^{\text{bind}}(n) + q_H/|\mathbb{F}| + \text{Adv}_{\mathcal{B}_2}^{\text{bind}}(n) + \text{Adv}_{\mathcal{B}_3}^{\text{rok}}(n)$. $\square$

C.3 Reduction $\Pi_{\text{rerand}} : \mathcal{R}_{lcCS} \rightarrow \mathcal{R}_{lcCS}$

Theorem 9. *Protocol Π_{rerand} in Fig. 13 is a reduction of knowledge from $\mathcal{R}_{lcCS}$ to $\mathcal{R}_{lcCS}$*

Proof. Let $\mathcal{A}$ be an adversary against the security game of Fig 2. On input pp , $\mathcal{A}$ outputs $(\phi_{1,a}, \pi, \mathbf{w}_{1,b})$. We will construct an extractor Ext_2 that on input $(\text{pp}, \phi_{1,a}, \phi_{1,b}, \pi, \mathbf{w}_{1,b})$ where $\phi_{1,b} \leftarrow \text{ReduceInstance}(\text{pp}, \phi_{1,a}, \pi)$, outputs a witness $\mathbf{w}_{1,b}$. Adversary wins if (a) $(\phi_{1,a}, \mathbf{w}_{1,a}) \notin \mathcal{R}_{lcCS}$, (b) $\phi_{1,b} \neq \perp$, and (c) $(\phi_{1,b}, \mathbf{w}_{1,b}) \in \mathcal{R}_{lcCS}$. We assume (b) and (c) hold and prove that for $\mathbf{w}_{1,b}$ outputted by Ext_2 , $(\phi_{1,b}, \mathbf{w}_{1,b}) \in \mathcal{R}_{lcCS}$ except with negligible probability.

Ext_2 parses $\vec{t} \leftarrow \mathbf{w}_{1,b}$ and outputs $\mathbf{w}_{1,a} = \vec{t}$. It is trivial that Ext_2 does not abort.

Outputs valid witness:

- Because $\vec{t}$ is a witness for $\phi_{1,b}$ in $\mathcal{R}_{lcCS}$ we have that for all $j \in [t]$, $\rho_j = \vec{r}_2^\top \mathbf{M}_j \vec{z} = \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{r}_2, \vec{y}) z(\vec{y})$, for $\vec{z} = \phi_t || \vec{t}$.
- Because ReduceInstance does not abort, it holds that

$$\text{sum}_m = G(\vec{r}) = \sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{r}_2) L_j(\vec{r}_2)$$

- Also, ReduceInstance checks that $\text{sum}_m = \sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{r}_2) \rho_j$. Replacing sum_m by the equality in the previous item, and the ρ_j s by the equalities in the first one, we get:

$$\sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{r}_2) L_j(\vec{r}_2) = \sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{r}_2) \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{r}_2, \vec{y}) z(\vec{y})$$

- $\vec{r}_2$ is sampled after the prover sends $L_j(\vec{X})$ and thus, by the Schwartz-Zippel lemma, except with probability $q_H * s/|\mathbb{F}|$:

$$\sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{X}) L_j(\vec{X}) = \sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{X}) \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{X}, \vec{y}) z(\vec{y})$$

- Finally, since ReduceInstance checks $G(\vec{r})$, the soundness of the sumcheck protocol implies that except with probability $\epsilon_{\text{sc}}(n)$:

$$\begin{aligned} \sum_{j=1}^t \gamma^{j-1} v_j &= \sum_{j=1}^t \sum_{\vec{x} \in \{0,1\}^s} \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{x}) L_j(\vec{x}) \\ &= \sum_{j=1}^t \sum_{\vec{x} \in \{0,1\}^s} \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{x}) \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{x}, \vec{y}) z(\vec{y}) \\ &= \sum_{j=1}^t \gamma^{j-1} \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{r}_1, \vec{y}) z(\vec{y}) \\ &= \sum_{j=1}^t \gamma^{j-1} (\vec{r}_1^\top \mathbf{M}_j \vec{z}) \end{aligned}$$


```


$$\frac{\Pi_{\text{rerand}}.\text{Reduce}(\text{pp}, \phi_1, \mathbf{w}_1) \mapsto (\phi_2, \mathbf{w}_2, \pi)}{\text{parse } (((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}_1, \phi_t, T), \vec{t}) \leftarrow (\phi_1, \mathbf{w}_1)}$$


$$\vec{z} \leftarrow (\phi_t \parallel \vec{t})$$

For  $j \in [t]$ :

$$L_j(\vec{X}) \leftarrow \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{X}, \vec{y}) z_1(\vec{y})$$


$$(\gamma, \sigma_\gamma) \leftarrow \text{osRO}((T, \vec{t}), \text{aux} = \phi_1) \quad \# \text{ osRO query}$$


$$\phi_0 \leftarrow (s, \sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{X}) L_j(\vec{X}), \{0, 1\}, \sum_{j=1}^t \gamma^{j-1} v_j)$$


$$(\phi_m, \perp, \pi_{\text{sc}}) \leftarrow \Pi_{\text{red-sc}}.\text{Reduce}(\text{pp}, \phi_0, \perp)$$

parse  $(m, m, G, \vec{r}_2, S, \text{sum}_m) \leftarrow \phi_m$ 
for  $j \in [t]$ :

$$\rho_j \leftarrow \sum_{\vec{y} \in \{0,1\}^{s'}} M_j(\vec{r}_2, \vec{y}) z_1(\vec{y})$$


$$\pi \leftarrow (\gamma, \sigma_\gamma, \pi_{\text{sc}}, \{L_j(X), \rho_j\}_{j=1}^t)$$


$$(\phi_2, \mathbf{w}_2) \leftarrow (((\mathbf{M}_j, \rho_j)_{j=1}^t, \vec{r}_2, \phi_t, T), \vec{t})$$

return  $(\phi_2, \mathbf{w}_2, \pi)$ 


$$\Pi_{\text{rerand}}.\text{ReduceInstance}(\text{pp}, \text{vk}, \phi_1, \pi) \mapsto \phi_2$$

parse  $((\mathbf{M}_j, v_j)_{j=1}^t, \vec{r}_1, \phi_t, T) \leftarrow \phi_1$ 
parse  $(\gamma, \sigma_\gamma, \pi_{\text{sc}}, \{L_j(X), \rho_j\}_{j=1}^t) \leftarrow \pi$ 
assert  $\text{Sign.Verify}(\text{vk}, (T, \phi_1, \gamma), \sigma_\gamma) \quad \# \text{ osRO check}$ 

$$\phi_0 \leftarrow (s, \sum_{j=1}^t \gamma^{j-1} \text{eq}(\vec{r}_1, \vec{X}) L_j(\vec{X}), \{0, 1\}, \sum_{j=1}^t \gamma^{j-1} v_j)$$


$$\phi_m \leftarrow \Pi_{\text{red-sc}}.\text{ReduceInstance}(\perp, \phi_0, \pi_{\text{sc}})$$

parse  $(m, m, G, \vec{r}_2, S, \text{sum}_m) \leftarrow \phi_m$ 
assert  $G(\vec{r}_2) = \text{sum}_m$ 

$$e \leftarrow \text{eq}(\vec{r}_1, \vec{r}_2)$$

assert  $e \sum_{j=1}^t \gamma^{j-1} \rho_j = \text{sum}_m$ 

$$\phi_2 \leftarrow ((\mathbf{M}_j, \rho_j)_{j=1}^t, \vec{r}_2, \phi_t, T)$$

return  $\phi_2$ 

```

Fig. 13. Reduction of knowledge Π_{rerand} from $\mathcal{R}_{\text{tccs}}$ to itself.

- Finally, the probability that $\sum_{j=1}^t \gamma^{j-1} v_j - \sum_{j=1}^t \gamma^{j-1} (\vec{r}_1^\top \mathbf{M}_j \vec{z}) = 0$ but $\sum_{j=1}^t \gamma^{j-1} v_j - \sum_{j=1}^t Y^{j-1} (\vec{r}_1^\top \mathbf{M}_j \vec{z}) \neq 0$ is, for any random $\gamma \in \mathbb{F}$, $t/|F|$. Let q_H be the amount of queries $\mathcal{A}$ can make to the oracle, $v_j = \vec{r}_1^\top \mathbf{M}_j \vec{z}$ for every $j \in [t]$, and thus $\vec{t}$ is a valid witness with probability $1 - q_H \frac{t}{|F|}$. We conclude that

$$\text{Adv}_{\mathcal{A}, \text{Ext}_2}^{\text{rok}}(n) \leq q_H \frac{s}{|F|} + \epsilon_{\text{sc}}(n) + q_H \frac{t}{|F|}.$$

□

D Deferred proofs for Sections 5 and 6

D.1 Proof of Theorem 2

Completeness: We need to show that if the **Setup** and $\mathcal{P}$ algorithms are run honestly, the verifier accepts. That is, that for $i \geq 1$, $\pi_{\text{step}, i+1} \leftarrow \mathcal{P}(\text{pp}, \text{vk}, \mathbb{F}, (i, z_0, z_i), w_i, \pi_{\text{step}, i})$, contains valid instance-witness pairs of $\mathcal{R}_{\text{prime}}$ and $\mathcal{R}_{\text{acc}}$. Assume that for $((\phi, w)_{\text{prime}, i}, (\phi, w)_{\text{acc}, i}) \leftarrow \pi_{\text{step}, i}$, $(\phi, w)_{\text{prime}, i} \in \mathcal{R}_{\text{prime}, i}$ and $(\phi, w)_{\text{acc}, i} \in \mathcal{R}_{\text{acc}}$. Since Π_{acc} is a NIroK that satisfies completeness, $(\phi, w)_{\text{acc}, i+1} \in \mathcal{R}_{\text{acc}}$. Then, $(\phi, w)_{\text{arith}, i+1} \in \mathcal{R}_{\text{arith}}$ by construction. Since Π_{prime} is also a NIroK, $(\phi, w)_{\text{inter}, i+1} \in \mathcal{R}_{\text{inter}}$ and π_{prime} is a valid proof of the reduction. Finally, by construction we have $(\phi, w)_{\text{prime}, i+1} \in \mathcal{R}_{\text{prime}}$, which concludes the proof. □

D.2 Proof of Theorem 3

The proof follows the intuition presented in Section 6.

Proof. (IVC Security) We prove that the IVC scheme defined in Section 6 is complete and knowledge-sound assuming that:

- $\Pi_{\text{prime}}^{(\text{ying})}$, $\Pi_{\text{prime}}^{(\text{yang})}$, $\Pi_{\text{acc}}^{(\text{ying})}$ and $\Pi_{\text{acc}}^{(\text{yang})}$ are NIroKs in the osROM, and
- $H^{(\text{ying})}$, $H^{(\text{yang})}$ are collision-resistant hash functions.

Completeness. Completeness follows from the completeness of the underlying NIroKs. The $\mathcal{P}$ algorithm is designed to correctly transfer values across half-cycles (as verified by the hash functions and circuit constraints) and runs the honest Reduce algorithms of NIroKs. Therefore, given an accepting proof — one where the hash checks pass and all proof items are in their corresponding relations — the prover will output accepting instance-witness pairs. Furthermore, since the honest prover uses the values (i, z_0, z_i) from the current accepting statement, the hash checks at step $(i+1)$ will also be satisfied.

Knowledge-soundness. The step extractor over cycles. We now proceed to formally described the step extractor. Let:

- $\text{Ext}_{\text{prime}}^{(\text{yang})}$ denote the extractor for $\Pi_{\text{prime}}^{(\text{yang})}$,
- $\text{Ext}_{\text{acc}}^{(\text{ying})}$ denote the extractor for $\Pi_{\text{acc}}^{(\text{ying})}$,
- $\text{Ext}_{\text{prime}}^{(\text{ying})}$ denote the extractor for $\Pi_{\text{prime}}^{(\text{ying})}$,
- $\text{Ext}_{\text{acc}}^{(\text{yang})}$ denote the extractor for $\Pi_{\text{acc}}^{(\text{yang})}$.

For all $i > 1$, the single-step extractor Ext_{step} proceeds as follows:

1. Parse π_i as $(\phi, w)_{\text{prime}, i}^{(\text{yang})}$, $(\phi, w)_{\text{acc}, i}^{(\text{ying})}$ and $(\phi, w)_{\text{acc}, i}^{(\text{yang})}$.
2. (Abort condition 1) Abort if $\mathcal{V}$ rejects π_i .
3. Run $\text{Ext}_{\text{prime}}^{(\text{yang})}$ to obtain $(\phi, w)_{\text{arith}, i}^{(\text{yang})}$ as follows:
 - Parse $\phi_{\text{prime}, i}^{(\text{yang})}$ as $(\phi_{\text{arith}, i}^{(\text{yang})}, \phi_{\text{inter}, i}^{(\text{yang})}, \pi_{\text{prime}, i}^{(\text{yang})})$.
 - Run $\text{Ext}_{\text{prime}}^{(\text{yang})}$

$$w_{\text{arith}, i}^{(\text{yang})} \leftarrow \text{Ext}_{\text{prime}}^{(\text{yang})}(\text{pp}, \phi_{\text{arith}, i}^{(\text{yang})}, \phi_{\text{inter}, i}^{(\text{yang})}, \pi_{\text{prime}, i}^{(\text{yang})}, w_{\text{prime}, i}^{(\text{yang})}, Q)$$

4. (Abort condition 2) Abort if $(\phi, w)_{\text{arith},i}^{(\text{yang})} \notin \mathcal{R}_{\text{arith}}^{(\text{yang})}$.
 5. Parse $w_{\text{arith},i}^{(\text{yang})}$ as

$$w_{\text{arith},i}^{(\text{yang})} = \left(\text{pp}^*, \text{vk}^*, i^*, \begin{pmatrix} \phi_{\text{arith},i}^{(\text{ying})}, \phi_{\text{inter},i}^{(\text{ying})}, \phi_{\text{acc},i-1}^{(\text{ying})}, \phi_{\text{acc},i}^*^{(\text{ying})}, \\ \pi_{\text{prime},i}^{(\text{ying})}, \pi_{\text{acc}}^{(\text{ying})} \end{pmatrix} \right)$$

6. (Abort condition 3) Abort if the extracted values do not match their equivalent values in the state-
 ment and proof:

$$(\text{pp}^*, \text{vk}^*, i^*, \phi_{\text{acc},i}^*^{(\text{ying})}) \neq (\text{pp}, \text{vk}, i, \phi_{\text{acc},i}^{(\text{ying})})$$

7. Set:

$$\phi_{\text{prime},i}^{(\text{ying})} = (\phi_{\text{arith},i}^{(\text{ying})}, \phi_{\text{inter},i}^{(\text{ying})}, \pi_{\text{prime},i}^{(\text{ying})})$$

8. Run $\text{Ext}_{\text{acc}}^{(\text{ying})}$ to obtain $(w_{\text{prime},i}^{(\text{ying})}, w_{\text{acc},i-1}^{(\text{ying})})$ as follows:

$$(w_{\text{prime},i}^{(\text{ying})}, w_{\text{acc},i-1}^{(\text{ying})}) \leftarrow \text{Ext}_{\text{acc}}^{(\text{ying})} \left(\text{pp}, (\phi_{\text{prime},i}^{(\text{ying})}, \phi_{\text{acc},i-1}^{(\text{ying})}), \begin{pmatrix} \phi_{\text{acc},i}^{(\text{ying})}, \pi_{\text{acc},i}^{(\text{ying})}, w_{\text{acc},i}^{(\text{ying})}, Q \end{pmatrix} \right)$$

9. (Abort condition 4) Abort if:

$$((\phi, w)_{\text{prime},i}^{(\text{ying})}, (\phi, w)_{\text{acc},i-1}^{(\text{ying})}) \notin (\mathcal{R}_{\text{prime}}^{(\text{ying})} \times \mathcal{R}_{\text{acc}}^{(\text{ying})})$$

10. Run $\text{Ext}_{\text{prime}}^{(\text{ying})}$ to obtain $w_{\text{arith},i}^{(\text{ying})}$ as follows:

$$w_{\text{arith},i}^{(\text{ying})} \leftarrow \text{Ext}_{\text{prime}}^{(\text{ying})} (\text{pp}, \phi_{\text{arith},i}^{(\text{ying})}, \phi_{\text{inter},i}^{(\text{ying})}, \pi_{\text{prime},i}^{(\text{ying})}, w_{\text{prime},i}^{(\text{ying})}, Q)$$

11. (Abort condition 5) Abort if $(\phi, w)_{\text{arith},i}^{(\text{ying})} \notin \mathcal{R}_{\text{arith}}^{(\text{ying})}$.
 12. Parse $w_{\text{arith},i}^{(\text{ying})}$ as:

$$w_{\text{arith},i}^{(\text{ying})} = \left(\text{pp}^\diamond, \text{vk}^\diamond, i^\diamond, z_0^\diamond, z_{i-1}^\diamond, z_i^\diamond, \omega_{i-1}, \begin{pmatrix} \phi_{\text{arith},i-1}^{(\text{yang})}, \phi_{\text{inter},i-1}^{(\text{yang})}, \phi_{\text{acc},i-1}^{(\text{yang})}, \phi_{\text{acc},i}^\diamond^{(\text{yang})}, \\ \pi_{\text{prime},i-1}^{(\text{yang})}, \pi_{\text{acc}}^{(\text{yang})} \end{pmatrix} \right)$$

13. (Abort condition 6) Abort if the extracted values do not match their equivalent values in the state-
 ment and proof:

$$(\text{pp}^\diamond, \text{vk}^\diamond, i^\diamond, z_0^\diamond, z_i^\diamond, \phi_{\text{acc},i}^\diamond^{(\text{yang})}) \neq (\text{pp}, \text{vk}, i, z_0, z_i, \phi_{\text{acc},i}^{(\text{yang})})$$

14. Set:

$$\phi_{\text{prime},i-1}^{(\text{yang})} = (\phi_{\text{arith},i-1}^{(\text{yang})}, \phi_{\text{inter},i-1}^{(\text{yang})}, \pi_{\text{prime},i-1}^{(\text{yang})})$$

15. Run $\text{Ext}_{\text{acc}}^{(\text{yang})}$ to obtain $(w_{\text{prime},i-1}^{(\text{yang})}, w_{\text{acc},i-1}^{(\text{yang})})$ as follows:
 16.

$$(w_{\text{prime},i-1}^{(\text{yang})}, w_{\text{acc},i-1}^{(\text{yang})}) \leftarrow \text{Ext}_{\text{acc}}^{(\text{yang})} \left(\text{pp}, (\phi_{\text{prime},i-1}^{(\text{yang})}, \phi_{\text{acc},i-1}^{(\text{yang})}), \begin{pmatrix} \phi_{\text{acc},i}^{(\text{yang})}, \pi_{\text{acc}}^{(\text{yang})}, w_{\text{acc},i}^{(\text{yang})}, Q \end{pmatrix} \right)$$

17. (Abort condition 7) Abort if:

$$((\phi, w)_{\text{prime},i-1}^{(\text{yang})}, (\phi, w)_{\text{acc},i-1}^{(\text{yang})}) \notin (\mathcal{R}_{\text{prime}}^{(\text{yang})} \times \mathcal{R}_{\text{acc}}^{(\text{yang})})$$

18. Output $(z_{i-1}, \omega_{i-1}, \pi_{i-1})$ where z_{i-1} and ω_{i-1} are obtained in step 12, and π_{i-1} is defined as below:

$$\pi_{i-1} = \left[(\phi, w)_{\text{prime},i-1}^{(\text{yang})}, (\phi, w)_{\text{acc},i-1}^{(\text{yang})}, (\phi, w)_{\text{acc},i-1}^{(\text{yang})} \right]$$

Base case. If $i = 1$, the single-step extractor Ext_{step} follows the steps above until Step 13 (abort condition 6), and outputs (z'_0, ω_0) .

Claim. For all $i > 1$, let **Abort** denote the event that Ext_{step} aborts in the knowledge soundness experiment and $\mathcal{V}(\text{pp}, \text{vk}, \text{pp}, F, (i, z_0, z_i), \pi_i) = 1$ (see Definition 4), then

$$\Pr[\text{Abort}] \leq \text{negl}(n)$$

Proof. We give an upper bound for $\Pr[\text{Abort}]$ by inductively applying the law of total probability. Let **Abort**₁ denote the event “ Ext_{step} aborts at condition 1” and $\overline{\text{Abort}}_1$ denote its conjugate event. Then, by the law of total probability $\Pr[\text{Abort}] = \Pr[\text{Abort} \wedge \text{Abort}_1] + \Pr[\text{Abort} \wedge \overline{\text{Abort}}_1]$. We evaluate $\Pr[\text{Abort} \wedge \text{Abort}_1]$ directly and inductively evaluate $\Pr[\text{Abort} \wedge \overline{\text{Abort}}_1]$ by defining an event **Abort**₂ corresponding to the second abort condition and applying the law of total probability once again. We repeat the process until exhausting all abort conditions.

(Condition 1). The probability of **Abort** and **Abort**₁ is null. Indeed, the events $\mathcal{V}(\text{pp}, \text{vk}, \text{pp}, F, (i, z_0, z_i), \pi_i) = 1$ and **Abort**₁ are disjoint.

(Condition 2). If abort condition 1 does not occur, then $\pi_i \in (\mathcal{R}_{\text{prime}}^{(\text{yang})} \times \mathcal{R}_{\text{acc}}^{(\text{ying})} \times \mathcal{R}_{\text{acc}}^{(\text{yang})})$. In particular, it implies that

$$\begin{cases} \phi_{\text{inter},i}^{(\text{yang})} = \Pi_{\text{prime}}^{(\text{yang})}.\text{ReduceInstance}(\text{pp}, \phi_{\text{arith},i}^{(\text{yang})}, \pi_{\text{prime}}^{(\text{yang})}) \\ (\phi, \mathbf{w})_{\text{inter},i}^{(\text{yang})} \in \mathcal{R}_{\text{inter}}^{(\text{yang})} \end{cases}$$

Let $\mathcal{B}_2$ denote an adversary against the knowledge soundness property of the NIROK $\Pi_{\text{prime}}^{(\text{yang})}$ that runs $(\mathcal{F}, P^*)$ internally, forwards all calls to the open-and-sign oracle $\mathcal{O}$ and outputs $\phi_{\text{arith},i}^{(\text{yang})}, (\phi, \mathbf{w})_{\text{inter},i}^{(\text{yang})}, \pi_{\text{prime}}^{(\text{yang})}$. By definition of knowledge soundness of a NIROK, the probability that $\text{Ext}_{\text{prime}}^{(\text{yang})}$ outputs a witness $\mathbf{w}_{\text{arith},i}^{(\text{yang})}$ such that $(\phi, \mathbf{w})_{\text{arith},i}^{(\text{yang})} \notin \mathcal{R}_{\text{arith}}^{(\text{yang})}$ is $\text{Adv}_{\text{Ext}_{\text{prime}}^{(\text{yang})}, \mathcal{B}_2}^{\text{rok}}(n)$.

(Condition 3). Assume that abort conditions 1-2 did not occur. Therefore it holds from the verifier checks that $\phi_{\text{arith},i}^{(\text{yang})}.h_1 = H^{(\text{yang})}(\text{pp}, \text{vk}, i, \phi_{\text{acc},i}^{(\text{ying})})$. Furthermore it holds that $(\phi, \mathbf{w})_{\text{arith},i}^{(\text{yang})}$ is a satisfying assignment to the circuit $\mathcal{C}_{\text{step}}^{(\text{yang})}$. In particular, it holds that $\phi_{\text{arith},i}^{(\text{yang})}.h_1 = H^{(\text{yang})}(\text{pp}^*, \text{vk}^*, i^*, \phi_{\text{acc},i}^{(\text{ying})})$. If $(\text{pp}, \text{vk}, i, \phi_{\text{acc},i}^{(\text{ying})}) \neq (\text{pp}^*, \text{vk}^*, i^*, \phi_{\text{acc},i}^{(\text{ying})})$, then we can construct an explicit collision finder $\mathcal{B}_3$ for the hash function $H^{(\text{yang})}$. Therefore the probability of aborting on condition 3 knowing that Ext_{step} did not abort on conditions 1-2 is $\text{Adv}_{H^{(\text{yang})}, \mathcal{B}_3}^{\text{collision-resistance}}(n)$.

(Condition 4). Assume that abort conditions 1-3 did not occur. Therefore it holds that $(\phi, \mathbf{w})_{\text{arith},i}^{(\text{yang})}$ is a satisfying assignment to the circuit $\mathcal{C}_{\text{step}}^{(\text{yang})}$. In particular, it holds that

$$\phi_{\text{acc},i}^{(\text{ying})} = \Pi_{\text{acc}}^{(\text{ying})}.\text{ReduceInstance}(\text{pp}, (\phi_{\text{prime},i}^{(\text{ying})}, \phi_{\text{acc},i-1}^{(\text{ying})}), \pi_{\text{acc}}^{(\text{ying})})$$

Furthermore, it holds that $(\phi, \mathbf{w})_{\text{acc},i}^{(\text{ying})} \in \mathcal{R}_{\text{acc}}^{(\text{ying})}$ as per the verifier's checks. Similarly to abort condition 2 above, the probability that $\text{Ext}_{\text{acc}}^{(\text{ying})}$ fails to output satisfying witnesses for $(\phi_{\text{prime},i}^{(\text{ying})}, \phi_{\text{acc},i}^{(\text{ying})})$ corresponds to the advantage of an explicit adversary $\mathcal{B}_4$ in the NIROK knowledge-soundness game. Therefore, the probability that Ext_{step} aborts on condition 4 knowing that it did not abort on conditions 1-3 is $\text{Adv}_{\text{Ext}_{\text{acc}}^{(\text{ying})}, \mathcal{B}_4}^{\text{rok}}(n)$.

(Condition 5). Assume that abort conditions 1-4 did not occur. Then $(\phi, \mathbf{w})_{\text{prime},i}^{(\text{ying})} \in \mathcal{R}_{\text{prime}}^{(\text{ying})}$. By definition of $\mathcal{R}_{\text{prime}}^{(\text{ying})}$, we know that

$$\begin{cases} \phi_{\text{inter},i}^{(\text{yang})} = \Pi_{\text{prime}}^{(\text{yang})}.\text{ReduceInstance}(\text{pp}, \phi_{\text{arith},i}^{(\text{yang})}, \pi_{\text{prime}}^{(\text{yang})}) \\ (\phi, \mathbf{w})_{\text{inter},i}^{(\text{yang})} \in \mathcal{R}_{\text{inter}}^{(\text{yang})} \end{cases}$$

Let $\mathcal{B}_5$ denote an adversary against the knowledge soundness property of the NIRoK $\Pi_{\text{prime}}^{(\text{ying})}$ that runs $(\mathcal{F}, P^*)$ internally, forwards all calls to the open-and-sign oracle $\mathcal{O}$ and outputs $\phi_{\text{arith},i}^{(\text{ying})}, (\phi, \mathbf{w})_{\text{inter},i}^{(\text{ying})}, \pi_{\text{prime}}^{(\text{ying})}$. By definition of knowledge soundness of a NIRoK, the probability that $\text{Ext}_{\text{prime}}^{(\text{ying})}$ outputs a witness $\mathbf{w}_{\text{arith},i}^{(\text{ying})}$ such that $(\phi, \mathbf{w})_{\text{arith},i}^{(\text{ying})} \notin \mathcal{R}_{\text{arith}}^{(\text{ying})}$ is $\text{Adv}_{\text{Ext}_{\text{prime}}^{(\text{ying})}, \mathcal{B}_5}^{\text{rok}}(n)$.

(Condition 6). Assume that abort conditions 1-5 did not occur. Firstly, it holds from the verifier checks that $\phi_{\text{arith},i}^{(\text{yang})}.h_0 = H^{(\text{ying})}(\text{pp}, \text{vk}, i, z_0, z_i, \phi_{\text{acc},i}^{(\text{yang})})$. Secondly, since $(\phi, \mathbf{w})_{\text{arith},i}^{(\text{yang})}$ is a satisfying assignment to $\mathcal{C}_{\text{step}}^{(\text{yang})}$, it holds that $\phi_{\text{arith},i}^{(\text{yang})}.h_0 = \phi_{\text{arith},i}^{(\text{ying})}.h_1$. Furthermore it holds that $(\phi, \mathbf{w})_{\text{arith},i}^{(\text{ying})}$ is a satisfying assignment to the circuit $\mathcal{C}_{\text{step}}^{(\text{ying})}$. In particular, it holds that

$$\begin{aligned} \phi_{\text{arith},i}^{(\text{ying})}.h_1 &= H^{(\text{ying})}(\text{pp}^\diamond, \text{vk}^\diamond, i^\diamond, z_0^\diamond, z_i^\diamond, \phi_{\text{acc},i}^\diamond)^{(\text{yang})} \\ &= \phi_{\text{arith},i}^{(\text{yang})}.h_0 \end{aligned}$$

If $(\text{pp}, \text{vk}, i, z_0, z_i, \phi_{\text{acc},i}^{(\text{yang})}) \neq (\text{pp}^\diamond, \text{vk}^\diamond, i^\diamond, z_0^\diamond, z_i^\diamond, \phi_{\text{acc},i}^\diamond)^{(\text{yang})}$, then we can construct an explicit collision finder $\mathcal{B}_6$ for the hash function $H^{(\text{ying})}$. Therefore the probability of aborting on condition 6 knowing that Ext_{step} did not abort on conditions 1-5 is $\text{Adv}_{H^{(\text{ying})}, \mathcal{B}_6}^{\text{collision-resistance}}(n)$.

(Condition 7). Assume that abort conditions 1-6 did not occur. Therefore it holds that $(\phi, \mathbf{w})_{\text{arith},i}^{(\text{ying})}$ is a satisfying assignment to the circuit $\mathcal{C}_{\text{step}}^{(\text{ying})}$. In particular, it holds that

$$\phi_{\text{acc},i}^{(\text{yang})} = \Pi_{\text{acc}}^{(\text{ying})}.\text{ReduceInstance}(\text{pp}, (\phi_{\text{prime},i-1})^{(\text{yang})}, \phi_{\text{acc},i-1}^{(\text{yang})}, \pi_{\text{acc}}^{(\text{ying})})$$

Furthermore, it holds that $(\phi, \mathbf{w})_{\text{acc},i}^{(\text{yang})} \in \mathcal{R}_{\text{acc}}^{(\text{yang})}$ as per the verifier's checks. Similarly to abort condition 4 above, the probability that $\text{Ext}_{\text{acc}}^{(\text{yang})}$ fails to output satisfying witnesses for $(\phi_{\text{prime},i-1})^{(\text{yang})}, \phi_{\text{acc},i-1}^{(\text{yang})}$ corresponds to the advantage of an explicit adversary $\mathcal{B}_7$ in the NIRoK knowledge-soundness game. Therefore, the probability that Ext_{step} aborts on condition 7 knowing that it did not abort on conditions 1-6 is $\text{Adv}_{\text{Ext}_{\text{acc}}^{(\text{yang})}, \mathcal{B}_7}^{\text{rok}}(n)$.

(Final bound). Putting everything together, we find that

$$\begin{aligned} \Pr[\text{Abort}] &\leq \text{Adv}_{\text{Ext}_{\text{prime}}^{(\text{yang})}, \mathcal{B}_2}^{\text{rok}}(n) + \text{Adv}_{H^{(\text{yang})}, \mathcal{B}_3}^{\text{collision-resistance}}(n) \\ &\quad + \text{Adv}_{\text{Ext}_{\text{acc}}^{(\text{ying})}, \mathcal{B}_4}^{\text{rok}}(n) + \text{Adv}_{\text{Ext}_{\text{prime}}^{(\text{ying})}, \mathcal{B}_5}^{\text{rok}}(n) \\ &\quad + \text{Adv}_{H^{(\text{ying})}, \mathcal{B}_6}^{\text{collision-resistance}}(n) + \text{Adv}_{\text{Ext}_{\text{acc}}^{(\text{yang})}, \mathcal{B}_7}^{\text{rok}}(n) \end{aligned}$$

If all 4 NIRoKs have negligible knowledge soundness error and the hash functions $H^{(\text{ying})}$ and $H^{(\text{yang})}$ are collision-resistant, then all of the above terms are negligible in n . We have therefore shown that

$$\Pr[\text{Abort}] \leq \text{negl}(n)$$

Claim. Let $\text{Abort}_{\text{base}}$ denote the event that Ext_{step} aborts in the knowledge soundness experiment and $\mathcal{V}(\text{pp}, \text{vk}, \text{pp}, F, (1, z_0, z_1), \pi_1) = 1$ (see Definition 4), then

$$\Pr[\text{Abort}_{\text{base}}] \leq \text{negl}(n)$$

Proof. The reasoning follows as in the case $i > 1$ with the minor change that the extractor procedure is shorter and, in particular, is not concerned with running $\text{Ext}_{\text{acc}}^{(\text{yang})}$. Therefore it holds that

$$\begin{aligned} \Pr[\text{Abort}] &\leq \text{Adv}_{\text{Ext}_{\text{prime}}^{(\text{yang})}, \mathcal{B}_2}^{\text{rok}}(n) + \text{Adv}_{H^{(\text{yang})}, \mathcal{B}_3}^{\text{collision-resistance}}(n) \\ &\quad + \text{Adv}_{\text{Ext}_{\text{acc}}^{(\text{ying})}, \mathcal{B}_4}^{\text{rok}}(n) + \text{Adv}_{\text{Ext}_{\text{prime}}^{(\text{ying})}, \mathcal{B}_5}^{\text{rok}}(n) \\ &\quad + \text{Adv}_{H^{(\text{ying})}, \mathcal{B}_6}^{\text{collision-resistance}}(n) \\ &\leq \text{negl}(n) \end{aligned}$$

Claim. if the extractor does not abort, it outputs $(z_{i-1}, \omega_{i-1}, \pi_{i-1})$ such that

1. $z_i = F(z_{i-1}, \omega_{i-1})$, and
2. $i = 1 \implies z_{i-1} = z_0$, and
3. $i > 1 \implies \mathcal{V}(\text{pp}, \text{vk}, \text{pp}, F, (i, z_0, z_{i-1}), \pi_{i-1}) = 1$.

Proof. We prove each item of the claim individually.

(Item 1, state transition). Since there was no abort on condition 5, it holds that $(\phi, \mathbf{w})_{\text{arith}, i}^{(\text{ying})}$ is a satisfying assignment to $\mathcal{C}_{\text{step}}^{(\text{ying})}$. In particular, it implies that $z_i^\diamond = F(z_{i-1}, \omega_{i-1})$. Since there was no abort on condition 6, it holds that $z_i = F(z_{i-1}, \omega_{i-1})$, as claimed.

(Item 2, base case). Given that the extractor did not abort on condition 5, it holds that the extracted $\mathbf{w}_{\text{arith}, 1}^{(\text{ying})}$ is such that $\mathcal{C}_{\text{step}}^{(\text{ying})}((\phi, \mathbf{w})_{\text{arith}, 1}^{(\text{ying})}) = 1$. Therefore as per constraint 4, it holds that $z_{i-1} = z_0$ as required.

(Item 3, recursive case). Firstly, since there was no abort on conditions 4 and 7, it holds that

$$\pi_{i-1} \in (\mathcal{R}_{\text{prime}}^{(\text{yang})} \times \mathcal{R}_{\text{acc}}^{(\text{ying})} \times \mathcal{R}_{\text{acc}}^{(\text{yang})})$$

We will now show that $\phi_{\text{arith}, i-1}^{(\text{yang})}$ (contained in $\phi_{\text{prime}, i-1}^{(\text{yang})}$) satisfies the conditions on h_0 and h_1 checked by the verifier. Firstly note that since the extractor did not abort on condition 5, it holds that $(\phi_{\text{prime}, i}^{(\text{ying})}, \mathbf{w}_{\text{prime}, i}^{(\text{ying})}) \in \mathcal{R}_{\text{prime}}^{(\text{ying})}$. We make use of the following facts:

1. $\phi_{\text{prime}, i-1}^{(\text{yang})}.h_0$ is checked in the i -th run of $\mathcal{C}_{\text{step}}^{(\text{ying})}$ (constraint 1).
2. $\phi_{\text{prime}, i-1}^{(\text{yang})}.h_1$ is copied to $\phi_{\text{prime}, i}^{(\text{ying})}.h_0$ in the i -th run of $\mathcal{C}_{\text{step}}^{(\text{ying})}$ (constraint 2).
3. $\phi_{\text{prime}, i}^{(\text{yang})}.h_0$ is checked in the i -th run of $\mathcal{C}_{\text{step}}^{(\text{yang})}$ (constraint 1).

(Validity of h_0 using fact 1). Since the extractor did not abort on condition 5, we know that

$$\mathcal{C}_{\text{step}}^{(\text{ying})}(\phi_{\text{arith}, i}^{(\text{ying})}, \mathbf{w}_{\text{arith}, i}^{(\text{ying})}) = 1$$

Furthermore, since the extractor did not abort on condition 6, we know that the extracted values $\text{pp}^\diamond, \text{vk}^\diamond, i^\diamond, z_0^\diamond$ are equal to those in the statement $\text{pp}, \text{vk}, i, z_0$. We can therefore conclude that

$$\phi_{\text{prime}, i-1}^{(\text{yang})}.h_0 = H^{(\text{ying})}(\text{pp}, \text{vk}, i-1, z_0, z_{i-1}, \phi_{\text{acc}, i-1}^{(\text{yang})})$$

as expected by the verifier $\mathcal{V}$.

(Validity of h_1 using facts 2 and 3). Since $(\phi_{\text{arith}, i}^{(\text{yang})}, \mathbf{w}_{\text{arith}, i}^{(\text{yang})})$ satisfy the circuit $\mathcal{C}_{\text{step}}^{(\text{yang})}$ (as above), it holds that:

$$\phi_{\text{prime}, i-1}^{(\text{yang})}.h_1 = \phi_{\text{arith}, i}^{(\text{ying})}.h_0$$

Given that the extractor did not abort on condition 2, we know that

$$\mathcal{C}_{\text{step}}^{(\text{yang})}(\phi_{\text{arith}, i}^{(\text{yang})}, \mathbf{w}_{\text{arith}, i}^{(\text{yang})}) = 1$$

Furthermore, since the extractor did not abort on condition 3, we know that the extracted values $\text{pp}^*, \text{vk}^*, i^*, z_0^*$ are equal to those in the statement $\text{pp}, \text{vk}, i, z_0$. We can therefore conclude that

$$\begin{aligned} \phi_{\text{prime}, i-1}^{(\text{yang})}.h_1 &= \phi_{\text{arith}, i}^{(\text{ying})}.h_0 \\ &= H^{(\text{ying})}(\text{pp}, \text{vk}, i-1, \phi_{\text{acc}, i-1}^{(\text{yang})}) \end{aligned}$$

as expected by the verifier $\mathcal{V}$. □